ALIŞTIRMA DEFTERİ · C++ CİLDİ

Keyifli Alıştırmalar

C++ cildi

Programlamaya ve Algoritmalara Keyifli Bir Başlangıç'ın alıştırma defteri. Hazırlık ve altı bölüm; her konuda dört basamak.

H Hazırlık · **I** İlk Adımlar · **II** Veri · **III** Zanaat · **IV** Özyineleme · **V** Çizgeler · **VI** Dinamik Programlama

Çözüm yok, ipucu var.

Yanıt sizde — ya da yan masadaki arkadaşınızda.

ISINMA
~10 dakika

Kitapçığın programını çalıştırın, bir satırını değiştirin.

ALIŞTIRMA
20–30 dakika

Aynı kalıpla yeni, küçük bir iş. `make test` "Doğru!" diyene kadar.

ZORLAYICI
45–90 dakika

Kalıbı bükün, iki kalıbı birleştirin ya da bir hatayı bulun.

KEŞİF
açık uçlu

Beklenen çıktı yok. Bir şey inşa edin, arkadaşlarınıza gösterin.

Bülent Başaran

2026 · birinci sürüm

CC BY-SA 4.0 · kod MIT

github.com/bulent2k2/cpp_ogreniyoruz

HAZIRLIK

Tarayıcıda, Hiçbir Şey Yüklemeden

Kitapçık ilk bölümde hemen tür ve taşmaya giriyor. Buradaki üç konu, o kapıdan girmeden önce elinizin alışması için: yazdırmak, okumak, hesaplamak; derleyicinin söylediğini anlamak; ikili çalışmak. Hepsi tarayıcıda, hiçbir şey yüklemeden.

Hazırlık | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Zanaat | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

KURAL

YAPAY ZEKÂ İLE ÇALIŞMA KURALI

Bugün hepinizin elinde bir model var; bu defter onu yasaklamıyor, yerini söylüyor. **Isınma ve Alıştırma** basamaklarında sormayın: o sorular kas yapmak için, kası başkası çalıştıramaz. **Zorlayıcı**'da yalnızca ipucu isteyin, kod istemeyin (“hangi veri yapısı?”, “neyi gözden kaçırıyorum?”). **Keşif**'te serbest; ama yapıştırdığınız her satırı bir arkadaşınıza açıklayabilmelisiniz, açıklayamadığınız satır sizin değildir. “Yanıt yerine” kutusu da aynı ilkenin uzantısı: önce kitapçıktaki altbölüm, sonra yapay zekâ.

SÜRE

NE KADAR SÜRER?

Basamak zaten zorluğu söylüyor; süre de ona bağlı. `ISINMA` on dakika: kitapçığın programını çalıştırıp bir satırını değiştirmek. `ALIŞTIRMA` yirmi–otuz dakika: aynı kalıpla yeni, küçük bir iş. `ZORLAYICI` kırk beş–doksan dakika: kalıbı bükmek, iki kalıbı birleştirmek ya da bir hata bulmak. `KEŞİF` açık uçlu: bir akşam da sürebilir, bir hafta da. Bu ölçünün dışına taşanların başlığında bir not var: `~2 saat` gibi. Süreyi aşmak sorun değil; hiç uğraşmadan atlamak sorun.

İKİLİ

İKİLİ ÇALIŞMA VE KOD OKUMA LİSTESİ

Sürücü–seyrüseferci düzeni Hazırlık'ta kalmasın: her bölümde iki Zorlayıcı soru `İKİLİ` imini taşıyor. Onları ikili yazın; bitince kodu değişin ve birbirinizinkini şu beş soruyla okuyun. Beşine de "evet" diyemediğiniz kod bitmemiştir.

1. Adlar anlaşılır mı? `x`, `tmp`, `a2` yerine ne demek istediğini söyleyen adlar var mı?
2. Tekrar var mı? Aynı üç satır iki yerde geçiyorsa işleve çıkmalı.
3. Sınır durumu denendi mi? Boş girdi, tek öge, en büyük değer, eksi sayı.
4. Test var mı? En azından bir `assert` ya da altın dosya; "çalıştırdım, oldu" test değildir.
5. Çıktı biçimi doğru mu? Boşluk, satır sonu, sıra; `make test` sizin yerinize bakar.

BİÇİM

BAŞKA BİÇİMLERDE

Bu defterin tamamı (C++ cildi) tek dosya olarak da iner:

- **PDF** — A4'e basılmak üzere düzenlendi. Kâğıtta tıklanacak bir şey olmadığı için üç kademeli ipuçları açık basılır: takılmadan okumayın, bir kâğıtla örtün.
- **EPUB** — telefon, tablet ve e-kitap okuyucular için; ipuçları orada da açık kutular.

H.1

Yazdır, oku, hesapla

Bir program dünyayla üç yoldan konuşur: ekrana yazar, klavyeden okur, arada hesap yapar. Bu beş alıştırma başka hiçbir şey yapmıyor. Hepsini OnlineGDB'de çözebilirsiniz; girdi isteyenler için sayfanın altındaki *Input* kutusunu kullanın.

`2` `2` `1`

YANIT YERİNE

ders 1-4 kitapçık: Tezgâhınızı kurun kopya sayfaları

ISINMA H-1-I1 İlk satır, ikinci satır

Aşağıdaki program tek satır yazıyor. Çalıştırın. Sonra ikinci bir satır daha yazsın: `Ben bir programım.` İki ayrı `cout` satırı mı, tek satırda iki `endl` mi — ikisini de deneyin, çıktı aynı mı?

ana.cpp — BAŞLANGIÇ

```cpp
// H-1-I1 · İlk satır, ikinci satır
// İkinci bir satır ekleyin: Ben bir programım.
#include <iostream>
using namespace std;

int main() {
  cout << "Merhaba!" << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
Merhaba!
Ben bir programım.
```

Dizin: `alistirma/kod/H-1-I1/`

ISINMA H-1-I2 Bölme üç türlü

Programı önce olduğu gibi çalıştırın ve üç satırı dikkatle okuyun: `7 / 2` neden `3`? `7 % 2` ne demek? Sonra `7`'leri `9` yapın. Beklenen çıktı aşağıda; `9.0 / 2` satırına dikkat.

ana.cpp — BAŞLANGIÇ

```cpp
// H-1-I2 · Bölme üç türlü
// Çalıştırın, sonra 7'leri 9 yapın.
#include <iostream>
using namespace std;

int main() {
  cout << "7 / 2 = "   << 7 / 2   << endl;   // tam bölme
  cout << "7 % 2 = "   << 7 % 2   << endl;   // kalan
  cout << "7.0 / 2 = " << 7.0 / 2 << endl;   // kesirli bölme
  return 0;
}
```

çıktı — BEKLENEN-ÇIKTI

```
9 / 2 = 4
9 % 2 = 1
9.0 / 2 = 4.5
```

Dizin: `alistirma/kod/H-1-I2/`

ALIŞTIRMA H-1-A1 İki sayı, üç sonuç

Klavyeden iki tam sayı okuyun; toplamını, farkını ve çarpımını üç satırda yazın. Girdi `12 5` olsun. `cin >> a >> b;` iki sayıyı art arda okur, aralarındaki boşluğu kendisi atlar.

çıktı — BEKLENEN-ÇIKTI · GİRDİ: 12 5

```
Toplam: 17
Fark: 7
Çarpım: 60
```

Dizin: `alistirma/kod/H-1-A1/`

ALIŞTIRMA H-1-A2 Bir sözcük, bir sayı

Bu sefer bir sözcük ve bir sayı okuyun: `elma 3`. Çıktı tek satır: `3 tane elma`. Sözcük için `string` türü gerekiyor; `#include <string>` satırını unutmayın.

çıktı — BEKLENEN-CİKTİ · GİRDİ: ELMA 3

```
3 tane elma
```

Dizin: `alistirma/kod/H-1-A2/`

ZORLAYICI H-1-Z1 Üç sayının ortalaması

Üç tam sayı okuyun, ortalamasını yazın. Girdi `3 4 4`; beklenen çıktı `Ortalama: 3.66667`. İlk denemenizde büyük ihtimalle `3` çıkacak. Neden? `H-1-I2`'deki üç satırı yeniden okuyun; toplamı bölmeden önce kesirli bir türe çevirmenin bir yolunu bulun.

DÜŞÜNCE

Üç tam sayının toplamı tam sayı; 11'i 3'e bölünce C++ kesri atar, 3 kalır.

YAKLAŞIM

Bölmeden önce taraflardan birini kesirli türe çevirin: `3.0` ile bölmek ya da toplamı `double`'a atamak yeter.

İLK SATIR

```
double ortalama{(a + b + c) / 3.0};
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 4 4

```
Ortalama: 3.66667
```

Dizin: `alistirma/kod/H-1-Z1/`

H.2

Hata iletisini okumak

Derleyici bağırınca çoğu kişi iletiyi okumadan koda bakar. Tersini yapın: önce iletiyi okuyun. Satır numarası, sözcük, bazen de öneri orada. Bu dört programın her birinde bir şey bozuk; görev kodu değil, iletiyi okuyup düzeltmek.

1 2 1

> **YANIT YERİNE**
>
> ders 5: derleyici ve terminal kitapçık III: Hata bulmanın üç yolu kopya sayfaları

İletiler İngilizce; ilk hafta en sık karşılaşacağınız on tanesinin Türkçesi ve çoğu zamanki nedeni aşağıda. Satır numarası her zaman iletinin başında: `ana.cpp:7:12` yedinci satır, on ikinci sütun demek.

İLETİ
`expected ';' before ...`
`'x' was not declared in this scope`
`'cout' was not declared` / `'string' does not name a type`
`no match for 'operator<<'` / `no matching function for call to`
`expected '}' at end of input`
`warning: unused variable 'x'`
`warning: comparison of integer expressions of different signedness`
`warning: control reaches end of non-void function`
`undefined reference to 'f()'`

İLETİ

`Segmentation fault`

ISINMA H-2-I1

Bir karakter eksik

Program derlenmiyor. İletide satır numarasını bulun, o satıra bakın. Düzeltince aşağıdaki çıktıyı vermeli.

ana.cpp — BAŞLANGIÇ

```cpp
// H-2-I1 · Bir karakter eksik
#include <iostream>
using namespace std;

int main() {
  cout << "Virgül değil, noktalı virgül." << endl
  return 0;
}
```

çıktı — BEKLENEN-ÇİKTİ

```
Virgül değil, noktalı virgül.
```

Dizin: `alistirma/kod/H-2-I1/`

ALIŞTIRMA H-2-A1

Üç hata, üç ileti

Bu sefer üç ayrı hata var. Derleyici hepsini birden söyleyebilir ya da ilkini düzeltince yenisi çıkabilir. Her ileti için önce “hangi satır, hangi sözcük” diye sorun. İpucu: hatalardan biri bir harfin büyüklüğü, biri bir başlık dosyası, biri de yine o noktalı virgül.

ana.cpp — BAŞLANGIÇ

```cpp
// H-2-A1 · Üç hata, üç ileti
using namespace std;

int main() {
  Int kac{5};
  for (int k{0}; k < kac; ++k)
    cout << "merhaba " << k + 1 << endl
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
merhaba 1
merhaba 2
merhaba 3
merhaba 4
merhaba 5
```

Dizin: `alistirma/kod/H-2-A1/`

ALIŞTIRMA H-2-A2

Derleyici kızmıyor ama yanlış

Bu program derleniyor, çalışıyor ve yanlış yanıt veriyor: `sayi` 3 olduğu hâlde `yedi` yazıyor. `if` satırına çok dikkatli bakın. `-Wall` ile derlerseniz derleyici bir uyarı da verir; onu da okuyun, ne öneriyor?

ana.cpp — BAŞLANGIÇ

```cpp
// H-2-A2 · Derleyici kızmıyor ama yanlış
#include <iostream>
using namespace std;

int main() {
  int sayi{3};
  if (sayi = 7) cout << "yedi" << endl;
  else          cout << "yedi değil" << endl;
  return 0;
}
```

çıktı — BEKLENEN-ÇİKTİ

```
yedi değil
```

Dizin: `alistirma/kod/H-2-A2/`

ZORLAYICI H-2-Z1

diff ne diyor?

Bu program da derleniyor, çalışıyor, ama beklenen çıktıyla *bir* farkı var. Kodu okumadan önce `make test` çalıştırın ve `diff`'in söylediğini anlamaya çalışın: `<` ile başlayan satır beklenen, `>` ile başlayan sizin çıktınız. Bu defterin bütün alıştırmaları bu yöntemle sınanıyor; `diff` okumayı burada öğrenin. Tarayıcıda çalışıyorsanız iki çıktıyı gözle karşılaştırın.

DÜŞÜNCE

Program 1'den 9'a kadar yazıyor; beklenen çıktıda bir sayı daha var.

YAKLAŞIM

`diff`'teki `<` satırı doğrusu, `>` satırı sizinki; ikisini yan yana koyup hangi sayının eksik olduğuna bakın, sonra döngünün sınırına.

İLK SATIR

`for (int k{1}; k <= 10; ++k)` satırının `<` mü `<=` mi olacağına diff karar versin.

ana.cpp — BAŞLANGIÇ

```cpp
// H-2-Z1 · diff ne diyor?
#include <iostream>
using namespace std;

int main() {
  for (int k{1}; k < 10; ++k)
    cout << k << " ";
  cout << endl;
  return 0;
}
```

çıktı — BEKLENEN-ÇİKTİ

```
1 2 3 4 5 6 7 8 9 10
```

Dizin: `alistirma/kod/H-2-Z1/`

H.3

İkili çalışma: sürücü ve seyrüseferci

Kitapçık ikili çalışmayı öneriyor; burada deneyin. Biri klavyede (*sürücü*), öteki yanında sesli düşünüyor, hataları görüyor, sıradaki adımı söylüyor (*seyrüseferci*). On beş dakikada bir yer değiştirin. Tek başınaysanız da olur: sürücü sizsiniz, seyrüseferci de siz; yüksek sesle konuşun.

1 1 1

> **YANIT YERİNE**
>
> kitapçık: Yalnız gidebilirsiniz, ama ikili daha hızlı ders 1-4

ISINMA H-3-I1

Çerçeve

Seyrüseferci okusun, sürücü yazsın: üç satırlık bir çerçeve. Üstte ve altta on bir yıldız, ortada `* Merhaba *`. Yıldız sayısını elle sayın; `diff` bir fazlasını bile affetmez.

çıktı — BEKLENEN-CİKTİ

```
***********
* Merhaba *
***********
```

Dizin: `alistirma/kod/H-3-I1/`

ALIŞTIRMA H-3-A1

Yedinin çarpım tablosu

Yer değiştirin. `7 x 1 = 7`'den `7 x 10 = 70`'e, on satır. Döngüyle yazın; on tane `cout` satırı yazan takım kendi kendine “Kendini tekrar etme” diye fısıldasın.

çıktı — BEKLENEN-CİKTİ

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Dizin: `alistirma/kod/H-3-A1/`

KEŞİF H-3-K1 Yıldızlardan bir harf

Takımınızın adının baş harfini yıldızlarla, yedi satırda çizin. Doğru çıktı yok, bu yüzden `beklenen-cikti` de yok; `make` ile derleyip bakın. Önce kâğıda çizin, sonra satır satır yazın. Sıkı takımlar döngüyle çizer: `K` harfinin çaprazını `for` ile yazabilir misiniz?

DÜŞÜNCE

Yedi satırlık bir resim; her satır yıldız ve boşluklardan oluşan bir yazı.

YAKLAŞIM

Önce kâğıda 7×7 kareli bir ızgara çizip harfi yıldızlayın; her satırı bir `cout` satırı yapın, sonra tekrar eden satırları döngüye alın.

İLK SATIR

`for (int k{0}; k < 7; ++k) cout << satir(k) << endl;` gibi bir yardımcı işlev de olur.

Dizin: `alistirma/kod/H-3-K1/` — beklenen çıktı yok

BÖLÜM I

İlk Adımlar

Kitapçığın birinci bölümü beş yapıtaşı veriyor: değer, ad, tür, karar, tekrar; bir de işlev. Burada her birine bir merdiven var. Isınmalar kitapçıktaki programla başlıyor; Keşifler Project Euler'e çıkıyor.

Hazırlık | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Zanaat | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

I.1

Değer, ad, tür

Kitapçığın `01-turler.cpp` programı dört türü gösteriyor. Aşağıdaki beş alıştırma o programdan başlayıp türlerin sınırlarına kadar gidiyor.

1 2 1 1

YANIT YERİNE

kitapçık I.1: Değer, ad, tür ders 1-4 kod: 01-turler.cpp

ISINMA I-1-I1 Deneyin kutusu

Kitapçığın *Deneyin* kutusu: `ogrenci`'yi `false` yapın, `char basHarfi{'B'}` yerine `int basHarfi{'B'}` yazın. Çalıştırmadan önce dört satırın nasıl değişeceğini tahmin edin, sonra beklenen çıktıyla karşılaştırın. `66` nereden geldi?

çıktı — BEKLENEN-CİKTİ

```
Merhaba! Ben senin ilk programın.
Yaş      : 16
Boy      : 1.72 m
Baş harfi: 66
Öğrenci  : 0
Öğrenci  : false
On yıl sonra: 26
```

Dizin: `alistirma/kod/I-1-I1/`

ALIŞTIRMA I-1-A1 Dört değişken, dört tür

Bir kitabı dört değişkenle tanımlayın: adı (`string`), sayfa sayısı (`int`), fiyatı (`double`), rafta mı (`bool`). Değerler: `Keyifli Alıştırmalar` , `120` , `49.9` , `true` . Dört satırda, aşağıdaki biçimde yazın. `boolalpha`'yı hatırlayın.

çıktı — BEKLENEN-CİKTİ

```
Kitap : Keyifli Alıştırmalar
Sayfa : 120
Fiyat : 49.9 TL
Rafta : true
```

Dizin: `alistirma/kod/I-1-A1/`

ALIŞTIRMA I-1-A2

Kaç bayt?

Kitapçık “bellek upuzun bir şerit” diyor. Her tür o şeritten kaç bayt alıyor? `sizeof(int)` söyler. Beş tür için yazın: `char`, `int`, `long long`, `double`, `bool`. (Beklenen çıktı bugünkü yaygın bilgisayarlar için; başka bir makinede başka çıkarsa bu bir hata değil, bir keşif.)

çıktı — BEKLENEN-CİKTİ

```
char      : 1 bayt
int       : 4 bayt
long long : 8 bayt
double    : 8 bayt
bool      : 1 bayt
```

Dizin: `alistirma/kod/I-1-A2/`

ZORLAYICI I-1-Z1

Harfler de sayı

`I-1-I1`'deki 66, `'B'` harfinin bellekteki sayısı (ASCII). Madem harf sayıdır, harfle hesap yapılır: bir döngüyle `A`'dan `Z`'ye bütün büyük harfleri tek satırda yazın. Sonra iki satır daha: `'B' + 1` kaç eder, `char('B' + 1)` hangi harf? Döngü değişkeninin türünü iyi seçin.

DÜŞÜNCE

Harf bir sayıdır; A'dan Z'ye gitmek 65'ten 90'a saymaktır.

YAKLAŞIM

Döngü değişkenini `char` yapın ve `'A'`'dan `'Z'`'ye `++h` ile ilerleyin; `'B' + 1` ise `int`, onu harfe çevirmek için `char(...)`.

İLK SATIR

```
for (char h{'A'}; h <= 'Z'; ++h) cout << h;
```

çıktı — BEKLENEN-CİKTİ

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
'B' + 1 = 67
char('B' + 1) = C
```

Dizin: `alistirma/kod/I-1-Z1/`

KEŞİF I-1-K1 Sınırları keşfedin

Kitapçık `numeric_limits<int>::max()`'i gösterdi. `<limits>` başlığını kurcalayın: `short`, `int`, `long long` için en büyük değer; `double` için `epsilon()` (birden sonra temsil edilebilen ilk sayı ile 1 arasındaki fark). Beklenen çıktı bu dördü için; sonra `min()`, `digits`, `lowest()` ile kendiniz oynayın. Hangi tür ne zaman gerekir, takımınızla tartışın: nüfus, sıcaklık, bir harf notu, kapı açık mı.

DÜŞÜNCE

Her türün en büyük değeri `<limits>`'te hazır; dört satır, dört tür.

YAKLAŞIM

Başlangıç dosyasındaki `short` satırını kopyalayıp türü değiştirin; `double` için `max()` yerine `epsilon()`.

İLK SATIR

```
cout << numeric_limits<long long>::max() << endl;
```

çıktı — BEKLENEN-CİKTİ

```
short en büyük     : 32767
int en büyük       : 2147483647
long long en büyük : 9223372036854775807
double epsilon     : 2.22045e-16
```

Dizin: `alistirma/kod/I-1-K1/`

I.2

Her türün bir sınırı var: taşma

Yarışma sorularında en çok can yakan hata. Kitapçığın `02-tasma.cpp` programıyla başlayıp sınırın tam yerini bulana kadar gidiyoruz.

1 2 1

YANIT YERİNE

kitapçık I.1: Her türün bir sınırı var ders 1-4 kod: 02-tasma.cpp

ISINMA I-2-I1

Küp küçülünce

`a`'yı `2000` yerine `1000` yapın. Hangi satırlar değişir, hangileri değişmez? Önce tahmin edin: 1000'in küpü `int`'e sığar mı? Kitapçıktaki sınırla karşılaştırın.

çıktı — BEKLENEN-CİKTİ

```
1000'in küpü (int) : 1000000000
3000'ün küpü (int) : 1230196224
3000'ün küpü (long long): 27000000000
karekök(1000'in küpü) : 31622.8
int'in en büyüğü               : 2147483647
unsigned long long'un en büyüğü : 18446744073709551615
```

Dizin: `alistirma/kod/I-2-I1/`

ALIŞTIRMA I-2-A1

Faktöriyel nerede taşar?

`n! = 1 · 2 · … · n`. Bir döngüyle hesaplayın; `12!` ve `13!`'ü önce `int`, sonra `long long` ile. `20!`'i de `long long` ile ekleyin. Dört satır. Hangi noktada `int` yalan söylemeye başlıyor?

çıktı — BEKLENEN-CİKTİ

```
12! (int)       : 479001600
13! (int)       : 1932053504
13! (long long) : 6227020800
20! (long long) : 2432902008176640000
```

Dizin: `alistirma/kod/I-2-A1/`

ALIŞTIRMA I-2-A2

Sıfırın bir eksiği

`unsigned int` eksi sayı tutamaz. Peki `0`'dan `1` çıkarınca ne olur? Ya en büyük değere `1` eklenince? İki satırda yazın. Kilometre saati benzetmesini hatırlayın.

çıktı — BEKLENEN-CİKTİ

```
unsigned 0 - 1          = 4294967295
unsigned en büyük + 1   = 0
```

Dizin: `alistirma/kod/I-2-A2/`

ZORLAYICI I-2-Z1

Neden tam 2147483647?

Kitapçığın *Takım işi* kutusundaki soru. Kâğıtta çözdükten sonra programla doğrulayın: `2^31`, `2^31 - 1`, `2^32` ve `2^63 - 1`. Çarpı işaretiyle değil, kaydırma işleciyle: `1LL << 31` ne demek? `1 << 31` yazarsanız ne olur, neden `LL` gerekti? Son satır için dikkat: `1LL << 63` taşar; önce `62`'yi düşünün.

DÜŞÜNCE

2^{31} bir sayının 31 kere ikiye katlanması; kaydırma işleci tam bunu yapar.

YAKLAŞIM

`1LL << 31` 64 bitte hesaplar; `1 << 31` ise 32 bitte taşar. $2^{63} - 1$ için `1LL << 63` taşar: iki kere $2^{62} - 1$ artı 1 diye düşünün ya da `numeric_limits` ile doğrulayın.

İLK SATIR

```
cout << (1LL << 31) - 1 << endl;
```

çıktı — BEKLENEN-CİKTİ

```
2^31     = 2147483648
2^31 - 1 = 2147483647
2^32     = 4294967296
2^63 - 1 = 9223372036854775807
```

Dizin: `alistirma/kod/I-2-Z1/`

I.3

Kararlar ve tekrarlar

`if`, `for`, `while`. Kitapçık bu üçünü Project Euler'in birinci sorusuyla gösterdi; biz de oradan başlıyoruz ve ikinci soruya kadar gidiyoruz.

1 2 2 1

YANIT YERİNE

kitapçık I.2: Kararlar ve tekrarlar ders 5: PE 1 ders 6: Fibonaççi kod: 04-pe1.cpp

ISINMA I-3-I1

Bin yerine yüz

Kitapçığın `04-pe1.cpp` programında sınırı `1000` yerine `100` yapın. Çalıştırmadan tahmin etmeyin, kâğıtta hesaplayın: 3'ün katları, 5'in katları, 15'in katları. Sonra çalıştırıp karşılaştırın.

çıktı — BEKLENEN-CİKTİ

```
Project Euler, birinci problem: 2318
```

Dizin: `alistirma/kod/I-3-I1/`

ALIŞTIRMA I-3-A1

Fizz, Buzz, FizzBuzz

Dünyanın en çok sorulan iş görüşmesi sorusu. 1'den 20'ye kadar sayın; 3'e bölünende `Fizz`, 5'e bölünende `Buzz`, ikisine de bölünende `FizzBuzz`, yoksa sayının kendisini yazın. Her biri ayrı satır. `if`'lerin sırası önemli; 15'i düşünün.

çıktı — BEKLENEN-CİKTİ

```
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
```

Dizin: `alistirma/kod/I-3-A1/`

ALIŞTIRMA I-3-A2

Basamaklar

Bir tam sayı okuyun; kaç basamağı olduğunu ve basamaklarının toplamını tek satırda yazın. `H-1-I2`'deki `%` ve `/` burada iş görüyor: `2026 % 10` son basamak, `2026 / 10` gerisi. Sayı sıfır olana kadar `while`.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 2026

```
2026 sayısının 4 basamağı var, basamakları toplamı 10.
```

Dizin: `alistirma/kod/I-3-A2/`

ZORLAYICI İKİLİ I-3-Z1

Klasik tuzağı kendiniz kurun

Kitapçığın *Klasik tuzak* kutusu: `if`'i ikiye bölünce 15'in katları iki kere sayılıyor. Tuzağı bilerek kurun: iki ayrı `if` ile "yanlış" toplamı hesaplayın, sonra 15'in katlarını çıkararak düzeltin, sonra da `or`'lu doğru hâlle karşılaştırın. Üç satır: yanlış, yamalı, doğru. Yamalı ile doğru eşit çıkmalı. Hangisi daha okunaklı?

DÜŞÜNCE

Üç toplam: 3'e bölünenler artı 5'e bölünenler (15'ler iki kere), 15'leri bir kere çıkarınca yamalı, tek `or`'lu koşulla doğru.

YAKLAŞIM

Tek döngü, üç sayaç; her sayı için üç ayrı `if` bloğu. Yamalı sürüm, yanlış toplamdan 15'in katlarını bir kere düşer.

İLK SATIR

```
if (bolunuyorMu(s, 3)) yanlis += s; if (bolunuyorMu(s, 5)) yanlis += s;
```

çıktı — BEKLENEN-CİKTİ

```
iki if ile  : 266333
15 çıkarınca: 233168
or ile      : 233168
```

Dizin: `alistirma/kod/I-3-Z1/`

ZORLAYICI I-3-Z2

Collatz'ın inatçı dizisi

Bir sayı alın: çiftse ikiye bölün, tekse üçle çarpıp bir ekleyin; 1'e varana kadar. Kimse bunun her sayı için bittiğini kanıtlayamadı, ama 27 için biter: kaç adımda? Yolda görülen en yüksek değer ne? Girdi `27`. Sayı taşabilir mi diye düşünün, türünüzü ona göre seçin.

DÜŞÜNCE

Bir sayı 1'e inene kadar çift/tek kuralıyla dönüşüyor; adımları ve yolda görülen en büyüğü sayın.

YAKLAŞIM

`while (n != 1)` döngüsü, içinde `n % 2` ile ikiye ayrılan bir `if`; her adımda sayaç ve `max`. 3n+1 ara değerleri `int`'i aşabilir: `long long`.

İLK SATIR

```
while (n != 1) { n = (n % 2 == 0) ? n / 2 : 3 * n + 1; ++adim;
enBuyuk = max(enBuyuk, n); }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 27

```
27 için 111 adım, en yüksek değer 9232.
```

Dizin: `alistirma/kod/I-3-Z2/`

KEŞİF I-3-K1 Project Euler 2

İkinci soru: Fibonaççi dizisinde (1, 2, 3, 5, 8, ...) dört milyonu aşmayan *çift* terimlerin toplamı. Döngüyle; özyineleme Bölüm IV'te. Yanıtı bulunca Project Euler hesabınıza girin, yeşil tik alın. Sonra bir adım öteye: her üçüncü Fibonaççi sayısının çift olduğunu fark ettiniz mi? Neden?

DÜŞÜNCE

Fibonaççi terimlerini üretip çift olanları topluyorsunuz; dört milyonu geçince durun.

YAKLAŞIM

İki değişken yeter: `a` ve `b`; her adımda `(a, b) = (b, a + b)`. Üçüncü bir geçici değişken gerekir.

İLK SATIR

```
long long a{1}, b{2}, toplam{0}; while (b <= 4000000) { ... }
```

çıktı — BEKLENEN-CİKTİ

```
Project Euler, ikinci problem: 4613732
```

Dizin: `alistirma/kod/I-3-K1/`

I.4

İşlev: bir kere yaz, çok kere kullan

Kitapçık `bolunuyorMu()` için "tek satır, buna değer mi? Kesinlikle" dedi. Burada beş işlev yazacaksınız; her biri bir satırdan biraz uzun.

1 2 1 1

YANIT YERİNE

kitapçık I.2: İşlev ders 1-4 ders 8: Kendini tekrar etme

ISINMA I-4-I1

Kare ve küp

İki işlev: `kare(int)` ve `kup(int)`. İkisi de `int` döndürsün. `main` hazır, yalnız işlevleri yazın. Sonra `kup`'u `kare`'yi kullanarak yazabilir misiniz?

ana.cpp — BAŞLANGIÇ

```cpp
// I-4-I1 · Kare ve küp
#include <iostream>
using namespace std;

// kare(int) ve kup(int) işlevlerini buraya yazın

int main() {
  cout << "12'nin karesi: " << kare(12) << endl;
  cout << "12'nin küpü  : " << kup(12) << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
12'nin karesi: 144
12'nin küpü  : 1728
```

Dizin: `alistirma/kod/I-4-I1/`

ALIŞTIRMA I-4-A1

Üçün en büyüğü

`enBuyuk(int, int, int)` üç sayının en büyüğünü döndürsün. Girdi `7 42 19`. Önce `if`'lerle yazın; sonra önce ikisinin en büyüğünü bulan iki girdili bir işlev yazıp üçlüyü onunla kurun. Hangisi kısa?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 7 42 19

```
En büyük: 42
```

Dizin: `alistirma/kod/I-4-A1/`

ALIŞTIRMA I-4-A2

Asal mı?

`asalMi(int)` bir `bool` döndürsün. Sonra 50'den küçük asalları tek satırda yazın: her sayıdan sonra bir boşluk, en sonda `endl`. Bölenleri nereye kadar denemek gerekir: `n`'e kadar mı, `n/2`'ye mi, `√n`'e mi? Üçü de doğru yanıt verir; farkı Bölüm III'te ölçeceksiniz.

çıktı — BEKLENEN-CİKTİ

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
```

Dizin: `alistirma/kod/I-4-A2/`

ZORLAYICI I-4-Z1

Öklid'in işlevi

İki sayının en büyük ortak böleni için Öklid'in 2300 yıllık yöntemi: büyük sayıyı küçüğe böl, kalanı al; kalan sıfır olana kadar. `ebob(int, int)` işlevini döngüyle yazın (özyinelemesini Bölüm IV'te göreceksiniz). Sonra `ekok`'u `ebob`'dan türetin: `a · b / ebob`. Girdi `1071 462`.

DÜŞÜNCE

Öklid: `ebob(a, b) = ebob(b, a % b)`, kalan sıfır olana kadar.

YAKLAŞIM

`while (b != 0)` döngüsünde kalanı hesaplayıp `a`'yı `b`, `b`'yi kalan yapın; ekok için çarpmadan önce bölün ki taşmasın: `a / ebob * b`.

İLK SATIR

```
int ebob(int a, int b) { while (b != 0) { int k{a % b}; a = b; b = k; } return a; }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 1071 462

```
ebob(1071, 462) = 21
ekok(1071, 462) = 23562
```

Dizin: `alistirma/kod/I-4-Z1/`

KEŞİF I-4-K1

Project Euler 3

Üçüncü soru: `600851475143` sayısının en büyük asal çarpanı. Kitapçığın uyarısı: bu sayı `int`'e sığmıyor. `I-4-A2`'deki `asalMi`'yi her bölen için çağırmak çok yavaş olur; başka bir yol düşünün: en küçük bölenden başlayıp sayıyı bölerek küçültün. Asallık sınaması gerekmeyebilir. Neden?

DÜŞÜNCE

Sayıyı en küçük bölenden başlayarak bölüp küçültün; bölen artık bölmüyorsa bir artırın.

YAKLAŞIM

`for (long long b{2}; b * b <= sayi; ++b) while (sayi % b == 0) sayi /= b;` döngüsünden sonra kalan sayı (1 değilse) en büyük asal çarpan. Bölenler asal olmasa da olur; neden?

İLK SATIR

```
long long b{2}; while (b * b <= sayi) { ... }
```

çıktı — BEKLENEN-CİKTİ

```
Project Euler, üçüncü problem: 6857
```

Dizin: `alistirma/kod/I-4-K1/`

I.5

İlk oyununuz

Kitapçığın tahmin oyunu rastgele sayı tutuyor; rastgele programı sınamak zordur. İlk alıştırma oyunu sınanabilir yapıyor, gerisi o temelin üstüne çıkıyor. Girdi isteyen alıştırmalarda tahminler `girdi` dosyasından geliyor; tarayıcıda *Input* kutusuna yazın.

1 1 2 1

YANIT YERİNE

kitapçık I.3: İlk oyununuz ders 1-4: rastgele sayılar, girdi kod: 03-tahmin.cpp

ISINMA I-5-I1 Rastgeleyi kapatın

`03-tahmin.cpp`'de `gizli`'yi rastgele seçen satırı `const int gizli{37};` yapın. Artık oyun sınanabilir: girdi `50 25 37` ile aşağıdaki konuşma çıkmalı. “Bildin! 3 tahminde” sayısı nereden geliyor, `toplamHak - hak` ifadesini okuyun.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 50 25 37

```
1 ile 100 arasında bir sayı tuttum. 7 hakkın var.
Tahminin: Daha küçük. Kalan hak: 6
Tahminin: Daha büyük. Kalan hak: 5
Tahminin: Bildin! 3 tahminde.
```

Dizin: `alistirma/kod/I-5-I1/`

ALIŞTIRMA I-5-A1 Kaç hak yeter?

Kitapçığın *Deneyin* kutusu: hak sayısını sınıra bakarak hesaplayın. Bir `hakSayisi(int sinir)` işlevi yazın: 2^n sınırı geçene kadar sayan bir `while`. Üç sınır için yazın: 100, 1000, 1000000. `log2` kullanmayın, elle; sonra `<cmath>` ile doğrulayın.

çıktı — BEKLENEN-CİKTİ

```
100 için 7 hak
1000 için 10 hak
1000000 için 20 hak
```

Dizin: `alistirma/kod/I-5-A1/`

ZORLAYICI İKİLİ I-5-Z1

Tersine oyun

Kitapçığın birinci alıştırması: siz sayı tutun, bilgisayar bulsun. Her tahminden sonra siz `b` (daha büyük), `k` (daha küçük) ya da `d` (doğru) yazın. Bilgisayar akıllı oyuncu olsun: her seferinde kalan aralığın ortasını söylesin. Girdi `b k b d` ile tahminler 50, 75, 62, 68 olmalı. İki değişken yeter: `alt` ve `ust`. Bu, ikili aramanın ta kendisi.

DÜŞÜNCE

Bilgisayar her seferinde aralığın ortasını söylüyor; yanıtınıza göre aralık yarıya iniyor.

YAKLAŞIM

`alt` ve `ust`; tahmin `(alt + ust) / 2`; `b` gelince `alt = tahmin + 1`, `k` gelince `ust = tahmin - 1`, `d` gelince dur.

İLK SATIR

`while (true) { int tahmin{(alt + ust) / 2}; cout << tahmin << endl; char y; cin >> y; ... }`

çıktı — BEKLENEN-CİKTİ · GİRDİ: B K B D

```
1 ile 100 arasında bir sayı tutun. Ben bulayım.
Tahminim: 50 (b/k/d)
Tahminim: 75 (b/k/d)
Tahminim: 62 (b/k/d)
Tahminim: 68 (b/k/d)
4 tahminde buldum: 68
```

Dizin: `alistirma/kod/I-5-Z1/`

ZORLAYICI I-5-Z2

Bozuk girdiye dayanmak

Kitapçığın oyunu bozuk girdide (`abc` gibi) oyunu bitiriyor: `if (not (cin >> tahmin)) break;`. Daha nazik olun: “Bu bir sayı değil, hak yanmadı.” deyip devam edin. Bunun için `cin.clear()` ile hatayı silmek, sonra bozuk sözcüğü okuyup atmak gerekiyor. Girdi gerçekten bitmişse yine çıkın (`cin.eof()`). `gizli` 37 olsun; girdi `abc 37`.

DÜŞÜNCE

Bozuk girdi `cin`'i hata durumuna sokar; durumu silip bozuk sözcüğü atmadan devam edemezsiniz.

YAKLAŞIM

`if (not (cin >> tahmin))` dalında önce `cin.eof()`'a bakın (bitti mi), değilse `cin.clear()`, sonra bozuk parçayı bir `string`'e okuyup atın ve `continue`.

İLK SATIR

```
if (cin.eof()) break; cin.clear(); string cop; cin >> cop; cout << "Bu bir sayı değil, hak yanmadı." << endl; continue;
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: ABC 37

```
1 ile 100 arasında bir sayı tuttum. 7 hakkın var.
Tahminin: Bu bir sayı değil, hak yanmadı.
Tahminin: Bildin! 1 tahminde.
```

Dizin: `alistirma/kod/I-5-Z2/`

KEŞİF I-5-K1 Oyunu büyütün

Beklenen çıktı yok; oyun sizin. Birkaç yön: oyun bitince “bir daha?” diye sorsun; iki oyuncu sırayla tahmin etsin, bilen kazansın; sınırı kullanıcı seçsin, hak sayısı `I-5-A1`'deki işlevle hesaplansın. En az birini yapın, ikili çalışın, sonra sınıftan birine oynatın: oyununuz onun yaptığı tuhaf şeylere dayanıyor mu?

DÜŞÜNCE

Oyunun gövdesi hazır; etrafına bir katman daha koyuyorsunuz: tekrar, iki oyuncu ya da seçilebilir sınır.

YAKLAŞIM

Oyunu bir `oyna()` işlevine taşıyın; `main` onu bir `do … while` içinde çağırsın. İki oyuncu için sıra değişkeni (`sira = 1 - sira`).

İLK SATIR

```
do { oyna(); cout << "bir daha? (e/h) "; cin >> yanit; } while (yanit == 'e');
```

Dizin: `alistirma/kod/I-5-K1/` — beklenen çıktı yok

I.6

Adların nerede geçerli olduğu

Kapsam. Kitapçık “yanıtı vermiyorum, çalıştırın” dedi. Biz de vermiyoruz; dört alıştırma, her biri bir tuzak.

1 2 1

YANIT YERİNE

kitapçık I.4: Adların nerede geçerli olduğu neden-namespace-kullanırız kod: 05-kapsam.cpp

ISINMA I-6-I1 Bir sözcük silin

Önce `05-kapsam.cpp`'yi olduğu gibi çalıştırıp iki soru işaretini doldurun. Sonra `g()` içindeki `int` sözcüğünü silin. Şimdi ne yazar? Çalıştırmadan önce yanıtı kâğıda yazın; beklenen çıktı değişmiş hâli için.

çıktı — BEKLENEN-CİKTİ

```
5 99
```

Dizin: `alistirma/kod/I-6-I1/`

ALIŞTIRMA I-6-A1 Döngüden sonra

Program derlenmiyor: `k` döngünün dışında tanınmıyor. Döngüyü değiştirmeden, `k`'yı döngüden sonra da yaşayacak biçimde tanımlayın. Çıktı `k = 10` olmalı. Kitapçığın kuralını hatırlayın: en dar kapsam — ama bu sefer döngüden *bir* adım geniş.

ana.cpp — BAŞLANGIÇ

```cpp
// I-6-A1 · Döngüden sonra
#include <iostream>
using namespace std;

int main() {
  for (int k{0}; k < 10; ++k) {
    // bir şey yapıyormuş gibi
  }
  cout << "k = " << k << endl;   // derlenmez: k burada yok
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
k = 10
```

Dizin: `alistirma/kod/I-6-A1/`

ALIŞTIRMA I-6-A2

İç içe gölgeler

Her süslü parantez yeni bir kapsam, her kapsamda yeni bir `x`. Program beş sayı yazıyor. Çalıştırmadan önce sırayı tahmin edin; sonra sınayın. Bu kod da kitapçığın dediği gibi "kasıtlı olarak kafa karıştırmak için" yazıldı. Kendi programınızda yapmayın.

ana.cpp — BAŞLANGIÇ

```cpp
// I-6-A2 · İç içe gölgeler: önce tahmin edin
#include <iostream>
using namespace std;

int main() {
  int x{1};
  cout << x;
  {
    int x{2};
    cout << " " << x;
    {
      int x{3};
      cout << " " << x;
    }
    cout << " " << x;
  }
  cout << " " << x << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
1 2 3 2 1
```

Dizin: `alistirma/kod/I-6-A2/`

ZORLAYICI I-6-Z1

Küreselsiz sayaç

Aşağıdaki program küresel bir `sayac` kullanıyor; `say()` onu artırıp yazıyor. Şimdi *ikinci* bir sayaç gerekiyor: `b` satırı ayrıca 1'den 2'ye saysın. Küresel değişkenle bunu yapmak ya iki işlev ya iki küresel demek. Küreseli tamamen kaldırın: `say` sayacı girdi olarak alsın, bir fazlasını döndürsün. Çıktı iki satır.

DÜŞÜNCE

Sayaç küresel olmasın; işlev onu yine de hatırlasın ya da dışarıdan alsın.

YAKLAŞIM

İki yol: işleve sayacı takma adla vermek (`int& sayac`) ya da işlevin içinde `static int sayac{0};` tutmak. İkisini de deneyin; hangisi sınamak için daha kolay?

İLK SATIR

```
void say(int& sayac) { ++sayac; cout << sayac << endl; }
```

ana.cpp — BAŞLANGIÇ

```cpp
// I-6-Z1 · Küreselsiz sayaç
// Küreseli kaldırın; ikinci satır: "b: 1 2"
#include <iostream>
using namespace std;

int sayac{0};              // küresel: bundan kurtulun

void say() {
  ++sayac;
  cout << " " << sayac;
}

int main() {
  cout << "a:";
  say(); say(); say();
  cout << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
a: 1 2 3
b: 1 2
```

Dizin: `alistirma/kod/I-6-Z1/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

“Bu bölümü geçtim mi?” sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` “Doğru!” desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

- `I-3-Z2` Collatz'ın inatçı dizisi
- `I-4-Z1` Öklid'in işlevi
- `I-5-Z1` Tersine oyun

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.

BÖLÜM II

Veriyi Düzenlemek

Kitapçığın ikinci bölümü hazır kapları tanıtıp kendi türünüzü yazdırıyor. Burada yedi konu, otuz beş alıştırma: dizi ve akıllı dizi, eşlem, küme, sıralı kaplar, kendi türünüz, kalıplar, adres ve takma ad. Keşifler Project Euler 10'a ve küçük bir robota çıkıyor.

Hazırlık · I. İlk Adımlar · II. Veriyi Düzenlemek · III. Zanaat · IV. Özyineleme · V. Çizgeler · VI. Dinamik Programlama

II.1

Dizi ve akıllı dizi

Kitapçığın `10-dizi.cpp` programı belleği gösteriyor. Buradaki başlangıç dosyasında adres satırları yok: adresler her çalıştırmada değişir, `diff` onları sınayamaz. Akıllı diziyle (`vector`) devam ediyor, Eratosthenes'in kalburuna çıkıyoruz.

1 2 2 1

> YANIT YERİNE
>
> kitapçık II.1: Dizi ders 8: dizi, akıllı dizi ders 13: iki boyutlu diziler kod: 10-dizi.cpp

ISINMA II-1-I1

short yerine int

Programdaki `short`'u `int` yapın. Üç sayı değişecek; çalıştırmadan önce hangileri, kaç olacak? `sizeof(dizi) / sizeof(dizi[0])` hesabı neden değişmiyor?

ana.cpp — BAŞLANGIÇ

```cpp
// II-1-I1 · short yerine int
#include <iostream>
using namespace std;

int main() {
  short dizi[]{3, 1, 4, 1, 5, 9, 2, 6};

  cout << "dizinin boyu     : " << sizeof(dizi)    << " bayt" << endl;
  cout << "bir ögenin boyu  : " << sizeof(dizi[0]) << " bayt" << endl;
  int ogeSayisi = sizeof(dizi) / sizeof(dizi[0]);
  cout << "öge sayısı       : " << ogeSayisi << endl;

  for (auto s : dizi) cout << s << " ";
  cout << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
dizinin boyu     : 32 bayt
bir ögenin boyu  : 4 bayt
öge sayısı       : 8
3 1 4 1 5 9 2 6
```

Dizin: `alistirma/kod/II-1-I1/`

ALIŞTIRMA II-1-A1

Toplam, en büyük, en küçük

Önce bir `n`, sonra `n` sayı okuyun; bir `vector<int>`'e koyun. Dört satır yazın: toplam, en büyük, en küçük, ortalama. Girdi `5` ve `3 1 4 1 5`. En büyüğü bulmak için ilk ögeyle başlayın, sıfırla değil; eksi sayılarda neden fark eder?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 3 1 4 1 5

```
Toplam: 14
En büyük: 5
En küçük: 1
Ortalama: 2.8
```

Dizin: `alistirma/kod/II-1-A1/`

ALIŞTIRMA II-1-A2

Tersten

Bu sefer kaç sayı geleceği belli değil: girdi bitene kadar okuyun (`while (cin >> x)`), `push_back` ile biriktirin, sonra tersten yazın. Girdi `3 1 4 1 5 9 2 6`. Sondan başa giden döngünün sınırlarına dikkat: `size()` işaretsizdir, `i >= 0` hiç yanlış olmaz.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 1 4 1 5 9 2 6

```
6 2 9 5 1 4 1 3
```

Dizin: `alistirma/kod/II-1-A2/`

ZORLAYICI II-1-Z1

Kimse uyarmıyor, biri uyarıyor

Kitapçık `dizi[12]`'nin sessizce çöp okuduğunu gösterdi. `vector`'ün `at()` yöntemi ise bağırır: `out_of_range` adlı bir kuraldışı durum (*exception*) fırlatır. `d.at(2)`'yi yazın; sonra `d.at(12)`'yi bir `try` bloğuna alıp `catch (const out_of_range&)` ile yakalayın ve `sınır dışı!` yazın. `<stdexcept>` başlığı gerekir.

DÜŞÜNCE

`at()` sınırı denetler ve aşımda bir istisna fırlatır; onu yakalamak sizin işiniz.

YAKLAŞIM

`try { d.at(12); } catch (const out_of_range&) { ... }`; `<stdexcept>` başlığı. `at(2)` try'ın dışında da olur.

İLK SATIR

```
try { cout << d.at(12) << endl; } catch (const out_of_range& e) { cout << "sınır dışı!" << endl; }
```

çıktı — BEKLENEN-CİKTİ

```
at(2) = 4
at(12) = at(12): sınır dışı!
```

Dizin: `alistirma/kod/II-1-Z1/`

ZORLAYICI II-1-Z2

Sütun toplamları

İki boyutlu akıllı dizi: önce satır ve sütun sayısı, sonra tablo. Her sütunun toplamını tek satırda yazın. Girdi `2 3`, sonra `1 2 3` ve `4 5 6`. Kitapçıktaki `vector<vector<int>> tablo(satir, vector<int>(sutun, 0))` kalıbını kullanın; döngülerin hangisinin dışta olacağını düşünün.

DÜŞÜNCE

Tabloyu okuduktan sonra her sütun için satırları dolaşıp toplayın.

YAKLAŞIM

Dış döngü sütun, iç döngü satır: `tablo[s][c]`. Okurken ise dış döngü satır. Toplamları ayrı bir `vector<int>`'te tutmak da olur.

İLK SATIR

```
for (int c{0}; c < sutun; ++c) { int t{0}; for (int s{0}; s < satir; ++s) t += tablo[s][c]; ... }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 2 3 1 2 3 4 5 6

```
Sütun toplamları: 5 7 9
```

Dizin: `alistirma/kod/II-1-Z2/`

KEŞİF II-1-K1 Eratosthenes'in kalburu

Project Euler 10: iki milyondan küçük asalların toplamı. `I-4-A2`'deki `asalMi` ile tek tek denemek dakikalar sürer. Kalbur: `vector<bool> asal(2000000, true)`; 2'den başlayıp her asalın katlarını eleyin. Toplam `int`'e sığmaz; I-2'yi hatırlayın. Kaç saniye sürdüğünü de ölçün (Bölüm III'te öğreneceksiniz; şimdilik duvar saatiyle).

> **DÜŞÜNCE**
>
> Her asalın katlarını işaretleyin; işaretlenmeyenler asaldır.

> **YAKLAŞIM**
>
> `for (int p{2}; p < sinir; ++p) if (asal[p]) for (int k{p * 2}; k < sinir; k += p) asal[k] = false;` — iç döngü `p * p`'den başlarsa daha hızlı; `long long` toplam.

> **İLK SATIR**
>
> `for (long long k{p * p}; k < sinir; k += p) asal[k] = false;`

çıktı — BEKLENEN-CİKTİ

```
Project Euler, onuncu problem: 142913828922
```

Dizin: `alistirma/kod/II-1-K1/`

II.2

Eşlem: en çok işinize yarayacak kap

Kitapçığın harf sayan `11-esleme.cpp` programıyla başlıyoruz. Üçüncü ve dördüncü alıştırma kitapçığın *Klasik tuzak* kutusunu kurup bozduruyor; Keşif, bölüm sonundaki sözlük alıştırması.

1 2 2 1

> **YANIT YERİNE**
>
> kitapçık II.2: Eşlem ders 9: eşlemlere giriş ders 12: eşlemlere devam kod: 11-esleme.cpp

ISINMA II-2-I1

Başka bir sözcük

`yazi`'yi `"kelebek kanadi"` yerine `"algoritma"` yapın. Kaç ayrı harf çıkar? Sondaki `z` ve `q` satırları ne olur? Çalıştırmadan önce harfleri elle sayın.

çıktı — BEKLENEN-CİKTİ

```
a -> 2
g -> 1
i -> 1
l -> 1
m -> 1
o -> 1
r -> 1
t -> 1
kaç ayrı harf: 8
z kaç kere: 0
kaç ayrı harf: 9  <- arttı!
q var mı: hayır
kaç ayrı harf: 9  <- değişmedi
```

Dizin: `alistirma/kod/II-2-I1/`

ALIŞTIRMA II-2-A1

Sözcük sayacı

Harf yerine sözcük: girdi bitene kadar sözcük okuyun (`string`), `map<string, int>` ile sayın, `sözcük -> adet` biçiminde yazın. Girdi `elma armut elma kiraz armut elma`. Çıktının alfabetik gelmesi için hiçbir şey yapmanız gerekmiyor; neden?

çıktı — BEKLENEN-CİKTİ · GİRDİ: ELMA ARMUT ELMA KİRAZ ARMUT ELMA

```
armut -> 2
elma -> 3
kiraz -> 1
```

Dizin: `alistirma/kod/II-2-A1/`

ALIŞTIRMA II-2-A2

Plaka defteri

Üç şehrin plakası bir eşlemde: Ankara 6, İstanbul 34, İzmir 35. Girdiden şehir adları okuyun; her biri için `şehir: plaka` ya da `şehir: yok` yazın. Girdi `İzmir` ve `Bursa`. `[]` ile *okumayın*; neden olmadığını bir sonraki alıştırma gösterecek.

çıktı — BEKLENEN-CİKTİ · GİRDİ: İZMİR BURSA

```
İzmir: 35
Bursa: yok
```

Dizin: `alistirma/kod/II-2-A2/`

ZORLAYICI İKİLİ II-2-Z1

Tuzağı kuran program

Aşağıdaki program kaç ayrı şehir sorulduğunu saymak istiyor ama yanlış sayıyor: `[]` ile okuduğu her şehri eşleme ekliyor. Önce olduğu gibi çalıştırıp yanlış sayıyı görün, sonra `contains` ile düzeltin. Girdi `İzmir Bursa Ankara Bursa` ; doğru yanıt üç şehir sorulmuş, ikisi defterde.

DÜŞÜNCE

`plaka[sehir]` olmayan şehri 0 değerle eşleme ekler; sayım bu yüzden şişiyor.

YAKLAŞIM

Okumak için `plaka.contains(sehir)` (ya da `find`); `[]` 'yi yalnız yazarken kullanın. Değişiklikten sonra `plaka.size()` 'ı da yazdırıp farkı görün.

İLK SATIR

```
if (plaka.contains(sehir)) ++bulunan;
```

ana.cpp — BAŞLANGIÇ

```cpp
// II-2-Z1 · Tuzağı kuran program
#include <iostream>
#include <map>
#include <set>
#include <string>
using namespace std;

int main() {
  map<string, int> plaka{{"Ankara", 6}, {"İstanbul", 34}, {"İzmir", 35}};
  set<string> sorulan;
  int bulunan{0};
  string sehir;
  while (cin >> sehir) {
    sorulan.insert(sehir);
    if (plaka[sehir] != 0) ++bulunan;      // tuzak burada
  }
  cout << "sorulan ayrı şehir: " << sorulan.size() << endl;
  cout << "defterde bulunan  : " << bulunan << endl;
  cout << "defterin boyu     : " << plaka.size() << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: İZMİR BURSA ANKARA BURSA

```
sorulan ayrı şehir: 3
defterde bulunan  : 2
defterin boyu     : 3
```

Dizin: `alistirma/kod/II-2-Z1/`

ZORLAYICI II-2-Z2

Anagram öbekleri

Harfleri aynı olan sözcükler bir öbek: `kalem`, `elmak`, `emlak`. Anahtar, harfleri sıralanmış sözcük (`<algorithm>` içindeki `sort`); değer, o anahtara düşen sözcüklerin `vector`'ü. Kitapçığın “bir şeyden bir şey kümesine eşlem” dediği yapı. Girdi `kalem elmak lame emlak masa sama`; her öbek bir satır; satırlar anahtar sırasıyla (eşlem sıralar), öbek içi geliş sırasıyla.

DÜŞÜNCE

Anagramların ortak anahtarı var: harfleri sıralanmış sözcük.

YAKLAŞIM

Her sözcüğün kopyasını `sort(k.begin(), k.end())` ile sıralayıp anahtar yapın; `obek[anahtar].push_back(sozcuk)`. Eşlem anahtarları sıralı tutar.

İLK SATIR

```
string anahtar{sozcuk}; sort(anahtar.begin(), anahtar.end());
obek[anahtar].push_back(sozcuk);
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: KALEM ELMAK LAME EMLAK MASA SAMA

```
aams: masa sama
aeklm: kalem elmak emlak
aelm: lame
```

Dizin: `alistirma/kod/II-2-Z2/`

KEŞİF II-2-K1 Çok tanımlı sözlük

Kitapçığın birinci alıştırması: `map<string, vector<string>>` ile bir sözcüğün birden çok anlamı olsun. Girdi komut satırları: `ekle sözcük anlam` ve `ara sözcük`. Arama, anlamları virgülle ayırıp yazsın; sözcük yoksa `yok`. Bir `ekle` bir de `ara` işlevi yazın; `main` yalnız komutları dağıtsın. Sonra kendiniz büyütün: `sil` komutu, anlam sayısı, en çok anlamlı sözcük.

DÜŞÜNCE

Bir sözcüğün birden çok anlamı: değer bir yazı değil, yazı dizisi.

YAKLAŞIM

`map<string, vector<string>>`; `ekle` `push_back` yapar, `ara` önce `contains` ile bakar sonra virgülle yazar. Komutu okuyup `if (komut == "ekle")` ile dağıtın.

İLK SATIR

```
void ekle(Sozluk& s, const string& sozcuk, const string& anlam) { s[sozcuk].push_back(anlam); }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: EKLE KALEM YAZMA_ARACI EKLE KALEM TÜKENMEZ ARA KALEM ARA DEFTER

```
kalem: yazma_aracı, tükenmez
defter: yok
```

Dizin: `alistirma/kod/II-2-K1/`

II.3

Küme ve hızlı küme işlemleri

Küme, eşlemin değeri atılmış hâli: sıralı, tekrarsız. Üyelik sorusu, kesişim, birleşim; sonra kitapçığın “bir şeyden bir şey kümesine eşlem” yapısı. Keşif, depodaki “sayı kümesini ikiye bölmek” yazısına çıkıyor.

1 2 1 1

YANIT YERİNE

kitapçık II.2: Küme, eşlemin küçük kardeşi hızlı küme işlemleri sayı kümesini ikiye bölmek ders 9

ISINMA II-3-I1

Harf kümesi

Program bir yazının harflerini kümeye atıp yazıyor: tekrarlar düşüyor, sıra alfabetik. `yazi`'yi `"programlama"` yapın; kaç ayrı harf?

ana.cpp — BAŞLANGIÇ

```cpp
// II-3-I1 · Harf kümesi: yazi -> "programlama"
#include <iostream>
#include <set>
#include <string>
using namespace std;

int main() {
  string yazi{"kelebek kanadi"};
  set<char> harfler;
  for (char h : yazi)
    if (h != ' ') harfler.insert(h);
  for (char h : harfler) cout << h;
  cout << endl << harfler.size() << " ayrı harf" << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
aglmopr
7 ayrı harf
```

Dizin: `alistirma/kod/II-3-I1/`

ALIŞTIRMA II-3-A1

Tekrar var mı?

Kitapçığın üçüncü alıştırmasının küme sürümü: sayıları okurken kümeye ekleyin; `insert` bir çift döndürür, ikincisi “yeni miydi” der. İlk tekrar edeni yazın, yoksa `Tekrar yok`. Girdi `3 1 4 1 5`. Öteki iki yöntemi (sıralama, iç içe döngü) Bölüm III'te ölçeceksiniz.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 1 4 1 5

```
Tekrar eden: 1
```

Dizin: `alistirma/kod/II-3-A1/`

ALIŞTIRMA II-3-A2

Kesişim, birleşim, fark

İki küme okuyun: önce boy, sonra ögeler; iki kere. Üç satır: kesişim, birleşim, birinciden ikincinin farkı. Girdi `4` / `1 4 6 9` / `3` / `4 9 10`. `contains` ile döngü yeter; `<algorithm>`'daki `set_intersection` sonra.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 1 4 6 9 3 4 9 10

```
Kesişim: 4 9
Birleşim: 1 4 6 9 10
Fark: 1 6
```

Dizin: `alistirma/kod/II-3-A2/`

ZORLAYICI II-3-Z1

Bölenler eşlemi

Kitapçıktaki `map<int, set<int>>` yapısı: 1'den 12'ye her sayı için bölenlerinin kümesi. Kurduktan sonra iki soru: tam iki böleni olanlar hangileri (asallar), en çok böleni olan hangisi? İkinci soruda eşitlik olursa küçüğü seçin. Çıktı iki satır.

DÜŞÜNCE

Eşlem kurulu; iki soru iki döngü: kümesinin boyu 2 olanlar, boyu en büyük olan.

YAKLAŞIM

`for (const auto& [n, kume] : bolenler)` ile gezin; `kume.size() == 2` asal demek. En çok için `>` ile karşılaştırın ki eşitlikte küçük kalsın.

İLK SATIR

```
if (kume.size() > enCok) { enCok = kume.size(); hangisi = n; }
```

çıktı — BEKLENEN-CİKTİ

```
asallar: 2 3 5 7 11
en çok böleni olan: 12 (6 bölen)
```

Dizin: `alistirma/kod/II-3-Z1/`

KEŞİF II-3-K1 İkiye bölünür mü?

Bir sayı kümesi iki parçaya, toplamları eşit olacak biçimde bölünebilir mi? Depodaki *sayı kümesini ikiye bölmek* yazısı bu sorunun peşinde. Burada küçük bir başlangıç: en çok 20 sayı; her sayının "birinci parçada mı" kararını bir bit olarak düşünün, `0`'dan `2ⁿ−1`'e bütün desenleri deneyin. Girdi `5` / `3 1 4 2 2`; çıktı `Evet, 6 ve 6` ya da `Hayır`. Bölüm IV bunu özyinelemeyle yeniden yapacak.

DÜŞÜNCE

Her sayı için iki seçenek; *n* sayı için 2^n desen; desen bir sayının bitleri.

YAKLAŞIM

`for (int desen{0}; desen < (1 << n); ++desen)`; k'ıncı bit açıksa (`desen & (1 << k)`) sayı birinci parçada. İki toplamı karşılaştırın, ilk eşitlikte durun.

İLK SATIR

```
if (desen & (1 << k)) birinci += d[k]; else ikinci += d[k];
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 3 1 4 2 2

```
Evet, 6 ve 6
```

Dizin: `alistirma/kod/II-3-K1/`

II.4

Sıranın önemli olduğu kaplar

Yığın, kuyruk, öncelik sırası: aynı iki işlem, üç farklı çıkış kuralı. Kitapçığın `12-kaplar.cpp` programı ve *Deneyin* kutusuyla başlıyoruz; dengeli parantez alıştırması iki basamağa yayılıyor.

1 2 1 1

YANIT YERİNE

kitapçık II.3: Sıranın önemli olduğu kaplar ders 21: yığın, kuyruk, öncelik sırası ders 22: std::stack kod: 12-kaplar.cpp

ISINMA II-4-I1

Küçükten büyüğe

Kitapçığın *Deneyin* kutusu: `priority_queue`'nun üçüncü tür girdisine `greater<int>` verin: `priority_queue<int, vector<int>, greater<int>>`. `<vector>` ve `<functional>` başlıkları gerekebilir. Yalnız öncelik satırı değişmeli.

çıktı — BEKLENEN-CİKTİ

```
yığın  : 5 1 4 1 3
kuyruk : 3 1 4 1 5
öncelik: 1 1 3 4 5
```

Dizin: `alistirma/kod/II-4-I1/`

ALIŞTIRMA II-4-A1

Parantez: tek sayaçla

Kitapçığın ikinci alıştırmasının ilk yarısı: yalnız `(` ve `)`. Her satır bir yazı; `dengeli` ya da `dengesiz` yazın. Yığın gerekmez: bir sayaç, açılınca artır, kapanınca azalt; sıfırın altına inerse ya da sonda sıfır değilse dengesiz. Girdi `(()())`, `(()`, `)(`.

çıktı — BEKLENEN-CİKTİ · GİRDİ: (()()) (())(

```
dengeli
dengesiz
dengesiz
```

Dizin: `alistirma/kod/II-4-A1/`

ALIŞTIRMA II-4-A2

Parantez: üç çeşit, yığınla

İkinci yarı: `()`, `[]`, `{}`. Tek sayaç artık yetmez: `([)]` sayaçla dengeli görünür ama değil. Açılanı yığına atın; kapanan gelince tepedekiyle eşleşiyor mu bakın. Girdi `([]{})`, `([)]`, `{[}`. Sayaç neden yetmedi, bir cümleyle yazın.

çıktı — BEKLENEN-CİKTİ · GİRDİ: ([]{}) ([)] {[}

```
dengeli
dengesiz
dengesiz
```

Dizin: `alistirma/kod/II-4-A2/`

ZORLAYICI İKİLİ II-4-Z1

Acil servis sırası

Hastalar numarayla geliyor, her birinin bir aciliyet derecesi var (büyük önce). Aynı derecedekilerde önce gelen önce. `priority_queue<pair<int,int>>` çiftleri önce birinci, eşitse ikinci üyeye göre sıralar; ikinci üyeyi öyle seçin ki önce gelen kazansın (ipucu: geliş sırasının eksilisi). Girdi `5`, sonra `derece numara` çiftleri. Çıktı: numaralar çağrılma sırasıyla, tek satır.

DÜŞÜNCE

Öncelik sırası en büyük çifti önce verir; eşit derecede önce gelen önce çıkmalı.

YAKLAŞIM

Çiftin ikinci üyesi geliş sırasının eksilisi olsun: daha önce gelen daha büyük (daha az eksi). Numara ayrıca bir eşlemde tutulur ya da üçlü kullanılır.

İLK SATIR

```
sira.push({derece, -k}); numara[k] = no;
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 2 101 5 102 2 103 9 104 5 105

```
Çağrılma sırası: 104 102 105 101 103
```

Dizin: `alistirma/kod/II-4-Z1/`

KEŞİF II-4-K1 Ters Lehçe hesap makinesi

`3 4 + 2 *` yazımında işleç sayılardan sonra gelir; parantez gerekmez. Yığınla hesaplanır: sayı gelince yığına, işleç gelince iki sayı çek, işle, sonucu yığına. Girdi bir satır; sonucu yazın. Girdi `3 4 + 2 *`. Bölme ve çıkarmada sıraya dikkat: önce çekilen sağdaki. Sonra kendiniz büyütün: hatalı girdi, kesirli sayılar, `^`.

DÜŞÜNCE

Sayı gelince yığına, işleç gelince iki sayı çekip işle; sonuç tek sayı kalır.

YAKLAŞIM

Sözcüğü okuyun; işleçse `b = y.top(); y.pop(); a = y.top(); y.pop();` sonra `a op b` (sıra önemli); değilse `stoll` ile sayıya çevirip `push`.

İLK SATIR

```
if (s == "+" or s == "-" or s == "*" or s == "/") { ... } else y.push(stoll(s));
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 4 + 2 *

```
Sonuç: 14
```

Dizin: `alistirma/kod/II-4-K1/`

II.5

Kendi türünüzü yazmak

Kitapçığın `Konum` türüyle başlıyoruz; kendi `Kesir` türünüze ve küçük bir robota kadar gidiyoruz. İşlemci tanımlamak (`operator<<`, `operator+`) burada alışkanlık olsun.

1 2 1 1

YANIT YERİNE

kitapçık II.4: Kendi türünüzü yazmak sınıf ve yapı kavramı ders 10: soyutlama, Araba türü ders 11: yapıcılar, operator<< kod: 13-konum.cpp

ISINMA II-5-I1

Başka bir nokta

`Konum k{3, 4}`'ü `{6, 8}` yapın. Uzaklık neden tam 10 çıkıyor? Kaydırmadan sonraki uzaklığı çalıştırmadan önce yaklaşık tahmin edin.

çıktı — BEKLENEN-CİKTİ

```
(6, 8) başlangıca uzaklığı: 10
(8, 10) şimdi uzaklığı: 12.8062
```

Dizin: `alistirma/kod/II-5-I1/`

ALIŞTIRMA II-5-A1

Dikdörtgen

`struct Dikdortgen`: `en` ve `boy`; `alan()` ve `cevre()` yöntemleri, ikisi de `const`. Girdiden `en boy` çiftleri okuyun, her biri için bir satır. Girdi `3 4` ve `10 2`.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 4 10 2

```
3x4: alan 12, çevre 14
10x2: alan 20, çevre 24
```

Dizin: `alistirma/kod/II-5-A1/`

ALIŞTIRMA II-5-A2

Konum artı Konum

`Konum` için `operator+` yazın: iki konumun toplamı, bileşenleri toplanmış yeni bir konum. Üye olmayan bir işlev olarak, `operator<<`'ın yanına. Sonra `(3, 4) + (1, 2) = (4, 6)` satırını yazdırın. `const Konum&` almanın nedenini II.7'de göreceksiniz.

çıktı — BEKLENEN-CİKTİ

```
(3, 4) + (1, 2) = (4, 6)
```

Dizin: `alistirma/kod/II-5-A2/`

ZORLAYICI II-5-Z1

Kendi Kesir türünüz

Kitapçığın dördüncü alıştırması. Pay ve payda; yapıcı kesri sadeleştirsin (I-4-Z1'deki `ebob`); `+`, `*` ve `<<`. İki satır yazdırın; ikincisi `2/4` ile `3/6`'dan kurulsa da çıktıda `1/2 * 1/2 = 1/4` görünmeli, çünkü yapıcı daha baştan sadeleştirir. Yapıcı sözdizimi: `Kesir(int p, int q) : pay{p}, payda{q} { ... }`. Payda sıfır gelirse ne yapacağınızı şimdilik yorum satırına yazın.

DÜŞÜNCE

Yapıcı kesri en baştan sadeleştirir; sonra `+`, `*` ve `<<` olağan tanımlar.

YAKLAŞIM

Yapıcıda `ebob(pay, payda)` ile ikisini de bölün; `operator+` yeni bir `Kesir` döndürsün (yapıcı yine sadeleştirir). Payda eksi gelirse işareti paya taşımayı düşünün.

İLK SATIR

```
Kesir operator+(const Kesir& a, const Kesir& b) { return {a.pay * b.payda + b.pay * a.payda, a.payda * b.payda}; }
```

çıktı — BEKLENEN-CİKTİ

```
1/2 + 1/3 = 5/6
1/2 * 1/2 = 1/4
```

Dizin: `alistirma/kod/II-5-Z1/`

KEŞİF II-5-K1

Küçük robot ~bir hafta

Kitapçığın `enum class`'ı iş başında. Bir robot `(0, 0)`'da, kuzeye bakıyor. Komut dizisi okuyun: `I` bir adım ileri, `D` sağa dön. Sonunda konumunu ve yönünü yazın. `enum class Yon { Kuzey, Dogu, Guney, Bati }`; sağa dönmek bir sonraki yöne geçmek (Batı'dan sonra Kuzey). Girdi `IIDIDDI`. Sonra büyütün: `S` sola dön, `G` geri, izlediği yolu çizdirin.

DÜŞÜNCE

Robotun durumu üç şey: x, y ve yön; her komut bunlardan birini değiştirir.

YAKLAŞIM

`enum class` değerini `int`'e çevirip bir artırıp 4'e göre kalan alarak dönün: `Yon((int(yon) + 1) % 4)`. İleri için yöne göre `switch`.

İLK SATIR

```
void sagaDon() { yon = Yon((int(yon) + 1) % 4); }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: IIDIDDI

```
Konum (0, 2), yön Batı
```

Dizin: `alistirma/kod/II-5-K1/`

II.6

Kalıplar: tür de bir değişken olabilir

Kitapçığın `14-kalip.cpp` programındaki `ekle` ve `enBuyuk` kalıplarıyla başlayıp kendi kap kalıbınıza kadar gidiyoruz.

1 1 1 1

YANIT YERİNE

kitapçık II.5: Kalıplar ders 12: işlev ve tür kalıpları generic-containers kod: 14-kalip.cpp

ISINMA II-6-I1

Yazılar da karşılaştırılır

`enBuyuk('a', 'z')` satırını `enBuyuk(string("elma"), string("armut"))` yapın. Hangisi büyük çıkıyor, neden? Yorumdaki `ekle("ke", "lebek")` satırını da bir kere açıp derleyicinin ne dediğini okuyun, sonra geri kapatın.

çıktı BEKLENEN-CİKTİ

```
7
7.75
kelebek
elma
```

Dizin: `alistirma/kod/II-6-I1/`

ALIŞTIRMA II-6-A1

Her türden toplam

`template <typename T> T toplam(const vector<T>& d)`: ögeleri toplayıp döndürsün. Bir `vector<int>{1,2,3,4,5}` ve bir `vector<double>{1.5, 2.5, 3.5}` ile çağırın. Toplamı hangi değerle başlatacaksınız: `0` mı, `T{}` mi? İkisini de deneyin.

çıktı BEKLENEN-CİKTİ

```
15
7.5
```

Dizin: `alistirma/kod/II-6-A1/`

ZORLAYICI II-6-Z1

Kutu ve Konum

Kitapçığın `Kutu<X>` tür kalıbını yazın; sonra `Kutu<int> k1{42}` ve kendi `Konum` türünüzle `Kutu<Konum> k2{{3, 4}}`. İkisinin `al()` çıktısını yazdırın. `Konum` için `operator<<` gerekiyor; II-5-A2'den alın. Bir de takma ad: `template <typename T> using Dizi = vector<T>;` deyip `Dizi<Kutu<int>>` kurabiliyor musunuz?

DÜŞÜNCE

Tek bir değeri saran kalıp tür: içinde `X` türünde bir alan ve onu veren `al()`.

YAKLAŞIM

`template <typename X> struct Kutu { X icerik; X al() const { return icerik; } };` — `Kutu<Konum> k2{{3, 4}}` iç süslü parantezle Konum'u kurar.

İLK SATIR

```
template <typename X> struct Kutu { X icerik; ... };
```

çıktı — BEKLENEN-CİKTİ

```
42
(3, 4)
```

Dizin: `alistirma/kod/II-6-Z1/`

KEŞİF II-6-K1 Kendi yığınınız

`std::stack`'i siz yazın: `template <typename T> class Yigin`, içinde bir `vector<T>`; `koy`, `cikar`, `tepe`, `bosMu`. Bir `Yigin<int>`'e 3, 1, 4 koyup boşalana kadar yazdırın; bir `Yigin<string>`'e `bir`, `iki` koyup aynısını yapın. Sonra II-4-A2'deki parantez denetimini kendi yığınınızla çalıştırın: değişen tek şey tür adı olmalı. Depodaki `generic-containers` dizini aynı fikrin daha büyük hâli.

DÜŞÜNCE

Yığın aslında sonundan eklenip çıkarılan bir `vector`.

YAKLAŞIM

`koy` → `push_back`, `cikar` → `pop_back`, `tepe` → `back()`, `bosMu` → `empty()`. Kalıp parametresi `T` her yere.

İLK SATIR

```
template <typename T> class Yigin { vector<T> v; public: void koy(const T& x) { v.push_back(x); } ... };
```

çıktı — BEKLENEN-CİKTİ

```
4 1 3
iki bir
```

Dizin: `alistirma/kod/II-6-K1/`

II.7

Adres ve takma ad

Kitapçığın `15-adres.cpp` programı: üç `ekle`, üç sonuç. Başlangıç dosyasında son satır yok, adres her çalıştırmada değişir. Buradan değiş tokuşa, `const Dizi&` alışkanlığına ve adres aritmetiğine.

`1` `2` `1`

YANIT YERİNE

kitapçık II.6: Adres ve takma ad adres değişkenleri ders 1-4 (üçüncü ders) algoritma ders 27 kod: 15-adres.cpp

ISINMA II-7-I1

Kopyayı takma ada çevirin

`ekleK`'nin girdisini `Sayi k` yerine `Sayi& k` yapın; başka hiçbir şeye dokunmayın. Üç satır nasıl değişir? Tek bir `&` imgesiyle "etkisiz" işlev neden etkili oldu?

çıktı — BEKLENEN-CİKTİ

```
kopyadan sonra   : 20
adresten sonra   : 30
takma addan sonra: 40
```

Dizin: `alistirma/kod/II-7-I1/`

ALIŞTIRMA II-7-A1

Değiş tokuş

`void degisTokus(int& a, int& b)` : iki değişkenin değerlerini takas etsin. Üçüncü bir değişken gerekir. Girdi `3 7`, çıktı `7 3`. Aynı işlevi `int a, int b` ile yazıp neden işe yaramadığını görün; `std::swap`'ı sonra söyleyeceğiz.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 7

```
7 3
```

Dizin: `alistirma/kod/II-7-A1/`

ALIŞTIRMA II-7-A2

Kopyalamayın, değiştirmeyin

Kitapçığın *Püf noktası*: iki işlev yazın. `int toplam(const vector<int>& d)` kabı kopyalamadan toplasın; `void ikiyeKatla(vector<int>& d)` her ögeyi yerinde iki katına çıkarsın. Dizi `{3, 1, 4, 1, 5}`; toplam, katlanmış dizi, yeni toplam: üç satır. `toplam` içinde bir ögeyi değiştirmeyi deneyin; derleyici ne der?

çıktı — BEKLENEN-CİKTİ

```
toplam 14
6 2 8 2 10
toplam 28
```

Dizin: `alistirma/kod/II-7-A2/`

ZORLAYICI II-7-Z1

Adresle yürümek

Temel dizide adres aritmetiği: `int* p = dizi;` ile başlayıp `p != dizi + n` olana kadar `++p`; her adımda `*p`'yi yazın. Sonra `*(dizi + 3)` ile `dizi[3]`'ün aynı şey olduğunu gösterin. `++p` adresi kaç bayt ilerletiyor? Kitapçığın `10-dizi` çıktısındaki "ikincininki" satırıyla bağlayın.

DÜŞÜNCE

Adres bir konum; `++p` bir sonraki ögeye, yani `sizeof(int)` bayt ileri.

YAKLAŞIM

`for (int* p{dizi}; p != dizi + n; ++p) cout << *p;` — adresleri `cout << p` ile yazdırıp iki ardışık adresin farkına bakın.

İLK SATIR

```
for (int* p{dizi}; p != dizi + n; ++p) cout << *p << " ";
```

çıktı — BEKLENEN-CİKTİ

```
3 1 4 1 5 9 2 6
dördüncü öge: 1 = 1
```

Dizin: `alistirma/kod/II-7-Z1/`

II.8

Geri dönüş: taşma ve tahmin oyunu, kaplarla

Unutma eğrisi tam burada çalışır: I. bölümde öğrendiğiniz iki şey, bu bölümün kaplarıyla bir daha. Kısa iki soru; ikisi de yirmi dakikalık.

2

YANIT YERİNE

kitapçık I.1: Her türün bir sınırı var kitapçık I.3: İlk oyununuz kitapçık II.2: Eşlem

ALIŞTIRMA II-8-A1

Toplamın taştığı yer

I. bölümün taşması, bu sefer bir dizide. Girdi: *n*, sonra *n* tane artı tam sayı (her biri `int`'e sığar). Sayıları baştan toplarken ara toplam ilk kez `2147483647`'yi hangi sayıda aşıyor? Toplamı `long long`'da tutun; `int`'te tutsaydınız ne olurdu, bir yorum satırıyla yazın. Çıktı: `toplam 3. sayıda int'i aştı` ve son satırda gerçek toplam; hiç aşmıyorsa `taşma yok`.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 2000000000 100000000 50000000 1 1

```
toplam 3. sayıda int'i aştı
toplam: 2150000002
```

Dizin: `alistirma/kod/II-8-A1/`

ALIŞTIRMA II-8-A2

Skor tablosu

Tahmin oyununun oyuncuları ve kaç tahminde bildikleri satır satır geliyor: `ada 5`, `can 7`... Girdi bitene kadar okuyun (`while (cin >> ad >> sayi)`), her oyuncunun **en iyi** (en küçük) sonucunu bir `map<string,int>`'te tutun ve sonunda tabloyu yazdırın: `ada: en iyi 4`. Eşlem adları kendiliğinden sıraladı, siz bir şey yapmadınız; sırayı tahmin sayısına göre vermek isteseydiniz ne yapardınız?

çıktı — BEKLENEN-CİKTİ · GİRDİ: ADA 5 CAN 7 ADA 4 ECE 6 CAN 9

```
ada: en iyi 4
can: en iyi 7
ece: en iyi 6
```

Dizin: `alistirma/kod/II-8-A2/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

"Bu bölümü geçtim mi?" sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` "Doğru!" desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

- `II-2-Z2` Anagram öbekleri
- `II-4-Z1` Acil servis sırası
- `II-3-K1` İkiye bölünür mü?

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.

BÖLÜM III

Programcının Zanaatı

Kitapçığın üçüncü bölümünde yeni bir C++ kavramı yok; alışkanlık var. Burada da öyle: terminal, make, kendi kendini sınayan program, hata avı, ölçüm ve sadeleştirme. Bu bölümün alıştırmaları tarayıcıda yapılmaz; kendi bilgisayarınızda bir terminal gerekiyor. Windows'ta WSL (Ubuntu) açın: `g++`, `make` ve `diff` orada hazır, kurulum tarifi kitapçığın kapağında. Kitapçığın kapağındaki Tezgâhınızı kurun bölümü bunun için.

Hazırlık · I. İlk Adımlar · II. Veriyi Düzenlemek · III. Zanaat · IV. Özyineleme · V. Çizgeler · VI. Dinamik Programlama

III.1

Tarayıcıdan terminale

Dört alıştırma, dördü de terminalde. Girdiyi dosyadan vermek, derleyici uyarılarını okumak, derleme ile bağlamayı ayırmak ve komut satırından girdi almak.

1 2 1

YANIT YERİNE

kitapçık III.1: Tarayıcıdan terminale ders 5: derleyici ve terminal ders 10: geliştirme ortamları kopya sayfaları ders 13: int main(int, char**)

ISINMA III-1-I1

Girdiyi dosyadan

Program girdi bitene kadar sayı okuyup kaç tane olduğunu ve toplamını yazıyor. Dizindeki `girdi` dosyasında kırk sayı var. Önce elle: `./prog` deyip birkaç sayı yazın, `Ctrl-D` ile bitirin. Sonra `./prog < girdi`. Sonra `make test`. Üçü aynı programı çalıştırıyor; fark yalnız girdinin nereden geldiği. Son olarak bir satır ekleyin: en büyük sayı.

ana.cpp — BAŞLANGIÇ

```cpp
// III-1-I1 · Girdiyi dosyadan
// ./prog < girdi ile çalıştırın; sonra "En büyük: ..." satırı ekleyin
#include <iostream>
using namespace std;

int main() {
  int x{0}, adet{0};
  long long toplam{0};
  while (cin >> x) {
    ++adet;
    toplam += x;
  }
  cout << "Adet: " << adet << endl;
  cout << "Toplam: " << toplam << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 37 74 10 47 84 20 57 94 30 67 3 40 77 13 50 87 23 60 97 33 70 6 43 80 16 53 90 26 63 100 36 73 9 46 83 19 56 93 29 66

```
Adet: 40
Toplam: 2060
En büyük: 100
```

Dizin: `alistirma/kod/III-1-I1/`

ALIŞTIRMA III-1-A1

-Wall ne diyor?

Program derleniyor ve doğru çıktı veriyor; ama bu dizinin `Makefile`'ı `-Wall -Werror` ile derliyor: her uyarı hata sayılır. Üç uyarı var. Her birini okuyun, nedenini anlayın, çıktıyı değiştirmeden giderin. Kitapçığın püf noktası: uyarılar bedava kod incelemesidir.

ana.cpp — BAŞLANGIÇ

```cpp
// III-1-A1 · -Wall ne diyor?
// Makefile -Wall -Werror ile derliyor: uyarıları giderin, çıktı aynı kalsın
#include <iostream>
#include <vector>
using namespace std;

int kacTane(const vector<int>& d, int aranan) {
  int sayac{0};
  for (int i{0}; i < d.size(); ++i)
    if (d[i] == aranan) ++sayac;
  return sayac;
}

int main() {
  vector<int> d{3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5};
  int gereksiz;
  int bes{kacTane(d, 5)};
  if (bes = 3) cout << "5 üç kere geçiyor" << endl;
  else         cout << "5 başka sayıda geçiyor" << endl;
  return 0;
}
```

çıktı — BEKLENEN-ÇİKTİ

```
5 üç kere geçiyor
```

Dizin: `alistirma/kod/III-1-A1/`

ALIŞTIRMA III-1-A2

İki adımda derleme

Dizinde üç dosya var: `ana.cpp`, `yardimci.cpp`, `yardimci.h`. `Makefile`'ın `derle` kuralı boş bırakıldı. Önce elle, kitapçıktaki gibi iki adımda derleyin: her `.cpp` için `-c` ile bir `.o`, sonra `.o`'ları `prog`'a bağlayın. Çalışınca aynı komutları `derle` kuralına yazın (sekmeyle!). `make test` geçmeli. `ls` ile `.o` dosyalarını görün; `make temizle` onları da silsin.

çıktı — BEKLENEN-ÇİKTİ

```
toplam   : 31
en büyük : 9
```

Dizin: `alistirma/kod/III-1-A2/` — dizinde ayrıca: `yardimci.cpp`, `yardimci.h`

ZORLAYICI III-1-Z1

Komut satırından girdi

Girdi her zaman `cin`'den gelmez; `./prog 3 14 15 92` gibi komutun yanına da yazılır. `int main(int argc, char** argv)`: `argc` kaç parça olduğu (programın adı dahil), `argv[k]` k'ıncı parça, yazı olarak. `stoi` ile sayıya çevirin; adedi ve toplamı yazın. Hiç sayı verilmezse `Kullanım: ./prog sayı sayı ...` yazıp çıkın. `make test` dört sayıyla çağırıyor; `./prog`'u boş da deneyin. Depodaki `sablon/ana.cpp` aynı kalıbı kullanıyor.

DÜŞÜNCE

`argv` yazı dizisi; `argv[0]` programın adı, gerisi girdi.

YAKLAŞIM

`argc < 2` ise kullanım yazıp dön; değilse `for (int k{1}; k < argc; ++k) toplam += stoi(argv[k]);`.

İLK SATIR

```
if (argc < 2) { cout << "Kullanım: ./prog sayı sayı ..." << endl; return 1; }
```

çıktı — BEKLENEN-CİKTİ

```
4 sayı, toplam 124
```

Dizin: `alistirma/kod/III-1-Z1/`

III.2

make: tek komutla her şey

Kitapçığın `asal-projesi` burada beş kez karşınıza çıkacak: önce gözlemleyin, sonra bozuk bir bağımlılığı düzeltin, bir dosya ekleyin, kuralları desenle yazın, en sonunda kendi şablonunuzu kurun.

1 2 1 1

YANIT YERİNE

kitapçık III.2: make kod: asal-projesi sablon/ dogrusal-cebir/Makefile ileri/dp/Makefile (ayrık derleme)

ISINMA III-2-I1 İki kere make

Kitapçığın asal projesi olduğu gibi. Sırayla: `make` (üç derleme komutu görürsünüz), bir daha `make` (hiçbir şey), `touch asal.h` sonra `make` (hangi dosyalar yeniden derlendi, neden ikisi birden?), `touch ana.cpp` sonra `make` (şimdi hangisi?). Gözlemlerinizi `ana.cpp` 'nin başına yorum olarak yazın. `make test` baştan geçer; bu alıştırmada iş değiştirmek değil, izlemek.

çıktı — BEKLENEN-CİKTİ

```
denemeler geçti.
360 sayısının asal çarpanları: 2 2 2 3 3 5
```

Dizin: `alistirma/kod/III-2-I1/` — dizinde ayrıca: `asal.cpp`, `asal.h`

ALIŞTIRMA III-2-A1 Eksik bağımlılık

Bu projede `ana.cpp`, başlık dosyasındaki `VARSAYILAN` sabitini kullanıyor. `Makefile` 'da bir satır eksik. Şunu yapın: `make`; sonra `asal.h` 'de `VARSAYILAN` 'ı 60'tan 84'e çevirin; `make`. Program hâlâ 60 diyor! Neden yeniden derlenmedi? `Makefile` 'daki eksik bağımlılığı bulup ekleyin; `make` bu sefer doğru dosyayı derlesin. `make test` 84'ü bekliyor.

çıktı — BEKLENEN-CİKTİ

```
denemeler geçti.
84 sayısının asal çarpanları: 2 2 3 7
```

Dizin: `alistirma/kod/III-2-A1/` — dizinde ayrıca: `asal.cpp`, `asal.h`

ALIŞTIRMA III-2-A2 Üçüncü dosya

`ana.cpp` artık `ebob.h` 'yi de içeriyor ve `ebob(360, 84)` 'ü yazdırıyor; ama o dosyalar yok ve `Makefile` onları bilmiyor. `ebob.h` ile `ebob.cpp` 'yi yazın (I-4-Z1'deki döngülü Öklid; başlık koruması `#ifndef` ile), `Makefile` 'a `ebob.o` kuralını ve bağımlılığını ekleyin. Hangi hedefler `ebob.o` 'ya bağlı olmalı?

çıktı — BEKLENEN-CİKTİ

```
denemeler geçti.
360 sayısının asal çarpanları: 2 2 2 3 3 5
ebob(360, 84) = 12
```

Dizin: `alistirma/kod/III-2-A2/` — dizinde ayrıca: `asal.cpp`, `asal.h`

ZORLAYICI III-2-Z1

Desen kuralı

Her `.cpp` için ayrı kural yazmak tekrar. `make`'in desen kuralı var: `%.o: %.cpp` diye tek kural, komutunda `$<` (ilk bağımlılık) ve `$@` (hedef). Değişkenler de: `CXXFLAGS = -std=c++23 -Wall`. Bu projenin `Makefile`'ını yeniden yazın: tek desen kuralı, kaynak listesi `$(wildcard *.cpp)`, nesne listesi `$(SRC:.cpp=.o)`. Başlık bağımlılığı desen kuralında nasıl söylenir? (Kitapçığın `kod/Makefile`'ı ve depodaki `ileri/dp/Makefile` örnek.) `make test` yine geçmeli.

DÜŞÜNCE

Tek desen kuralı bütün `.cpp`'leri kapsar; bağlama ayrı bir kural.

YAKLAŞIM

`SRC = $(wildcard *.cpp)`, `OBJ = $(SRC:.cpp=.o)`, `%.o: %.cpp` kuralında `$(CXX) $(CXXFLAGS) -c $< -o $@`; başlık bağımlılığı için `%.o: %.cpp %.h` ya da ayrı bir satır.

İLK SATIR

```
%.o: %.cpp\n\t$(CXX) $(CXXFLAGS) -c $< -o $@
```

çıktı — BEKLENEN-CİKTİ

```
denemeler geçti.
360 sayısının asal çarpanları: 2 2 2 3 3 5
```

Dizin: `alistirma/kod/III-2-Z1/` — dizinde ayrıca: `asal.cpp`, `asal.h`

KEŞİF III-2-K1 Kendi şablonunuz

Kitapçığın birinci alıştırması. Bu dizin bilerek yarım: `ana.cpp` var, `Makefile` yok, beklenen çıktı yok. Depodaki `sablon/` dizinine bakın, ama kopyalamayın; kendinizinkini yazın: `make`, `make test`, `make temizle`. Test hedefi `derle`'ye bağlı olsun (depodaki şablonun bir eksiği bu; CLAUDE.md'de yazıyor). Beklenen çıktıyı kendiniz üretip `beklenen-cikti`'ye kaydedin. Bundan sonraki her alıştırmayı bu şablondan açın.

DÜŞÜNCE

Şablon üç hedef: derle, test, temizle; test derlemeye bağımlı olsun.

YAKLAŞIM

`test: derle` satırı bağımlılığı kurar; beklenen çıktıyı bir kere kendiniz üretip dosyaya yönlendirin (`./prog > beklenen-cikti`), sonra her test `diff` eder.

İLK SATIR

```
test: derle\n\t./prog > test-ciktisi\n\tdiff beklenen-cikti test-ciktisi
```

Dizin: `alistirma/kod/III-2-K1/` — beklenen çıktı yok

III.3

Kendi kendini sınayan program

`assert` ve altın dosya. Beş alıştırma: önce kitapçığın bozduğu satırı siz bulun, sonra kendi denemelerinizi yazın, zamana bağlı bir çıktıyı sınanabilir yapın, bir türe denemeler ekleyin, en sonda testi koddan önce yazın.

1 2 1 1

YANIT YERİNE

kitapçık III.3: Kendi kendini sınayan program ders 22: assert e3/ (altın dosya) gun-sayimi/ (zamana bağlı çıktı) CLAUDE.md: altın dosya yöntemi

ISINMA III-3-I1

Yakalandı

Kitapçık “çarpan bulan işlevin son satırını silelim” demişti; burada silinmiş hâli var. `make test` çalıştırın, `assert`'in iletisini okuyun: dosya, satır, koşul. Sonra `asal.cpp`'yi onarın. Silinen satır neydi, neden gerekli? Hangi sayılar için eksik olduğu belli olmazdı?

çıktı — BEKLENEN-CİKTİ

```
denemeler geçti.
360 sayısının asal çarpanları: 2 2 2 3 3 5
```

Dizin: `alistirma/kod/III-3-I1/` — dizinde ayrıca: `asal.cpp`, `asal.h`

ALIŞTIRMA III-3-A1

Yanlış olması gerekeni de sınayın

`ebob` için bir `dene()` yazın: en az beş `assert`, en az biri *yanlış* olması gereken bir şeyi sınasın (`ebob(7, 13) != 7` gibi), biri de sınır durumu (`ebob(0, 5)`, `ebob(5, 5)`). `main` önce `dene()`'yi çağırıp `denemeler geçti.` yazsın, sonra girdideki çifti hesaplasın. Girdi `1071 462`.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 1071 462

```
denemeler geçti.
ebob(1071, 462) = 21
```

Dizin: `alistirma/kod/III-3-A1/`

ALIŞTIRMA III-3-A2

Her gün bozulan test

Program bir doğum tarihinden bugüne kaç gün geçtiğini yazıyor; depodaki `gun-sayimi/` gibi. Testi her gün bozuluyor, çünkü çıktı bugüne bağlı. Kitapçığın önerisi: böyle çıktıyı ya sabitleyin ya testin dışında tutun. İkisini de yapın: “bugün”ü komut satırından alın (verilmezse gerçek bugün), `make test` sabit bir tarihle çağırsın. Tarih hesabı için `<chrono>`'nun `year_month_day` ve `sys_days` türleri var; girdi `yıl ay gün`.

çıktı — BEKLENEN-CİKTİ

```
Doğumdan bu yana 5892 gün geçti.
```

Dizin: `alistirma/kod/III-3-A2/`

ZORLAYICI İKİLİ III-3-Z1

Kesir'e denemeler

II-5-Z1'deki `Kesir` türü için `dene()` yazın: sadeleştirme (`Kesir{2, 4}` payı 1 paydası 2), toplama, çarpma, eşitlik. Eşitlik için `operator==` gerekecek. Bir tuzak: `assert(Kesir{1, 2} == Kesir{1, 2})` derlenmez, çünkü `assert` bir makro ve süslü parantezin içindeki virgülü argüman ayracı sanır; ifadeyi bir çift parantez daha içine alın. Sonra `-DNDEBUG` ile derleyin: `c++ -std=c++23 -DNDEBUG ana.cpp`. Denemeler ne oldu? Bu seçenek ne işe yarar, yarışmada açık mı kapalı mı olmalı?

DÜŞÜNCE

Eşitlik için `operator==`; `assert` makrosu virgülü argüman ayracı sanır, parantezle sarın.

YAKLAŞIM

`bool operator==(const Kesir& a, const Kesir& b) { return a.pay == b.pay and a.payda == b.payda; }` (yapıcı sadeleştirdiği için yeter). `assert((Kesir{1, 2} == Kesir{2, 4}));`

İLK SATIR

```
assert((Kesir{2, 4} == Kesir{1, 2}));
```

çıktı — BEKLENEN-CİKTİ

```
denemeler geçti.
1/2 + 1/3 = 5/6
```

Dizin: `alistirma/kod/III-3-Z1/`

KEŞİF III-3-K1

Önce test, sonra kod

Tersten: denemeler hazır, işlev yok. `romaya(int)` bir sayıyı Roma rakamına çevirsin; `dene()` içindeki yedi `assert` ne beklediğini söylüyor. Önce derlemeyi geçecek en basit gövdeyi yazın (boş yazı döndüren), sonra denemeleri birer birer geçirin. Buna *test güdümlü geliştirme* deniyor. İpucu: değer-simge çiftlerini büyükten küçüğe bir dizide tutun; `900 → CM` gibi çıkarmalı çiftler de o dizide.

DÜŞÜNCE

Roma rakamı büyükten küçüğe çıkarma: 1987 = 1000 + 900 + 50 + 30 + 5 + 1 + 1.

YAKLAŞIM

Değer–simge çiftlerini (`{1000,"M"}, {900,"CM"}, …, {1,"I"}`) bir dizide tutun; her çift için `while (n >= deger) { sonuc += simge; n -= deger; }` .

İLK SATIR

```
const vector<pair<int, string>> tablo{{1000, "M"}, {900, "CM"}, {500, "D"}, ...};
```

ana.cpp — BAŞLANGIÇ

```cpp
// III-3-K1 · Önce test, sonra kod
#include <iostream>
#include <cassert>
#include <string>
using namespace std;

string romaya(int n) {
  // buraya yazın
  return "";
}

void dene() {
  assert(romaya(1) == "I");
  assert(romaya(4) == "IV");
  assert(romaya(9) == "IX");
  assert(romaya(14) == "XIV");
  assert(romaya(40) == "XL");
  assert(romaya(1987) == "MCMLXXXVII");
  assert(romaya(2026) == "MMXXVI");
  cout << "denemeler geçti." << endl;
}

int main() {
  dene();
  for (int n : {3, 58, 444, 1994}) cout << n << " = " << romaya(n) << endl;
  return 0;
}
```

çıktı BEKLENEN-CİKTİ

```
denemeler geçti.
3 = III
58 = LVIII
444 = CDXLIV
1994 = MCMXCIV
```

Dizin: `alistirma/kod/III-3-K1/`

III.4

Hata bulmanın üç yolu

Dört programın dördünde de hata var. Birincisinde `dout` numarasını tanıyın; sonra küçülterek, adım adım izleyerek ve son olarak sessiz bir taşmanın peşinde. Kitapçığın takım işi burada en çok işe yarar: biri okur, öteki “dur, burada ne olur?” der.

1 2 1

YANIT YERİNE

kitapçık III.4: Hata bulmanın üç yolu ders 15: dout kitapçık I: taşma

ISINMA III-4-I1

dout'u kapatın

Program kitapçığın `dout` numarasını kullanıyor, ama ayıklama anahtarı açık kalmış: `make test` ayıklama satırları yüzünden geçmiyor. Önce çıktıyı okuyun, `dout` satırlarının ne anlattığını görün; sonra anahtarı kapatın. Bir adım ötesi: `dout`'u `cout` yerine `cerr`'e bağlayın; ayıklama açıkken bile `make test` neden geçer?

ana.cpp — BAŞLANGIÇ

```cpp
// III-4-I1 · dout'u kapatın
#include <iostream>
#include <vector>
using namespace std;

bool yaz = true;                 // ayıklama açık kalmış
ostream nout{nullptr};           // hiçbir yere yazmayan akım
ostream& dout = yaz ? cout : nout;

int main() {
  int n{0}; cin >> n;
  vector<int> d(n);
  for (int& x : d) cin >> x;
  int enBuyuk{d[0]};
  for (int x : d) {
    dout << "x=" << x << " enBuyuk=" << enBuyuk << endl;
    if (x > enBuyuk) enBuyuk = x;
  }
  cout << "En büyük: " << enBuyuk << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 3 1 4 1 5

```
En büyük: 5
```

Dizin: `alistirma/kod/III-4-I1/`

ALIŞTIRMA III-4-A1

Küçülterek

Program kırk sayının tek olanlarını topluyor; beklenen çıktıyla uyuşmuyor. Kırk sayıya bakarak hatayı bulmak zor. Kitapçığın yöntemi: `girdi`'yi kopyalayıp yarıya indirin, hata duruyor mu? Yine yarıya. Üç dört adımda iki üç sayıya iner ve hata görünür. Hatayı bulunca girdiyi geri koyun (`git checkout girdi` ya da yedek), `make test`.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 40 66 29 93 56 19 83 46 9 73 36 100 63 26 90 53 16 80 43 6 70 33 97 60 23 87 50 13 77 40 3 67 30 94 57 20 84 47 10 74 37

```
Teklerin toplamı: 1006
```

Dizin: `alistirma/kod/III-4-A1/`

ALIŞTIRMA III-4-A2

Adım adım

Özyinelemeli Fibonaççi yanlış sonuç veriyor: `fib(10)` 55 olmalı. Kodu okumak yerine izleyin: OnlineGDB'de `fib`'in ilk satırına durak koyup *Debug*, ya da terminalde `c++ -g` ile derleyip `gdb ./prog`, `break fib`, `run`, birkaç `continue`, her durakta `print n` ve `backtrace`. Çağrılar beklediğiniz gibi mi dallanıyor? Hatayı bulunca düzeltin. (Özyinelemenin kendisi Bölüm IV'te; burada yalnız izleyin.)

ana.cpp — BAŞLANGIÇ

```cpp
// III-4-A2 · Adım adım
#include <iostream>
using namespace std;

long long fib(int n) {
  if (n < 2) return n;
  return fib(n - 1) + fib(n - 1);
}

int main() {
  for (int n : {1, 2, 5, 10, 20}) cout << "fib(" << n << ") = " << fib(n) << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
fib(1) = 1
fib(2) = 1
fib(5) = 5
fib(10) = 55
fib(20) = 6765
```

Dizin: `alistirma/kod/III-4-A2/`

ZORLAYICI III-4-Z1

Sessiz taşma

Program girdideki sayıların toplamını ve ortalamasını yazıyor; küçük girdilerde doğru, bu girdide yanlış. Hiçbir satır bariz bozuk değil. Küçülterek deneyin: kaç sayıya inince düzeliyor? Bir tahmin: toplamın büyüklüğüne bakın, Bölüm I'deki sınırı hatırlayın. Düzeltme tek sözcük. Böyle hatalar yarışmalarda “küçük testler geçti, büyükler yanlış” diye görünür.

DÜŞÜNCE

Küçük girdide doğru, büyükte yanlış: sayılar büyüyor, bir yerde sığmıyor.

YAKLAŞIM

Toplamın türüne bakın; girdideki sayılar ile sayısını çarpın, `int`'in sınırıyla karşılaştırın. Düzeltme tek sözcük: toplamın türü.

İLK SATIR

`long long toplam{0};`

ana.cpp — BAŞLANGIÇ

```cpp
// III-4-Z1 · Sessiz taşma
#include <iostream>
#include <vector>
using namespace std;

int main() {
  int n{0}; cin >> n;
  vector<int> d(n);
  for (int& x : d) cin >> x;
  int toplam{0};
  for (int x : d) toplam += x;
  cout << "Toplam: " << toplam << endl;
  cout << "Ortalama: " << toplam / n << endl;
  return 0;
}
```

çıktı

```
BEKLENEN-CİKTİ · GİRDİ: 20 200000000 200000007 200000014
200000021 200000028 200000035 200000042 200000049 200000056
200000063 200000070 200000077 200000084 200000091 200000098
200000105 200000112 200000119 200000126 200000133
```

```
Toplam: 4000001330
Ortalama: 200000066
```

Dizin: `alistirma/kod/III-4-Z1/`

Tahmin etmeyin, ölçün

Ölçüm sonuçları her makinede, her çalıştırmada değişir; `diff` onları sınayamaz. Bu konunun kuralı: sonuçlar `cout`'a, süreler `cerr`'e. `make test` yalnız `cout`'a bakar, siz ekranda ikisini de görürsünüz. Sürelerinizi bir tabloya yazın; o tablo Bölüm IV'teki karmaşıklık kavramının somut hâli.

`1` `1` `1` `1`

> **YANIT YERİNE**
>
> kitapçık III.5: Tahmin etmeyin, ölçün hız ölçümü kod: 20-hiz.cpp

ISINMA III-5-I1

Süreler cerr'e

Kitapçığın `20-hiz.cpp`'si: iki satırı süre yazıyor, bu yüzden `make test` geçmez. Süre satırlarını `cerr`'e yazın; `make test` geçsin, ekranda süreler yine görünsün. Sonra `./prog 2>/dev/null` ile süreleri susturun, `./prog >/dev/null` ile yalnız süreleri görün. `2>` ne demek?

çıktı — BEKLENEN-CİKTİ

```
dizi boyu         : 1000000 öge
(toplam 99990000)
```

Dizin: `alistirma/kod/III-5-I1/`

ALIŞTIRMA III-5-A1

Üç sınır, üç süre

I-4-A2'deki soru: `asalMi`'de bölenleri `n`'e kadar mı, `n/2`'ye mi, `√n`'e mi denemeli? Üçünü de yazın; 200000'e kadar asalları her biriyle sayın. Sayılar `cout`'a (üçü de aynı olmalı), süreler `cerr`'e. Kaç kat fark var? `-O2` ile fark değişiyor mu?

çıktı — BEKLENEN-CİKTİ

```
n'e kadar: 17984 asal
n/2'ye kadar: 17984 asal
köke kadar: 17984 asal
```

Dizin: `alistirma/kod/III-5-A1/`

ZORLAYICI İKİLİ III-5-Z1

Tekrar var mı, üç yol, üç boy

Kitapçığın üçüncü alıştırması. II-3-A1'in küme çözümüne iki rakip ekleyin: sıralayıp komşuları karşılaştırmak (`sort`) ve iç içe iki döngü. Veri üretimi hazır (tekrar en sonda). Dört boy: 10 000, 30 000, 100 000, 1 000 000 — iç içe döngü yalnız ilk ikisinde; 100 000'de yarım dakika, bir milyonda saatler sürer (deneyin, sonra `Ctrl-C`). Sonuçlar `cout`'a, süreler `cerr`'e; tabloyu yorum olarak dosyanın başına yazın. Boy üç katına çıkınca döngünün süresi kaç katına çıkıyor? On katına çıkınca kümenin ve sıralamanın? Üç yöntem, üç ayrı yanıt.

DÜŞÜNCE

Üç yöntem, dört boy; her ölçüm bir `chrono` çifti.

YAKLAŞIM

Her yöntemi ayrı işlev yapın; `auto t0 = saat::now();` … `duration_cast<milliseconds>`; süreleri `cerr`'e, sonuçları `cout`'a. Sıralama için `sort` sonra komşu karşılaştırma.

İLK SATIR

```
auto t0 = chrono::steady_clock::now(); bool s = tekrarSort(d);
auto t1 = chrono::steady_clock::now();
```

çıktı — BEKLENEN-CİKTİ

```
küme    n=10000: tekrar var
sırala  n=10000: tekrar var
döngü   n=10000: tekrar var
küme    n=30000: tekrar var
sırala  n=30000: tekrar var
döngü   n=30000: tekrar var
küme    n=100000: tekrar var
sırala  n=100000: tekrar var
küme    n=1000000: tekrar var
sırala  n=1000000: tekrar var
```

Dizin: `alistirma/kod/III-5-Z1/`

KEŞİF III-5-K1 -O2 farkı

Bu dizinin `Makefile`'ında iki hedef var: `make yavas` (`-O0`) ve `make hizli` (`-O2`). Aynı programı ikisiyle de derleyip çalıştırın; kitapçığın tablosunun kendi makinenizdeki hâlini doldurun. Sonra III-5-Z1'i de iki seçenekle ölçün: hangi yöntemde fark büyük, hangisinde küçük? Neden algoritma farkı derleyici farkından daha çok iş yapıyor? Beklenen çıktı yok; ölçümünüz çıktı.

DÜŞÜNCE

Aynı kod, iki derleme seçeneği; fark ölçümden çıkar.

YAKLAŞIM

`make yavas && ./prog`, sonra `make hizli && ./prog`; sayıları bir tabloya yazın. III-5-Z1 için Makefile'a aynı iki hedefi ekleyin.

İLK SATIR

```
hizli:\n\tg++ -std=c++23 -O2 ana.cpp -o prog
```

Dizin: `alistirma/kod/III-5-K1/` — beklenen çıktı yok

III.6

Kendini tekrar etme

Sadeleştirmenin ölçütü kitapçıkta: *sonunda `make test` hâlâ geçiyor mu? Geçiyorsa doğru sadeleştirdiniz.* Üç program; üçü de çalışıyor (biri hariç), üçü de tekrar dolu. İş: davranışı değiştirmeden kısaltmak — ve bir yerde, kısaltırken hatayı da bulmak.

`1` `1` `1`

YANIT YERİNE

kitapçık III.6: Kendini tekrar etme ders 8: DRY ve fayda.h ders 16: çizim projesi

ISINMA III-6-I1

Üç kere aynı döngü

Program üç yıldız karesi çiziyor; çizim döngüsü üç kere kopyalanmış. Döngüyü `kare(int boy)` işlevine çıkarın, üç çağrı kalsın. Çıktı bir karakter bile değişmemeli: `make test` sizin hakeminiz.

ana.cpp — BAŞLANGIÇ

```cpp
// III-6-I1 · Üç kere aynı döngü
#include <iostream>
using namespace std;

int main() {
  for (int s{0}; s < 2; ++s) {
    for (int k{0}; k < 2; ++k) cout << "*";
    cout << endl;
  }
  cout << endl;
  for (int s{0}; s < 3; ++s) {
    for (int k{0}; k < 3; ++k) cout << "*";
    cout << endl;
  }
  cout << endl;
  for (int s{0}; s < 5; ++s) {
    for (int k{0}; k < 5; ++k) cout << "*";
    cout << endl;
  }
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
**
**

***
***
***

*****
*****
*****
*****
*****
```

Dizin: `alistirma/kod/III-6-I1/`

ALIŞTIRMA III-6-A1

Dört tablo, bir hata

Dört çarpım tablosu, dört kopya döngü — ve kopyalardan birinde bir hata var, `make test` geçmiyor. Hatayı satır satır arayabilirsiniz; ama daha iyisi, dört bloğu tek bir `tablo(int)` işlevine çevirin: hata ya kendiliğinden kaybolur ya da tek yerde görünür. Kitapçığın cümlesi: kopyaladığınız kodu iki kere bakıma alırsınız.

ana.cpp — BAŞLANGIÇ

```cpp
// III-6-A1 · Dört tablo, bir hata
#include <iostream>
using namespace std;

int main() {
  cout << "6:";
  for (int k{1}; k <= 10; ++k) cout << " " << 6 * k;
  cout << endl;
  cout << "7:";
  for (int k{1}; k <= 10; ++k) cout << " " << 7 * k;
  cout << endl;
  cout << "8:";
  for (int k{1}; k <= 10; ++k) cout << " " << 7 * k;
  cout << endl;
  cout << "9:";
  for (int k{1}; k <= 10; ++k) cout << " " << 9 * k;
  cout << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
6: 6 12 18 24 30 36 42 48 54 60
7: 7 14 21 28 35 42 49 56 63 70
8: 8 16 24 32 40 48 56 64 72 80
9: 9 18 27 36 45 54 63 72 81 90
```

Dizin: `alistirma/kod/III-6-A1/`

ZORLAYICI III-6-Z1

Dört yön, bir sınır denetimi

Izgaradaki her hücre için dolu komşularını sayan program: dört yön, dört kopya sınır denetimi, biri yanlış; `make test` farkı gösteriyor, hangi satır olduğunu değil. `icerdeMi(satir, sutun)` işlevi ve yön listesi (`{{-1,0},{1,0},{0,-1},{0,1}}`) ile yeniden yazın; `Izgara` , `Satir` gibi takma adlar koyun; `a` , `b` , `c` adlarını anlamlı adlarla değiştirin. Bölüm V'teki gezi kalıpları tam olarak bu düzeni kullanacak; şimdiden alışın. Girdi: satır sütun, sonra ızgara (`#` dolu, `.` boş).

DÜŞÜNCE

Dört kopya sınır denetiminden biri yanlış; dört yönü bir listeye alınca denetim tek yerde olur.

YAKLAŞIM

`const vector<pair<int,int>> yonler{{-1,0},{1,0},{0,-1},{0,1}};` ve `icerdeMi(r, c)` ; iç döngü yönleri gezip `icerdeMi` ile denetler. Yanlış olan `>` mi `>=` mi?

İLK SATIR

```
bool icerdeMi(int r, int c) { return r >= 0 and r < satir and c >= 0 and c < sutun; }
```

ana.cpp BAŞLANGIÇ

```cpp
// III-6-Z1 · Dört yön, bir sınır denetimi
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main() {
  int a, b;
  cin >> a >> b;
  vector<string> g(a);
  for (auto& s : g) cin >> s;
  for (int i{0}; i < a; ++i) {
    for (int j{0}; j < b; ++j) {
      int c{0};
      if (i - 1 >= 0 and g[i - 1][j] == '#') ++c;
      if (i + 1 < a and g[i + 1][j] == '#') ++c;
      if (j - 1 > 0 and g[i][j - 1] == '#') ++c;
      if (j + 1 < b and g[i][j + 1] == '#') ++c;
      cout << c;
    }
    cout << endl;
  }
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 4 #..# .##. #...

```
0220
3112
0210
```

Dizin: `alistirma/kod/III-6-Z1/`

III.7

Geri dönüş: eski işlevleri sınamak

Bu bölümün alışkanlığı sınamak; sınanacak şey de önceki bölümlerden. İki kısa soru: I. bölümün `bolunuyorMu`'su ve II. bölümün harf sayacı, bu sefer `assert` ile.

2

> **YANIT YERİNE**
>
> kitapçık I.2: İşlev kitapçık II.2: Eşlem kitapçık III.3: Kendi kendini sınayan program

ALIŞTIRMA III-7-A1 bolunuyorMu için denemeler

I. bölümün tek satırlık işlevi `bolunuyorMu(a, b)`'yi yazın ve önce `dene()` içinde dört `assert` ile sınayın: 15 5'e bölünür, 15 4'e bölünmez, 0 her sayıya bölünür, **b sıfırsa yanıt yanlış** (programı çökertmeyin). Sonra girdideki çiftler için `15 5 -> evet` biçiminde yazın. Sıfıra bölme denemesini yazmadan önce işlevin ne yaptığına bakın: `15 % 0` çalışma zamanında çöker.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 15 5 15 4 0 3 7 0

```
denemeler geçti.
15 5 -> evet
15 4 -> hayır
0 3 -> evet
7 0 -> hayır
```

Dizin: `alistirma/kod/III-7-A1/`

ALIŞTIRMA III-7-A2

Harf sayacını sınayın

II. bölümün harf sayacını bir işleve koyun: `map<char,int> harfSay(const string&)`, boşlukları saymasın. Önce `dene()`: `harfSay("aab")` için a 2, b 1, ve `c` anahtarı **yok** (`contains` ile bakın; `[]` ile bakarsanız anahtarı yaratırsınız, II. bölümün klasik tuzağı). Sonra girdideki satırı `getline` ile okuyup sayımı `a: 2` biçiminde yazın.

çıktı — BEKLENEN-CİKTİ · GİRDİ: KELEBEK KANADİ

```
denemeler geçti.
a: 2
b: 1
d: 1
e: 3
i: 1
k: 3
l: 1
n: 1
```

Dizin: `alistirma/kod/III-7-A2/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

"Bu bölümü geçtim mi?" sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` "Doğru!" desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

- `III-1-Z1` Komut satırından girdi
- `III-3-Z1` Kesir'e denemeler
- `III-4-Z1` Sessiz taşma

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.

BÖLÜM IV

Özyineleme ve Arama

Kitapçığın dördüncü bölümü tek bir fikri ehlileştiriyor: kendini çağıran işlev. Burada altı konu, yirmi dokuz alıştırma: Hanoi, memoizasyon, altkümeler, permütasyonlar, sekiz vezir, karmaşıklık ve budama. Keşifler CSES'in Elma paylaşımı ve Izgara yolları sorularına, bir de Sudoku'ya çıkıyor.

Hazırlık | I. İlk Adımlar | II. Veriyi Düzenlemek | III. Zanaat | IV. Özyineleme | V. Çizgeler | VI. Dinamik Programlama

IV.1

Kendini çağıran işlev; Hanoi kuleleri

Taban durumu önce. Kitapçığın `34-hanoi.cpp`'siyle başlıyoruz; sonra iki klasik tanım, sonra Hanoi'yi saymak ve en sonda bir çözümün geçerli olup olmadığını denetleyen bir hakem.

1 2 1 1

YANIT YERİNE

kitapçık IV.1: Kendini çağıran işlev ders 7: özyineleme algoritma ders 9: Hanoi kod: 34-hanoi.cpp

ISINMA IV-1-I1

Dört disk

`n`'i 3 yerine 4 yapın. Kaç adım? Çalıştırmadan önce `2ⁿ − 1`'den hesaplayın; sonra çıktıda 4. diskin yalnız bir kere, 1. diskin sekiz kere taşındığını sayın. Neden?

çıktı — BEKLENEN-CİKTİ

```
1) 1. diski 1 -> 2
2) 2. diski 1 -> 3
3) 1. diski 2 -> 3
4) 3. diski 1 -> 2
5) 1. diski 3 -> 1
6) 2. diski 3 -> 2
7) 1. diski 1 -> 2
8) 4. diski 1 -> 3
9) 1. diski 2 -> 3
10) 2. diski 2 -> 1
11) 1. diski 3 -> 1
12) 3. diski 2 -> 3
13) 1. diski 1 -> 2
14) 2. diski 1 -> 3
15) 1. diski 2 -> 3
toplam 15 adım (2^4 - 1 = 15)
```

Dizin: `alistirma/kod/IV-1-I1/`

ALIŞTIRMA IV-1-A1

Faktöriyel ve toplam

İki özyinelemeli işlev: `faktoriyel(n)` ve `toplam(n)` (1'den n'e). İkisinde de önce taban durumunu yazın, sonra adımı. Girdi `10`. Döngüyle de yazılır; burada amaç özyinelemeli tanımı koda birebir çevirmek: `n! = n · (n−1)!`, `0! = 1`.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 10

```
10! = 3628800
1+...+10 = 55
```

Dizin: `alistirma/kod/IV-1-A1/`

ALIŞTIRMA IV-1-A2

Öklid, bu sefer özyinelemeli

I-4-Z1'deki döngülü `ebob`'u özyinelemeli yazın: `ebob(a, 0) = a`, `ebob(a, b) = ebob(b, a % b)`. Her çağrıda ekrana `ebob(a, b)` yazın ki zincir görünsün; en sonda sonuç. Girdi `1071 462`. Döngülü sürümle yan yana koyun: hangisi tanıma daha yakın?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 1071 462

```
ebob(1071, 462) -> ebob(462, 147) -> ebob(147, 21) -> ebob(21, 0) = 21
```

Dizin: `alistirma/kod/IV-1-A2/`

ZORLAYICI IV-1-Z1

Hanoi'yi saymak

Taşımaları yazdırmak yerine sayın: her diskin kaç kere taşındığını bir dizide tutun. Girdi `5`; her disk için bir satır, sonda toplam. Çıktıya bakınca her diskin bir üstündekinin yarısı kadar taşındığını göreceksiniz; neden? Buradan `2ⁿ − 1`'i türetin. Sonra `n = 64` için toplam adımı `long long` ile hesaplayın (taşımadan, formülle) ve saniyede bir hamleyle kaç yıl sürdüğünü yazın.

DÜŞÜNCE

`tasi`'nin ortasındaki "en büyüğü hedefe" satırı bir taşımadır; onu sayın.

YAKLAŞIM

`cout` satırının yerine `++kacKere[n];` ; toplamı dizinin toplamı olarak alın. 64 için formül: `(1ULL << 64) - 1` taşar; `numeric_limits<unsigned long long>::max()` tam odur.

İLK SATIR

```
void tasi(int n, ...) { if (n == 0) return; tasi(n-1, ...); ++kacKere[n]; tasi(n-1, ...); }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5

```
1. disk: 16 kere
2. disk: 8 kere
3. disk: 4 kere
4. disk: 2 kere
5. disk: 1 kere
toplam 31 adım
64 disk: 18446744073709551615 adım, yaklaşık 584 milyar yıl
```

Dizin: `alistirma/kod/IV-1-Z1/`

KEŞİF IV-1-K1 Hanoi hakemi

Kitapçığın *Deneyin* kutusu: `tasi()` 'nin üç satırının sırasını değiştirince çıktı geçerli bir çözüm mü? Göz yerine program karar versin. Üç çubuğu üç `stack<int>` ile canlandırın; her taşımada kaynak boş mu, büyük disk küçüğün üstüne mi geliyor diye bakın, bozuk hamlede durup `hata: k. adım` yazın. Girdi `4`; doğru sırayla `15 adım, hepsi geçerli`. Sonra satırların sırasını değiştirip hakemin ne dediğine bakın. Daha ileri: hakem, sonda bütün disklerin 3. çubukta olduğunu da denetlesin.

DÜŞÜNCE

Hakem her hamlede iki kural denetler: kaynak boş mu, büyük küçüğün üstüne mi geliyor.

YAKLAŞIM

`hamle(k, h)` içinde: `cubuk[k].empty()` → hata; `!cubuk[h].empty() and cubuk[h].top() < cubuk[k].top()` → hata; değilse diski taşıyın. Başta diskleri 1. çubuğa büyükten küçüğe koyun.

İLK SATIR

```
if (cubuk[kaynak].empty() or (not cubuk[hedef].empty() and cubuk[hedef].top() < cubuk[kaynak].top())) { hata = true; return; }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4

```
15 adım, hepsi geçerli
```

Dizin: `alistirma/kod/IV-1-K1/`

IV.2

Aynı işi iki kere yapmayın

Memoizasyon: hesapladığını bir kenara yaz. Kitapçığın `30-fib.cpp` 'siyle başlıyoruz; süreler III.5'in kuralıyla `cerr` 'e gidecek. Bölüm VI buraya geri dönecek.

`1` `2` `1`

YANIT YERİNE

kitapçık IV.2: Aynı işi iki kere yapmayın ders 7: bellekle hızlandırma dp-sade kod: 30-fib.cpp

ISINMA IV-2-I1 Süreler cerr'e, çağrılar cout'a

`30-fib.cpp` süreleri de yazdığı için sınanamıyor. Çağrı sayıları ve sonuç `cout`'ta kalsın, mikrosaniyeler `cerr`'e gitsin (III-5-I1'deki gibi). `fibYavas(40)` birkaç saniye sürer; bekleyin, kitapçığın tablosundaki 331 160 281 çağrıyı kendi gözünüzle görün.

çıktı — BEKLENEN-CİKTİ

```
fib(40) = 102334155
yavaş: 331160281 çağrı
hızlı: 79 çağrı
kontrol: aynı sonuç
```

Dizin: `alistirma/kod/IV-2-I1/`

ALIŞTIRMA IV-2-A1 Merdiven

Bir merdiveni her adımda bir ya da iki basamak çıkarak kaç farklı biçimde çıkabilirsiniz? `yol(n) = yol(n−1) + yol(n−2)`; taban durumlarını siz bulun. Önce memoizasyonsuz yazıp `n = 50`'nin bitmediğini görün; sonra bir `vector<long long>` bellekle. Üç satır: n = 10, 30, 50.

çıktı — BEKLENEN-CİKTİ

```
10 basamak: 89
30 basamak: 1346269
50 basamak: 20365011074
```

Dizin: `alistirma/kod/IV-2-A1/`

ALIŞTIRMA IV-2-A2 Pascal üçgeni

`C(n, k) = C(n−1, k−1) + C(n−1, k)`, kenarlarda 1. Bu tanımı özyinelemeli yazın; bellek olarak `map<pair<int,int>, long long>`. `C(30, 15)` ve `C(60, 30)`. Belleksiz hâliyle ikincisini de deneyin: kaç çağrı olur? (Hesaplayın, çalıştırmayın: `C(60, 30)`'un kendisi kadar, yani 10^{17}.)

çıktı — BEKLENEN-CİKTİ

```
C(30, 15) = 155117520
C(60, 30) = 118264581564861424
```

Dizin: `alistirma/kod/IV-2-A2/`

ZORLAYICI IV-2-Z1

En uzun Collatz zinciri

Project Euler 14: bir milyonun altında en uzun Collatz zincirini başlatan sayı. I-3-Z2'deki sayacı özyinelemeli yazın ve belleyin: `terim(n) = 1 + terim(sonraki)`, `terim(1) = 1` (Project Euler zincirin terim sayısını sayar, 27 için 112). Bellek bir `vector<int>` olsun ama dikkat: zincir bir milyonu aşan sayılardan geçer; onları bellemeyin, yalnız hesaplayın. Çıktı sayı ve terim sayısı.

DÜŞÜNCE

Terim sayısı özyinelemeli: `terim(n) = 1 + terim(sonraki)`; bir milyonun altındakileri belleyin.

YAKLAŞIM

`if (n < SINIR and bellek[n]) return bellek[n];` başta; sonucu yalnız `n < SINIR` iken kaydedin. Sonraki değer `long long`; en uzun için 1..999999 döngüsü.

İLK SATIR

```
int t = 1 + terim(n % 2 == 0 ? n / 2 : 3 * n + 1); if (n < SINIR) bellek[n] = t; return t;
```

çıktı — BEKLENEN-CİKTİ

```
Project Euler, on dördüncü problem: 837799 (525 terim)
```

Dizin: `alistirma/kod/IV-2-Z1/`

IV.3

Bütün altkümeler

Geri dönüşlü aramanın kalıbı: seç, derine in, bıraktığın gibi bırak. Kitapçığın `31-altkume.cpp`'siyle başlıyoruz; Zorlayıcı ve Keşif, kitapçığın birinci alıştırması olan CSES Elma paylaşımı.

1 2 1 1

YANIT YERİNE

kitapçık IV.3: Bütün olasılıkları gezmek algoritma ders 10: alt küme üretimi sayı kümesini ikiye bölmek kod: 31-altkume.cpp

ISINMA IV-3-I1

Dört öge

`kume`'yi `{3, 1, 2}` yerine `{4, 2, 7, 1}` yapın. Kaç altküme? İki yöntemin çıktısını yan yana koyun: sıraları aynı mı? Geri dönüşlü sürümde ilk yazılan altküme neden boş küme?

çıktı — BEKLENEN-CİKTİ

```
-- geri dönüşlü arama --
{ }
{ 1 }
{ 7 }
{ 7 1 }
{ 2 }
{ 2 1 }
{ 2 7 }
{ 2 7 1 }
{ 4 }
{ 4 1 }
{ 4 7 }
{ 4 7 1 }
{ 4 2 }
{ 4 2 1 }
{ 4 2 7 }
{ 4 2 7 1 }

-- bitlerle --
0  0000  { }
1  0001  { 4 }
2  0010  { 2 }
3  0011  { 4 2 }
4  0100  { 7 }
5  0101  { 4 7 }
6  0110  { 2 7 }
7  0111  { 4 2 7 }
8  1000  { 1 }
9  1001  { 4 1 }
10  1010  { 2 1 }
11  1011  { 4 2 1 }
12  1100  { 7 1 }
13  1101  { 4 7 1 }
14  1110  { 2 7 1 }
15  1111  { 4 2 7 1 }
```

Dizin: `alistirma/kod/IV-3-I1/`

ALIŞTIRMA IV-3-A1

Toplamı tutan altkümeler

Girdi: n, n sayı, bir hedef toplam. Toplamı hedefe eşit olan altkümeleri kitapçığın biçimiyle (`{ 3 2 }`) yazın, sonda kaç tane olduğunu. Girdi `4` / `3 1 2 4` / `5`. Kalıp `31-altkume`'nin aynısı; yalnız `yaz()` çağrısının önüne bir koşul geliyor. `secilen`'in toplamını her seferinde yeniden hesaplamak yerine bir sayaçla taşıyabilir misiniz?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 3 1 2 4 5

```
{ 1 4 }
{ 3 2 }
2 altküme
```

Dizin: `alistirma/kod/IV-3-A1/`

ALIŞTIRMA IV-3-A2

Her boydan kaç tane?

n ögeli kümenin her boydan kaç altkümesi var? Yazmak yerine sayın: `sayac[secilen.size()]`. Girdi `5`; her boy için bir satır, sonda toplam. Sayılar size tanıdık gelecek (IV-2-A2). Toplamın 2^n olduğunu bir cümleyle açıklayın.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5

```
boy 0: 1
boy 1: 5
boy 2: 10
boy 3: 10
boy 4: 5
boy 5: 1
toplam 32
```

Dizin: `alistirma/kod/IV-3-A2/`

ZORLAYICI İKİLİ IV-3-Z1

Elmaları bölmek

CSES Elma paylaşımı (*Apple Division*), küçük hâli: n elmayı iki gruba ayırın, ağırlık farkı en az olsun. Her elma için "birinci gruba mı" kararı; taban durumunda farkı hesaplayıp en küçüğü güncelleyin. Girdi `5` / `3 2 7 4 1`. Sonra aynısını bitlerle (II-3-K1) yazıp iki sonucu karşılaştırın. `n ≤ 20`: kaba kural ne diyor?

DÜŞÜNCE

Her elma iki gruptan birine; bütün bölüşmeler 2^n dal; tabanda farkı ölçün.

YAKLAŞIM

`ara(k, birinci, ikinci)`: taban `k == elma.size()`'da `abs(birinci - ikinci)` ile `enKucuk`'ü güncelleyin; adımda iki çağrı. İlk değer için `-1` yerine çok büyük bir sayı daha rahat.

İLK SATIR

```
ara(k + 1, birinci + elma[k], ikinci); ara(k + 1, birinci, ikinci + elma[k]);
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 3 2 7 4 1

```
En küçük fark: 1
```

Dizin: `alistirma/kod/IV-3-Z1/`

KEŞİF IV-3-K1 Elma paylaşımı, tam boy

Aynı soru, CSES'in sınırlarında: `n = 20`, ağırlıklar 10^9'a kadar. Girdi dosyası hazır. $2^{20} \approx$ bir milyon dal; kaba kurala göre saniyenin altı, ölçün. Sonra CSES'e gönderin, yeşil tik alın. Bir adım ötesi: ilk elmayı hep birinci gruba koyarak dalların yarısını atabilirsiniz; neden sonuç değişmez? Bu bir budama mı, simetri mi?

DÜŞÜNCE

Aynı arama, yalnız sayılar büyük: toplam `long long`.

YAKLAŞIM

IV-3-Z1'in kodu olduğu gibi; toplamları `long long` yapın. Simetri için ilk elmayı hep birinci gruba koyun: `ara(1, elma[0], 0)`.

İLK SATIR

```
ara(1, elma[0], 0);
```

çıktı

```
BEKLENEN-CİKTİ · GİRDİ: 20 829348952 658697903 488046854
317395805 146744756 976093707 805442658 634791609 464140560
293489511 122838462 952187413 781536364 610885315 440234266
269583217 98932168 928281119 757630070 586979021

En küçük fark: 17292306
```

Dizin: `alistirma/kod/IV-3-K1/`

IV.4

Bütün permütasyonlar

Aynı kalıp, bu sefer sıra önemli ve her öge bir kere. Kitapçığın `32-permutasyon.cpp`'sinden, kitapçığın üçüncü alıştırması olan güzel permütasyona ve küçük bir gezgin satıcıya.

1 2 1 1

YANIT YERİNE

kitapçık IV.3: Bütün permütasyonlar algoritma ders 11: permütasyon üretimi algoritma ders 13: deste sıralama kod: 32-permutasyon.cpp

ISINMA IV-4-I1 İzlemeyi kapatın, dörde çıkın

`izle`'yi `false`, `N`'i 4 yapın. Kaç permütasyon? Önce `N = 4` ile izlemeyi açık bırakıp çıktıyı bir kere okuyun: `+` işaretleri derinlik; geri dönüşler nerede?

çıktı — BEKLENEN-CİKTİ

```
1. permütasyon: 0123
2. permütasyon: 0132
3. permütasyon: 0213
4. permütasyon: 0231
5. permütasyon: 0312
6. permütasyon: 0321
7. permütasyon: 1023
8. permütasyon: 1032
9. permütasyon: 1203
10. permütasyon: 1230
11. permütasyon: 1302
12. permütasyon: 1320
13. permütasyon: 2013
14. permütasyon: 2031
15. permütasyon: 2103
16. permütasyon: 2130
17. permütasyon: 2301
18. permütasyon: 2310
19. permütasyon: 3012
20. permütasyon: 3021
21. permütasyon: 3102
22. permütasyon: 3120
23. permütasyon: 3201
24. permütasyon: 3210
```

Dizin: `alistirma/kod/IV-4-I1/`

ALIŞTIRMA IV-4-A1

Harflerin sırası

Sayılar yerine bir sözcüğün harfleri: girdiden sözcüğü okuyun, bütün dizilişlerini yazın, sonda kaç tane. Girdi `kedi`. Kalıp aynı: `eklendi` dizisi ve bir `string` biriktirici. Harf tekrar ederse (`anne`) ne olur? Deneyin; çözümü IV-4-A2'de.

çıktı — BEKLENEN-ÇİKTİ · GİRDİ: KEDİ

```
kedi
keid
kdei
kdie
kied
kide
ekdi
ekid
edki
edik
eikd
eidk
dkei
dkie
deki
deik
dike
diek
iked
ikde
iekd
iedk
idke
idek
24 diziliş
```

Dizin: `alistirma/kod/IV-4-A1/`

ALIŞTIRMA IV-4-A2 Hazırıyla: next_permutation

Kitapçığın püf noktası: `sort` edip `do … while (next_permutation(...))`. Aynı girdiyle (`kedi`) dizilişleri yazın; IV-4-A1'in çıktısıyla karşılaştırın: sıra farklı mı, sayı aynı mı? Sonra `anne` ile deneyin: bu sefer tekrarlı harfleri kendiliğinden tek sayar. Neden?

çıktı — BEKLENEN-ÇİKTİ · GİRDİ: KEDİ

```
deik
deki
diek
dike
dkei
dkie
edik
edki
eidk
eikd
ekdi
ekid
idek
idke
iedk
iekd
ikde
iked
kdei
kdie
kedi
keid
kide
kied
24 diziliş
```

Dizin: `alistirma/kod/IV-4-A2/`

ZORLAYICI IV-4-Z1

Güzel permütasyon

CSES 1070: 1..n'in öyle bir dizilişi ki komşu sayıların farkı hiç 1 olmasın. Önce `n = 5` için bütün dizilişleri üretip ilk uygun olanı yazın. Sonra kitapçığın asıl sorusu: üretmeden, doğrudan kurun — önce çiftler sonra tekler neden işe yarar? Hangi n'lerde hiç çözüm yok? Girdi `7`; çıktı iki satır: üretimle bulunan (sözlük sırasında ilk uygun olan; n ≤ 8 için) ve doğrudan kurulan. İkisi farklı; ikisi de doğru.

DÜŞÜNCE

İki yol: bütün dizilişleri deneyip ilk uygun olanı bulmak, ya da çiftleri sonra tekleri yazmak.

YAKLAŞIM

Üretim: `vector<int> p` 1..n, `do { if (guzelMi(p)) break; } while (next_permutation(...))`. Doğrudan: n = 1 için 1, n = 2 ve 3 için çözüm yok, n ≥ 4 için 2 4 6 … sonra 1 3 5 ….

İLK SATIR

```
bool guzelMi(const vector<int>& p) { for (size_t k{1}; k < p.size(); ++k) if (abs(p[k] - p[k-1]) == 1) return false; return true; }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 7

```
üretimle: 1 3 5 2 6 4 7
doğrudan: 2 4 6 1 3 5 7
```

Dizin: `alistirma/kod/IV-4-Z1/`

KEŞİF IV-4-K1

Gezgin satıcı, altı şehir ~2 saat

Altı şehir, aralarındaki uzaklıklar bir tabloda (girdi). 0'dan başlayıp hepsini bir kere dolaşıp 0'a dönen en kısa tur. Öteki beş şehrin bütün dizilişleri: 120 tur, her birinin uzunluğu. En kısasını ve uzunluğunu yazın. Sonra düşünün: 15 şehirde kaç tur? 20'de? Kaba kural ne diyor; bu sorunun neden "zor" sayıldığını bir cümleyle yazın.

DÜŞÜNCE

0 sabit; öteki beş şehrin her dizilişi bir tur; 120 tur, en kısası.

YAKLAŞIM

`vector<int> sira{1, 2, 3, 4, 5}` ; `do … while (next_permutation)` ; her turda `0→sira[0]→…→sira[4]→0` uzunluğunu toplayın.

İLK SATIR

```
int uzunluk{u[0][sira[0]]}; for (int k{1}; k < n - 1; ++k) uzunluk += u[sira[k-1]][sira[k]]; uzunluk += u[sira[n-2]][0];
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 6 0 7 9 4 8 6 7 0 3 5 9 8 9 3 0 6 2 7 4 5 6 0 5 3 8 9 2 5 0 4 6 8 7 3 4 0

```
En kısa tur: 0 1 2 4 5 3 0, uzunluk 23
```

Dizin: `alistirma/kod/IV-4-K1/`

IV.5

Sekiz vezir

Kalıp aynı, değişen yalnız "uygun mu" koşulu. Kitapçığın `33-vezir.cpp` 'sinden tahtaları çizmeye, tabloyu doğrulamaya, iki yoldan saymaya ve bir Sudoku çözücüye.

1 1 2 1

YANIT YERİNE

kitapçık IV.4: Sekiz vezir algoritma ders 12: 8 vezir kod: 33-vezir.cpp

ISINMA IV-5-I1

Altıya altı

`N`'i 6 yapın. Kaç çözüm? Kitapçığın tablosuyla karşılaştırın; 5×5'ten az çıkmasına şaşırın. `capraz1` ve `capraz2` dizilerinin boyu neden `2N-1`?

çıktı — BEKLENEN-CİKTİ

```
6 vezir bulmacasının çözüm sayısı: 4
```

Dizin: `alistirma/kod/IV-5-I1/`

ALIŞTIRMA IV-5-A1

Tahtaları çizin

Kitapçığın ikinci alıştırması: saymak yerine çizin. Her satırın vezirinin sütununu bir dizide tutun (`yer[satir] = s`); çözüm bulununca tahtayı `Q` ve `.` ile basın. Yalnız ilk üç çözümü yazın (sonra sayın). Girdi `8`. İlk üçüne bakın: birbirlerinin aynadaki hâli mi?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 8

```
Q.......
....Q...
.......Q
.....Q..
..Q.....
......Q.
.Q......
...Q....

Q.......
.....Q..
.......Q
..Q.....
......Q.
...Q....
.Q......
....Q...

Q.......
......Q.
...Q....
.....Q..
.......Q
.Q......
....Q...
..Q.....

8 vezir: 92 çözüm
```

Dizin: `alistirma/kod/IV-5-A1/`

ZORLAYICI IV-5-Z1

Tabloyu doğrulayın

Kitapçığın tablosu 4×4'ten 8×8'e; siz 4'ten 9'a kadar her tahta için çözüm sayısını tek programda yazın. `N` artık sabit değil; dizileri her `N` için yeniden kurun (`assign`). 9×9'da kaç çözüm, kaç saniye? 10, 11, 12'yi de deneyin ve süreleri `cerr`'e yazın: her artışta süre kaça katlanıyor?

DÜŞÜNCE

Aynı program, N dışarıdan: dizileri her N için yeniden boyutlandırın.

YAKLAŞIM

Döngüde `sutun.assign(N, false); capraz1.assign(2*N-1, false); capraz2.assign(2*N-1, false); cozum = 0; koy(0);`. Süre için `chrono`, `cerr`'e.

İLK SATIR

```
for (N = 4; N <= 9; ++N) { sutun.assign(N, false); ... koy(0); cout << N << "x" << N << ": " << cozum << endl; }
```

çıktı — BEKLENEN-CİKTİ

```
4x4: 2
5x5: 10
6x6: 4
7x7: 40
8x8: 92
9x9: 352
```

Dizin: `alistirma/kod/IV-5-Z1/`

ZORLAYICI İKİLİ IV-5-Z2

Erken mi, geç mi?

İki sayım: kitapçığınki her veziri koymadan önce denetliyor (*erken*); öteki her satıra rastgele bir vezir koyup ancak tahta dolunca denetliyor (*geç*). İkisi de doğru sayar; `koy()`'un kaç kere çağrıldığını sayın. `N = 6` için iki çağrı sayısını ve çözüm sayısını yazın. Geç denetim kaç kat fazla iş yapıyor? Bu, "uygun mu" koşulunu döngünün içine koymanın değeri; IV.6'daki budamanın ilk örneği.

DÜŞÜNCE

İki sayım: biri koymadan önce denetler, öteki hepsini koyup sonda bakar.

YAKLAŞIM

`erken(satir)`: döngüde `if (not uygun(satir, s)) continue;`. `gec(satir)`: koşulsuz koy, tabanda bütün satırları `uygun` ile denetle. İkisinde de girişte `++cagri`.

İLK SATIR

```
void gec(int satir) { ++cagri; if (satir == N) { if (hepsiUygun()) ++cozum; return; } for (int s{0}; s < N; ++s) { yer[satir] = s; gec(satir + 1); } }
```

çıktı — BEKLENEN-CİKTİ

```
erken: 4 çözüm, 153 çağrı
geç: 4 çözüm, 55987 çağrı
```

Dizin: `alistirma/kod/IV-5-Z2/`

KEŞİF IV-5-K1 **Sudoku** ~3 saat

Aynı kalıp, 81 kare: boş kareyi bul, 1'den 9'a dene, satırda-sütunda-kutuda yoksa koy, derine in, olmadıysa geri al. Girdi 9 satır, boşlar `0`. Çözülmüş tahtayı yazın. Önce `uygun(satir, sutun, rakam)` işlevini yazın ve tek başına sınayın; kutu numarası `(satir/3)*3 + sutun/3`. Kaç çağrıda çözüldü? Zor bir bulmaca bulup onu da deneyin.

> **DÜŞÜNCE**
>
> Sekiz vezirin aynısı, yalnız seçimler 1..9 ve “uygun” üç denetim.

> **YAKLAŞIM**
>
> `uygun`: satırda, sütunda ve `(r/3)*3, (c/3)*3` köşeli 3×3 kutuda rakam var mı. `coz()` ilk `'0'` kareyi bulsun; yoksa `true`.

> **İLK SATIR**
>
> `for (char r{'1'}; r <= '9'; ++r) if (uygun(i, j, r)) { t[i][j] = r; if (coz()) return true; t[i][j] = '0'; }`

çıktı — BEKLENEN-CİKTİ · GİRDİ: 530070000 600195000 098000060 800060003 400803001 700020006 060000280 000419005 000080079

```
534678912
672195348
198342567
859761423
426853791
713924856
961537284
287419635
345286179
```

Dizin: `alistirma/kod/IV-5-K1/`

IV.6

Karmaşıklık ve budama

Kaba kural: saniyede 10^8 işlem. Önce kitapçığın tablosunu kendiniz üretin; sonra ızgarada Hamilton patikaları: budamasız, kitapçığın budamasıyla, kendi budamanızla. Keşif, CSES'in meşhur Izgara yolları sorusu.

1 2 1 1

> **YANIT YERİNE**
>
> kitapçık IV.5: Karmaşıklık algoritma ders 13: Hamilton patikaları algoritma ders 14: arama uzayını budamak

ISINMA IV-6-I1

Tabloyu üretin

Kitapçığın karmaşıklık tablosunu program yazdırsın: n = 20, 1000, 10^6 için `log₂n`, `n`, `n·log₂n`, `n²` ve (yalnız n = 20 için) `2ⁿ`. Değerleri tam sayı olarak, `long long` ile; `log₂n`'i en yakın tam sayıya yuvarlayın (`round`; 1000 için 10). Sonra kaba kuralı uygulayın: her satırın yanına bir saniyede (10^8 işlem) sığıp sığmadığını yazın.

çıktı — BEKLENEN-CİKTİ

```
n=20
  log n   : 4 sığar
  n       : 20 sığar
  n log n : 80 sığar
  n^2     : 400 sığar
  2^n     : 1048576 sığar
n=1000
  log n   : 10 sığar
  n       : 1000 sığar
  n log n : 10000 sığar
  n^2     : 1000000 sığar
n=1000000
  log n   : 20 sığar
  n       : 1000000 sığar
  n log n : 20000000 sığar
  n^2     : 1000000000000 sığmaz
```

Dizin: `alistirma/kod/IV-6-I1/`

ALIŞTIRMA IV-6-A1

Hamilton patikaları, budamasız

Izgarada sol üstten sağ alta, her kareden tam bir kere geçen yollar. Geri dönüşlü arama: dört yöne git, ziyaret edileni işaretle, geri dönerken temizle (III-6-Z1'deki yön listesi ve `icerdeMi`). Patika sayısı ve `git()` çağrı sayısı. Girdi `4 5`. Sonra `4 4` deneyin: sıfır çıkar! Neden? (İpucu: tahtayı satranç tahtası gibi boyayın; her adım renk değiştirir.) 5×5'i de çalıştırıp süreyi not edin; IV-6-A2'de karşılaştıracaksınız.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 5

```
patika: 20, çağrı: 8076
```

Dizin: `alistirma/kod/IV-6-A1/`

ALIŞTIRMA IV-6-A2

Kitapçığın budaması

Aynı program; kitapçığın iki satırlık kuralını ekleyin: üst ve alt dolu (ya da tahta dışı) ama sağ ve sol boşsa — ya tersi — tahta ikiye bölündü, geri dön. Girdi `5 5`. IV-6-A1'i de `5 5` ile çalıştırın: patika sayısı aynı olmalı (değilse budama yanlış), çağrı sayısı çok daha az. Kaç kat? Kural için "dolu"nun tahta dışını da kapsadığına dikkat.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 5

```
patika: 104, çağrı: 7959
```

Dizin: `alistirma/kod/IV-6-A2/`

ZORLAYICI IV-6-Z1

Kendi budamanız

Kitapçığın dördüncü alıştırmasındaki "zor": başka bir budama kuralı bulun. Bir aday: hedefe *erken* varırsak (kalan > 0 iken) o dal ölmüştür, hemen dönün. Bir başkası: hedef kare dört yandan kapanmışsa. Her kuralı eklemeden önce kâğıtta "bu dalda gerçekten çözüm olamaz" diye bir kanıt yazın (kitapçığın beşinci alıştırması). Girdi `5 5`; patika sayısı yine 104 olmalı (değilse kural yanlış). Çağrı sayısı IV-6-A2'den az çıkarsa kuralınız yeni dallar kesiyor; eşit çıkarsa kitapçığın kuralı o dalları zaten kesiyormuş — bu da bir ölçüm. Üç sayıyı yan yana yazın: budamasız, kitapçığın, sizin.

> **DÜŞÜNCE**
>
> Yeni bir budama: hedefe erken varmak ya da hedefin kapanması dalı öldürür.

> **YAKLAŞIM**
>
> `git`'in başında: `if ((r, c) hedef and kalan > 0) return;`; kural numarasına göre `if (kural >= 1)` kitapçığın denetimi, `if (kural == 2)` sizinki. Üç kuralı üç kere çalıştırıp çağrıları yazın.

> **İLK SATIR**
>
> `if (r == S - 1 and c == 0 and kalan > 0) return;  // hedefe erken vardık`

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 5

```
budamasız: 104 patika, 90111 çağrı
kitapçık: 104 patika, 7959 çağrı
sizinki: 104 patika, 7959 çağrı
```

Dizin: `alistirma/kod/IV-6-Z1/`

KEŞİF IV-6-K1 Izgara yolları

CSES 1625 (*Grid Paths*): 7×7 ızgara, sol üstten sol alta, her kareden bir kere; 48 adımlık bir desen veriliyor, `?` serbest, `U D L R` zorunlu. Desene uyan patikaları sayın. Girdi CSES'in örneği; yanıt 201. Budamasız bitmez; kitapçığın kuralı ve IV-6-Z1'deki “hedefe erken varma” ile saniyeler. Daha hızlısı için CSES'in kendi kitabında bir bölüm var (depodaki `kitap/` dizininde Türkçesi).

DÜŞÜNCE

Desendeki harf adımın yönünü kesinleştirir; `?` dört yönü de serbest bırakır.

YAKLAŞIM

Yön listesini `U D L R` sırasında tutun; `desen[adim]` harfse yalnız o yöne gidin. Budamalar: kitapçığın ikiye bölme kuralı ve hedefe erken varma; ikisi birden gerekir.

İLK SATIR

```
for (int y{0}; y < 4; ++y) { if (desen[adim] != '?' and desen[adim] != YON[y]) continue; ... }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: ??????R??????
U??????????????????????????LD????D?

```
uyan patika: 201
```

Dizin: `alistirma/kod/IV-6-K1/`

IV.7

Geri dönüş: altkümeler ve permütasyonlar, kaplarda

Özyineleme üretir, kap toplar. II. bölümün kümesi bu bölümün iki kalıbıyla buluşuyor: altküme toplamlarını ve tekrarlı harflerin permütasyonlarını bir `set`'te biriktirin.

2

YANIT YERİNE

kitapçık II.2: Küme, eşlemin küçük kardeşi kitapçık IV.3: Bütün altkümeler kitapçık IV.3: Bütün permütasyonlar

ALIŞTIRMA IV-7-A1

Altküme toplamları, kümede

Girdi: *n* ve *n* sayı. Bütün altkümeleri üretin (`31-altkume` kalıbı) ama yazdırmayın; her altkümenin toplamını bir `set<int>`'e atın. Çıktı: kaç **farklı** toplam var, sonra toplamlar küçükten büyüğe tek satırda. Küme tekrarı da sıralamayı da sizin yerinize yapıyor; `{3, 1, 2, 4}` için 0'dan 10'a kadar her toplam çıkıyor mu?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 3 1 2 4

```
11 farklı toplam
0 1 2 3 4 5 6 7 8 9 10
```

Dizin: `alistirma/kod/IV-7-A1/`

ALIŞTIRMA IV-7-A2

Tekrarlı harflerin permütasyonları

`aab` yazısının harflerini permütasyon kalıbıyla dizin: altı dizilişten yalnızca üçü farklı (`aab`, `aba`, `baa`). Her dizilişi bir `set<string>`'e atın; küme tekrarları elesin. Girdi: tek satırda yazı. Çıktı: farklı permütasyon sayısı, sonra her biri bir satırda, alfabetik. Sonra II. bölümün `next_permutation`'ıyla aynı işi yapın: o tekrarları kendiliğinden atlar; neden?

çıktı — BEKLENEN-CİKTİ · GİRDİ: AAB

```
3 farklı permütasyon
aab
aba
baa
```

Dizin: `alistirma/kod/IV-7-A2/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

"Bu bölümü geçtim mi?" sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` "Doğru!" desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

`IV-3-Z1` Elmaları bölmek

`IV-1-Z1` Hanoi'yi saymak

`IV-5-Z2` Erken mi, geç mi?

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.

BÖLÜM V

Çizgeler ve Gezintiler

Kitapçığın beşinci bölümü beş satırlık bir kalıp öğretiyor ve onu değiştire değiştire yarım düzine ünlü problemi çözüyor. Burada dokuz konu, kırk alıştırma: çizgeyi saklamak, derinlemesine ve enlemesine gezi, odalar, labirent, iki takım, çok kaynaklı gezi, Dijkstra, Floyd–Warshall, Bellman–Ford. Keşiflerin çoğu CSES'in çizge bölümünden; girdiler CSES'in örnekleri.

Hazırlık · I. İlk Adımlar · II. Veriyi Düzenlemek · III. Zanaat · IV. Özyineleme · V. Çizgeler · VI. Dinamik Programlama

V.1

Çizge nedir, nasıl saklanır?

Noktalar ve bağlar. Bu konunun girdi biçimi bölüm boyunca aynı: ilk satırda nokta ve bağ sayısı, sonra her satırda bir bağ. Komşuluk listesi, komşuluk matrisi, dereceler ve 1736'dan kalma bir soru.

1 2 1

YANIT YERİNE

kitapçık V.1: Çizge nedir ders 19: çizgeler ve ağlar çizge kuramı

ISINMA V-1-I1 Yönsüzden yönlüye

Program bağları *yönsüz* okuyor: her `a b` satırı için hem `a`'nın listesine `b` hem `b`'nin listesine `a` ekleniyor. Tek satır silip çizgeyi *yönlü* yapın: yalnız `a → b`. Çıktıdaki hangi satırlar değişir? Girdi altı noktalı, altı bağlı bir çizge; sonraki alıştırmalar da aynı dosyayı kullanacak.

ana.cpp — BAŞLANGIÇ

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <string>
#include <algorithm>
using namespace std;
using Nokta = int;
template <class T> using Dizi = vector<T>;

// V-1-I1 · Yönsüzden yönlüye: bir satır silin
int main() {
  int n{0}, m{0};
  cin >> n >> m;
  Dizi<Dizi<Nokta>> komsular(n + 1);
  for (int k{0}; k < m; ++k) {
    Nokta a, b; cin >> a >> b;
    komsular[a].push_back(b);
    komsular[b].push_back(a);     // yönsüz: iki yöne de
  }
  for (Nokta p{1}; p <= n; ++p) {
    cout << p << ":";
    for (Nokta q : komsular[p]) cout << " " << q;
    cout << endl;
  }
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 6 6 1 2 1 3 2 4 3 4 4 5 5 6

```
1: 2 3
2: 4
3: 4
4: 5
5: 6
6:
```

Dizin: `alistirma/kod/V-1-I1/`

ALIŞTIRMA V-1-A1

Komşuluk matrisi

Aynı girdi, öteki temsil: `n × n` bir matris, `km[a][b]` bağ varsa 1. Yönsüz okuyun, matrisi satır satır yazın (sayılar arasında boşluk). Matris neden simetrik? Yönlü olsaydı olmazdı; bir deneyin. Kitapçığın tablosu: bu temsil ne zaman iyi?

çıktı — BEKLENEN-ÇİKTİ · GİRDİ: 6 6 1 2 1 3 2 4 3 4 4 5 5 6

```
0 1 1 0 0 0
1 0 0 1 0 0
1 0 0 1 0 0
0 1 1 0 1 0
0 0 0 1 0 1
0 0 0 0 1 0
```

Dizin: `alistirma/kod/V-1-A1/`

ALIŞTIRMA V-1-A2

Dereceler

Bir noktanın **derecesi** komşu sayısı. Her nokta için derecesini yazın; sonra en çok komşusu olan noktayı (eşitlikte küçük numaralı). Derecelerin toplamı bağ sayısının neden tam iki katı? Bu, çizge kuramının ilk teoremi.

çıktı — BEKLENEN-ÇİKTİ · GİRDİ: 6 6 1 2 1 3 2 4 3 4 4 5 5 6

```
1: 2
2: 2
3: 2
4: 3
5: 2
6: 1
en çok komşusu olan: 4 (3)
derece toplamı: 12
```

Dizin: `alistirma/kod/V-1-A2/`

ZORLAYICI V-1-Z1

Königsberg'in köprüleri

Euler'in 1736'daki yanıtı tek bir sayıya bakıyor: derecesi *tek* olan nokta sayısı. Sıfırsa her köprüden bir kere geçip başa dönen bir yürüyüş var (Euler çevrimi); tam ikiyse başa dönmeyen bir yürüyüş var (Euler yolu); başka bir sayıysa yok. Girdi Königsberg'in kendisi: dört kara parçası, yedi köprü (aynı iki nokta arasında birden çok bağ olabilir, buna dikkat). Programınız üç durumdan hangisi olduğunu söylesin; sonra girdiyi değiştirip öteki iki durumu da üretin.

DÜŞÜNCE

Euler'in ölçütü yalnızca derecelere bakıyor; çoklu bağlar dereceyi iki kere artırır.

YAKLAŞIM

`vector<int> derece(n + 1)`; her bağ için iki ucu bir artırın; tek dereceli sayısını sayın; 0 / 2 / başka diye üç dal.

İLK SATIR

```
for (int k{0}; k < m; ++k) { int a, b; cin >> a >> b; ++derece[a]; ++derece[b]; }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 7 1 2 1 2 1 3 1 3 1 4 2 4 3 4

```
tek dereceli nokta sayısı: 4
Euler yolu yok: her köprüden tam bir kere geçmek olanaksız
```

Dizin: `alistirma/kod/V-1-Z1/`

V.2

Beş satırlık kalıp: derinlemesine gezi

Kitapçığın `40-gezi.cpp`'si: aynı çizge, üç gezi. Buradan erişilebilen noktalara, "yol var mı" sorularına, gezi ağacına ve yığınlı gezinin sırasını özyinelemeliyle eşleştirmeye.

1 2 1 1

YANIT YERİNE

kitapçık V.2: Beş satırlık kalıp ders 20: çizgelerde gezintiler algoritma ders 16: derinlemesine gezi kalıpları kod: 40-gezi.cpp

ISINMA V-2-I1

Üçten başlayın

Üç geziyi de 1 yerine 3'ten başlatın. Çizge yönlü; 3'ten 2'ye bir yol var mı, yedi nokta da görünüyor mu? Önce kâğıtta okları izleyin, sonra çalıştırın. Sonra `bagla(6, 1)` satırını silip tekrar bakın: şimdi hangileri kayboldu, neden?

çıktı — BEKLENEN-CİKTİ

```
derinlemesine (özyineleme): 3 4 6 1 2 5 7
derinlemesine (yığınla)   : 3 4 7 5 6 1 2
enlemesine    (kuyrukla)  : 3 4 6 7 1 5 2
```

Dizin: `alistirma/kod/V-2-I1/`

ALIŞTIRMA V-2-A1

Erişilebilenler

Girdiden yönlü çizge okuyun (kitapçığın yedi noktalı çizgesi), 1'den derinlemesine gezin; erişilebilen noktaları küçükten büyüğe ve sayısını yazın. Gezinin sırası değil, kümesi önemli: `gezildi` dizisini sonunda tarayın.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 7 9 1 2 1 3 2 5 3 4 4 6 4 7 5 7 6 1 7 5

```
1'den erişilebilen: 1 2 3 4 5 6 7
7 nokta
```

Dizin: `alistirma/kod/V-2-A1/`

ALIŞTIRMA V-2-A2

Yol var mı?

Çizgeden sonra sorular geliyor: her satırda `a b` ; `a` 'dan `b` 'ye yol var mı? Her soru için geziyi sıfırlayıp `a` 'dan başlatın; `evet` ya da `hayır` . Girdi kitapçığın çizgesi ve dört soru. Sorular çoğalınca her seferinde baştan gezmek pahalı olur; V.3 bunun çaresini gösterecek (yönsüz çizgede).

çıktı — BEKLENEN-CİKTİ · GİRDİ: 7 9 1 2 1 3 2 5 3 4 4 6 4 7 5 7 6 1 7 5 4 1 7 3 2 6 5 7 6

```
1 -> 7: evet
3 -> 2: evet
6 -> 5: evet
7 -> 6: hayır
```

Dizin: `alistirma/kod/V-2-A2/`

ZORLAYICI V-2-Z1

Gezi ağacı

Derinlemesine gezi bir ağaç çizer: her noktaya ilk kez hangi noktadan geldik? Kalıba iki girdi ekleyin: `gez(bu, nereden, derinlik)`. Her nokta için "nereden, derinlik" yazın (1 için kök). Girdi kitapçığın çizgesi. En derin nokta hangisi? Kitapçığın çıktısındaki sırayla (`1 2 5 7 3 4 6`) karşılaştırın: ağaç o sırayı açıklıyor mu?

DÜŞÜNCE

Her noktaya ilk kez geldiğimiz nokta ve o andaki derinlik ağacın kendisi.

YAKLAŞIM

`gez(bu, onceki, d)`: girişte `nereden[bu] = onceki; derinlik[bu] = d;`, komşulara `gez(su, bu, d + 1)`. En derin için derinlik dizisinin en büyüğü.

İLK SATIR

```
void gez(Nokta bu, Nokta onceki, int d) { if (gezildi[bu]) return; gezildi[bu] = true; nereden[bu] = onceki; derinlik[bu] = d; for (Nokta su : komsular[bu]) gez(su, bu, d + 1); }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 7 9 1 2 1 3 2 5 3 4 4 6 4 7 5 7 6 1 7 5

```
1: kök
2: nereden=1 derinlik=1
3: nereden=1 derinlik=1
4: nereden=3 derinlik=2
5: nereden=2 derinlik=2
6: nereden=4 derinlik=3
7: nereden=5 derinlik=3
en derin: 6 (3)
```

Dizin: `alistirma/kod/V-2-Z1/`

KEŞİF V-2-K1 Yığın, özyinelemeyle aynı sırada

Kitapçığın çıktısında yığınlı gezi (`1 3 4 7 5 6 2`) özyinelemeliden (`1 2 5 7 3 4 6`) farklı sıra veriyor. Neden? Komşular yığına 2, 3 sırasıyla giriyor, 3 üstte kalıyor, önce o çıkıyor. `gez2`'yi öyle değiştirin ki sıra özyinelemeliyle aynı olsun; tek satır yeter (`<algorithm>`'da tersten yineleyici var, ya da döngüyü tersten kurun). Sonra düşünün: iki gezi de *doğru*; sıra ne zaman önemli olur?

DÜŞÜNCE

Yığın son koyduğunu önce verir; komşuları ters sırada koyarsanız ilkini önce alırsınız.

YAKLAŞIM

`for (auto it = komsular[bu].rbegin(); it != komsular[bu].rend(); ++it) yigin.push(*it);` — ya da indeksle tersten döngü.

İLK SATIR

```
for (auto it = komsular[bu].rbegin(); it != komsular[bu].rend(); ++it) yigin.push(*it);
```

çıktı — BEKLENEN-CİKTİ

```
derinlemesine (özyineleme): 1 2 5 7 3 4 6
derinlemesine (yığınla)   : 1 2 5 7 3 4 6
enlemesine    (kuyrukla)  : 1 2 3 5 4 7 6
```

Dizin: `alistirma/kod/V-2-K1/`

V.3

Kaç oda var? Bağlı parçalar

Izgarada görünmez çizge. Kitapçığın “etrafı duvarla çevirin” numarası, CSES'in *Counting Rooms* ve *Building Roads* soruları. Girdi biçimi CSES'inki: satır sütun, sonra plan.

1 2 1

YANIT YERİNE

kitapçık V.3: Kaç oda var algoritma ders 15: oda sayımı algoritma ders 16: yol ağı tasarımı

ISINMA V-3-I1 Çapraz da komşu olsun

Program odaları sayıyor; dört yönlü komşuluk. Komşuluğu sekiz yöne çıkarın (çaprazlar da bağlasın): oda sayısı ne olur? Girdi CSES'in örneğinin bir duvarı açılmış hâli (dört yönlüde yine 3 oda). Çıktı iki satır olsun: dört yönlü ve sekiz yönlü sayım. Duvar çerçevesi sayesinde köşegen çağrılar da taşmıyor; neden?

ana.cpp — BAŞLANGIÇ

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <string>
#include <algorithm>
using namespace std;
using Nokta = int;
template <class T> using Dizi = vector<T>;

// V-3-I1 · Çapraz da komşu olsun
// Çıktı: "4 yönlü: 3" ve "8 yönlü: ?"
using Tahta = Dizi<Dizi<bool>>;   // true: boş ve gezilmemiş

bool gez(int y, int x, Tahta& t) {
  if (not t[y][x]) return false;
  t[y][x] = false;
  gez(y - 1, x, t); gez(y + 1, x, t); gez(y, x - 1, t); gez(y, x + 1, t);
  return true;
}

Tahta oku(int& n, int& m) {
  cin >> n >> m;
  Tahta t(n + 2, Dizi<bool>(m + 2, false));     // çerçeve: duvar
  for (int y{1}; y <= n; ++y) {
    string satir; cin >> satir;
    for (int x{1}; x <= m; ++x) t[y][x] = satir[x - 1] == '.';
  }
  return t;
}

int main() {
  int n, m;
  Tahta t{oku(n, m)};
  int oda{0};
  for (int y{1}; y <= n; ++y) for (int x{1}; x <= m; ++x) if (gez(y, x, t)) ++oda;
  cout << "4 yönlü: " << oda << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 8 ######### #..#...# ###..#.# #..#...# #########

```
4 yönlü: 3
8 yönlü: 1
```

Dizin: `alistirma/kod/V-3-I1/`

ALIŞTIRMA V-3-A1 Odaları sayın, yığınla

Kitapçığın birinci alıştırması: CSES 1192, bu sefer özyineleme yerine `std::stack` ile. V-3-I1'in `gez`'ini yığınlı yazın (40-gezi'deki `gez2` kalıbı, karelerle). Çıktı yalnız oda sayısı; CSES'e gönderin. Büyük haritada (1000×1000) özyinelemeli sürüm neden çökebilir, yığınlı neden çökmez?

çıktı

```
BEKLENEN-CİKTİ · GİRDİ: 5 8 ######### #..#...# #####.#.# #..#...# #########
```

```
3
```

Dizin: `alistirma/kod/V-3-A1/`

ALIŞTIRMA V-3-A2 En büyük oda

Odaları saymak yetmiyor; en büyüğü kaç kare? `gez` bir `bool` yerine *kare sayısı* döndürsün: kendisi 1, artı dört komşudan dönenlerin toplamı. Girdi aynı; çıktı oda sayısı ve en büyük odanın kare sayısı.

çıktı

```
BEKLENEN-CİKTİ · GİRDİ: 5 8 ######### #..#...# #####.#.# #..#...# #########
```

```
3 oda, en büyüğü 8 kare
```

Dizin: `alistirma/kod/V-3-A2/`

ZORLAYICI İKİLİ V-3-Z1

Yol ağı tasarımı

CSES 1666, kitapçığın üçüncü alıştırması: n şehir, bazıları arasında yol; herkesin herkese ulaşması için en az kaç yeni yol gerekir, hangileri? Kitapçığın `parcayaEkle` kalıbı: parçaları numaralayın, her parçanın temsilcisini saklayın, temsilcileri zincir gibi bağlayın. Girdi CSES örneği (`4 2`, yollar 1-2, 3-4). Çıktı: yeni yol sayısı, sonra yollar. Başka bir doğru yanıt da olabilir; CSES hepsini kabul eder, bizim beklenen çıktı temsilcileri küçükten büyüğe zincirliyor.

DÜŞÜNCE

Her parçanın bir temsilcisi var; parçaları ardışık temsilcilerle bağlamak yeter.

YAKLAŞIM

`parcayaEkle` ile her noktayı numaralayın; her parça numarası için ilk görülen noktayı `temsilci` dizisine yazın; yeni yollar `temsilci[i]–temsilci[i+1]`.

İLK SATIR

```
vector<Nokta> temsilci; for (Nokta p{1}; p <= n; ++p) if (parca[p] == 0) { temsilci.push_back(p); parcayaEkle(p, temsilci.size()); }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 2 1 2 3 4

```
1
1 3
```

Dizin: `alistirma/kod/V-3-Z1/`

V.4

Labirentte en kısa yol: enlemesine gezi

Kuyruk, noktaları başlangıca uzaklık sırasıyla işler; bu yüzden ilk varış en kısa yoldur. Kitapçığın `43-labirent.cpp`'sinden uzaklık haritasına, CSES'in *Labyrinth*'ine, genel çizgede en kısa yola ve satranç atına.

1 2 1 1

YANIT YERİNE

kitapçık V.3: Labirentte en kısa yol algoritma ders 15: labirentte yol bulma algoritma ders 17: enlemesine gezi kod: 43-labirent.cpp

ISINMA V-4-I1 Önce B'yi taşıyın, sonra bir duvar yıkın

Kitapçığın labirentinde A'dan B'ye 10 adım: dört aşağı, altı sağa; bu kadar kısa bir yolu hiçbir duvar yıkımı kısaltamaz (neden?). İki değişiklik yapın. Önce B'yi ortadaki odaya taşıyın: dördüncü satırı `#.#.B.#.#` yapın, altıncı satırdaki B'yi `.` yapın; yol kaç adıma uzadı? Sonra ikinci satırda A'nın sağındaki duvarı yıkın (`#A.....#`); şimdi kaç adım, tarifi ne? Her ikisini de çalıştırmadan önce kâğıtta sayın. Beklenen çıktı ikinci durumun.

```
● çıktı                                                    BEKLENEN-CİKTİ

en kısa yol 5 adım: RRDDR
```

Dizin: `alistirma/kod/V-4-I1/`

ALIŞTIRMA V-4-A1 Uzaklık haritası

Yol tarifi yerine harita: A'dan her kareye kaç adım? Enlemesine gezi her karenin uzaklığını "geldiğim karenin uzaklığı + 1" diye yazsın. Haritayı basın: duvar `##`, ulaşılamayan `..`, ötekiler iki haneli uzaklık (`setw(2)`). Girdi CSES biçiminde (satır sütun, plan; A başlangıç, B hedef). B'deki sayı V-4-A2'nin yanıtı olacak.

```
● çıktı   BEKLENEN-CİKTİ · GİRDİ: 5 8 ######### #.A#...# #.##.#B# #.....# #########

#################
## 1 0## 8 910##
## 2#### 7## 9##
## 3 4 5 6 7 8##
#################
```

Dizin: `alistirma/kod/V-4-A1/`

ALIŞTIRMA V-4-A2

Labyrinth

CSES 1193, kitapçığın ikinci alıştırması: A'dan B'ye var mı, kaç adım, tarifi (`U D L R`). Kitapçığın "nereden geldim" numarası: her kareye geliş yönünü yazın, B'den geriye yürüyüp tarifi okuyun, ters çevirin. Çıktı CSES biçimi: `YES`, adım sayısı, tarif; ya da `NO`. Birden çok en kısa yol olabilir; bizim beklenen çıktı yönleri U, D, L, R sırasıyla deneyen sürümünki.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 8 ######## #.A#...# #.##.#B# #......# ########

```
YES
9
LDDRRRRRU
```

Dizin: `alistirma/kod/V-4-A2/`

ZORLAYICI V-4-Z1

Genel çizgede en kısa yol

Izgara değil, komşuluk listesi: yönsüz çizgede 1'den her noktaya en az kaç bağ? Enlemesine gezi, `uzaklik` ve `nereden` dizileriyle. Her nokta için uzaklığı yazın, sonra 1'den n'e yolu (`nereden` zincirini geri sararak). Girdi altı noktalı çizge. Yol yoksa `yol yok`.

DÜŞÜNCE

Enlemesine gezi uzaklığı verir; `nereden` dizisi yolu geri sarar.

YAKLAŞIM

Kuyrukla gezide komşuya girerken `uzaklik[su] = uzaklik[bu] + 1; nereden[su] = bu;`. Yolu `n`'den başlayıp `nereden`'i izleyerek bir vektöre koyun, sonra ters çevirin.

İLK SATIR

```
for (Nokta p{n}; p != 0; p = nereden[p]) yol.push_back(p); reverse(yol.begin(), yol.end());
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 6 6 1 2 1 3 2 4 3 4 4 5 5 6

```
1: 0
2: 1
3: 1
4: 2
5: 3
6: 4
1 -> 6: 1 2 4 5 6
```

Dizin: `alistirma/kod/V-4-Z1/`

KEŞİF V-4-K1

Satranç atı ~2 saat

At, 8×8 tahtada `a1`'den `h8`'e en az kaç hamlede gider? Çizge görünmez: her karenin komşuları atın sekiz sıçraması. Enlemesine gezi; girdi iki kare (`a1 h8`). Sonra kitapçığın *Neden en kısa?* kutusunu kendi sözlerinizle yazın: kuyruktan çıkan karelerin uzaklığı neden hiç azalmaz? Bu bir tümevarım; dört beş cümle yeter, kod yazmaktan daha değerli.

DÜŞÜNCE

Tahta bir çizge: kare düğüm, atın sıçraması bağ; komşular hesaplanır, saklanmaz.

YAKLAŞIM

Sekiz sıçramayı iki dizide tutun (`DX`, `DY`); kuyrukla gezin, `uzaklik[8][8]` −1 ile başlasın. Harfi ve rakamı sayıya çevirme kodu hazır.

İLK SATIR

```
const int DX[8]{1,2,2,1,-1,-2,-2,-1}, DY[8]{2,1,-1,-2,-2,-1,1,2};
```

çıktı — BEKLENEN-ÇİKTİ · GİRDİ: A1 H8

```
a1 -> h8: 6 hamle
```

Dizin: `alistirma/kod/V-4-K1/`

V.5

İki takım kurmak; çevrim bulmak

Aynı kalıp, “gezildi” yerine “hangi takımda”. Kitapçığın iki gerçek hatasından birini siz bulacaksınız; sonra CSES'in *Building Teams* ve *Round Trip* soruları.

1 2 1

YANIT YERİNE

kitapçık V.3: İki takım kurmak algoritma ders 18: iki parça çizgeler algoritma ders 19: çevrim bulma

ISINMA V-5-I1 Kitapçığın ilk hatası

Program iki takım kuruyor; ama kitapçığın *İki hata, iki ders* kutusundaki ilk hata burada: bir `else` eksik (süslü ayraçlar yerinde, yalnız sözcük yok), taban durumu yok, gezi durmuyor (çökene kadar kendini çağırır). Yeri bulup `else`'i koyun. Girdi: beş kişi, üç arkadaşlık. Kitapçık bu hatayı `gdb` ile bulmuş; siz de deneyin: `backtrace` sonsuz tekrarı gösterir.

ana.cpp — BAŞLANGIÇ

```
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <string>
#include <algorithm>
using namespace std;
using Nokta = int;
template <class T> using Dizi = vector<T>;

// V-5-I1 · Kitapçığın ilk hatası: eksik else
enum class Takim { Yok, Bir, Iki };
Dizi<Dizi<Nokta>> sinif;
Dizi<Takim> takim;

bool gez(Nokta a) {
  for (Nokta b : sinif[a]) {
    if (takim[b] != Takim::Yok) {
      if (takim[b] == takim[a]) return false;
    }
    {                                    // buraya bir sözcük eksik
      takim[b] = (takim[a] == Takim::Bir) ? Takim::Iki : Takim::Bir;
      if (not gez(b)) return false;
    }
  }
  return true;
}

int main() {
  int n{0}, m{0};
  cin >> n >> m;
  sinif.assign(n + 1, {}); takim.assign(n + 1, Takim::Yok);
  for (int k{0}; k < m; ++k) { Nokta a, b; cin >> a >> b; sinif[a].push_back(b);
sinif[b].push_back(a); }
  for (Nokta p{1}; p <= n; ++p)
    if (takim[p] == Takim::Yok) {
      takim[p] = Takim::Bir;
      if (not gez(p)) { cout << "IMPOSSIBLE" << endl; return 0; }
    }
  for (Nokta p{1}; p <= n; ++p) cout << (takim[p] == Takim::Bir ? 1 : 2) << " ";
  cout << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 3 1 2 1 3 4 5

```
1 2 2 1 2
```

Dizin: `alistirma/kod/V-5-I1/`

ALIŞTIRMA V-5-A1

Üçgen

Kitapçığın dördüncü alıştırması: CSES 1668. V-5-I1'in düzeltilmiş programı bunu zaten çözüyor; şimdi mümkün olmayan bir girdi: 1-2, 2-3, 3-1 üçgeni. Çıktı `IMPOSSIBLE`. Sonra düşünün: hangi çizgeler ikiye bölünemez? Kitapçığın ipucu: tek sayıda bağdan oluşan bir çevrim. Girdiye bir dörtgen (çift çevrim) ekleyip bölünebildiğini de görün.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 3 1 2 2 3 3 1

```
IMPOSSIBLE
```

Dizin: `alistirma/kod/V-5-A1/`

ALIŞTIRMA V-5-A2

Çevrim var mı?

Yönsüz çizgede çevrim var mı? Derinlemesine gezide zaten gezilmiş bir komşuya rastlamak çevrim demek — *bir istisnayla*: az önce geldiğimiz nokta (yönsüz bağ iki yöne de görünür). Kalıba "bir önceki nokta" girdisini ekleyin. Girdi altı noktalı çizge: çevrim var (1-2-4-3). Çıktı `çevrim var` ya da `çevrim yok`; 5-6 bağını silip bir de ağaç deneyin.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 6 6 1 2 1 3 2 4 3 4 4 5 5 6

```
çevrim var
```

Dizin: `alistirma/kod/V-5-A2/`

ZORLAYICI V-5-Z1

Gidiş dönüş

CSES 1669 (*Round Trip*), kitapçığın beşinci alıştırması: çevrimi *yazın*. V-5-A2'ye `nereden` dizisini ekleyin; gezilmiş bir komşuya (`su`) rastlayınca `bu` 'dan `nereden` zinciriyle `su` 'ya kadar geri yürüyün, başa `su` 'yu da koyun. Çıktı CSES biçimi: çevrimin nokta sayısı (başlangıç iki kere sayılır), sonra noktalar. Girdi CSES örneği. Başka çevrimler de doğru; bizim beklenen çıktı komşuları girdi sırasıyla gezen sürümünki.

DÜŞÜNCE

Gezilmiş bir komşuya (geldiğin yer hariç) rastlamak çevrim demek; yolu `nereden` 'den okuyun.

YAKLAŞIM

`gez(bu, onceki)` komşu `su` gezilmişse ve `su != onceki` ise: `bu` 'dan `nereden` zinciriyle `su` 'ya kadar noktaları toplayın, başa `su` koyun, `true` döndürün.

İLK SATIR

```
for (Nokta p{bu}; p != su; p = nereden[p]) cevrim.push_back(p); cevrim.push_back(su); reverse(...); cevrim.insert(cevrim.begin(), su);
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 6 1 3 1 2 5 3 1 5 2 4 4 5

```
4
1 5 3 1
```

Dizin: `alistirma/kod/V-5-Z1/`

V.6

Bir kuyruk, birçok başlangıç

Bütün kaynakları aynı kuyruğa koyun; tek gezi, iş bölümü. Kitapçığın kırk kat hızlanan canavar problemi burada: önce uzaklık haritası, sonra iki yöntemin karşılaştırması, sonra CSES'in *Monsters* sorusu.

1 1 2 1

YANIT YERİNE

kitapçık V.4: Bir kuyruk, birçok başlangıç algoritma ders 20: canavarlardan kaçış, giriş algoritma ders 21: canavarlardan kaçış

ISINMA V-6-I1 En yakın canavar

Program her karenin en yakın canavara uzaklığını hesaplıyor: bütün `M`'ler önce kuyruğa, sonra tek gezi. Haritayı basıyor. Bir satır ekleyin: canavardan en uzak kare kaç adım uzakta? Girdi CSES'in canavar örneği.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 8 ######### #M..A..# #.#.M#.# #M#..#.. #.#######

```
#################
## 0 1 2 1 2 3##
## 1## 1 0## 4##
## 0## 2 1## 5 6
## 1#############
en uzak kare: 6 adım
```

Dizin: `alistirma/kod/V-6-I1/`

ALIŞTIRMA V-6-A1 Yavaş ve hızlı, yan yana

Kitapçığın iki parçasını aynı programda yazın: *yavaş* (her canavar için ayrı gezi, her karede küçüğü tut) ve *hızlı* (hepsi aynı kuyrukta). İkisi de aynı haritayı vermeli; kuyruktan kaç kare çıktığını sayın. Çıktı: `aynı` ya da `FARKLI`, sonra iki sayı. Girdi aynı. Kitapçığın `c × n × m` / `c + n × m` oranı burada kaç?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 8 ######### #M..A..# #.#.M#.# #M#..#.. #.#######

```
aynı
yavaş: 48 çıkarma
hızlı: 16 çıkarma
```

Dizin: `alistirma/kod/V-6-A1/`

ZORLAYICI İKİLİ V-6-Z1

Monsters ~2 saat

CSES 1194: labirentin kenarına canavarlardan önce varabilir misiniz? İki gezi: önce canavarların uzaklık haritası (V-6-I1), sonra sizin geziniz — bir kareye yalnız canavardan *daha erken* varabiliyorsanız girin. Kenara varınca `YES`, adım sayısı ve tarif (V-4-A2'deki "nereden" numarası). Girdi CSES örneği; beklenen çıktı U, D, L, R sırasıyla deneyen sürümünki, başka tarifler de doğru.

DÜŞÜNCE

İki uzaklık haritası: canavarlarınki önce, sizinki sonra; bir kareye yalnız erken varırsanız girin.

YAKLAŞIM

Çok kaynaklı gezi (bütün `M`'ler kuyruğa) → `canavar[r][c]`; sonra `A`'dan gezi, komşuya geçiş koşulu `benim + 1 < canavar[nr][nc]`; kenara varınca `nereden` ile tarifi geri sarın.

İLK SATIR

```
if (uzak[nr][nc] == -1 and uzak[r][c] + 1 < canavar[nr][nc]) { uzak[nr][nc] = uzak[r][c] + 1; nereden[nr][nc] = {r, c}; kuyruk.push({nr, nc}); }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 8 ######## #M..A..# #.#.M#.# #M#..#.. #.#######

```
YES
5
RRDDR
```

Dizin: `alistirma/kod/V-6-Z1/`

ZORLAYICI V-6-Z2

En güvenli kare

Tersine soru: duvarlardan en uzak kare hangisi? Bütün duvarları (ve tahtanın dışını) kaynak yapın; her boş karenin en yakın duvara uzaklığı çıkar; en büyüğünü bulun. Çerçeve numarası burada işinize yarar: dışarıyı da duvar sayın. Girdi aynı plan; çıktı en güvenli kare(ler)in uzaklığı ve kaç kare olduğu.

DÜŞÜNCE

Her boş karenin en yakın duvara uzaklığı: duvarları kaynak yapıp enlemesine gezin.

YAKLAŞIM

Haritayı bir satır ve sütun çerçeveyle büyütüp dışı duvar sayın; bütün duvarlar uzaklık 0 ile kuyruğa; gezi bitince boş karelerin en büyük uzaklığı ve o uzaklıkta kaç kare.

İLK SATIR

```
vector<string> g(n + 2, string(m + 2, '#')); for (int r{0}; r < n; ++r) for (int c{0}; c < m; ++c) g[r+1][c+1] = plan[r][c];
```

çıktı

```
BEKLENEN-CİKTİ · GİRDİ: 5 8 ######### #M..A..# #.#.M#.# #M#..#.. #.#######
```

```
en yakın duvara en uzak: 1 adım, 16 kare
```

Dizin: `alistirma/kod/V-6-Z2/`

KEŞİF V-6-K1

Monsters, büyük haritada

V-6-Z1'in programı, 40×60'lık üretilmiş bir haritada (girdi hazır). Önce yanıtı alın; sonra III.5'teki gibi ölçün: canavar sayısını kat kat artırsanız “yavaş” yöntem kaç kat yavaşlar, “hızlı” ne kadar? Sonra CSES'e gönderin (1000×1000, 1 s sınırı). Bir adım ötesi: canavarın da sizin de aynı anda vardığınız kareyi kim alır; CSES'in kuralı ile programınızın `>=`'si uyuşuyor mu?

DÜŞÜNCE

Aynı program, büyük girdi; iş ölçmek ve sınır durumunu okumak.

YAKLAŞIM

V-6-Z1'i kopyalayın; `chrono` ile süreyi `cerr`'e yazın; canavar sayısını girdide artırıp yeniden ölçün. CSES kuralı: aynı anda varılan kare canavarındır, `<` katı olmalı.

İLK SATIR

```
auto t0 = chrono::steady_clock::now();
```

BEKLENEN-CİKTİ · GİRDİ: 40 60



çıktı

```
YES
19
DRRRDRRRURRRRURRRRU
```

Dizin: `alistirma/kod/V-6-K1/`

V.7

Dijkstra: bir noktadan hepsine

Enlemesine gezi, kuyruğu öncelik sırasıyla değiştirince ağırlıklı yollara da çalışır. Kitapçığın `41-dijkstra.cpp`'si, yol kurma, Dijkstra'yı kırmak (eksi ağırlık), “eski kayıt” satırı ve CSES'in *Flight Discount*'u.

1 2 1 1

YANIT YERİNE

kitapçık V.5: Dijkstra algoritma ders 23: en kısa yol (Dijkstra) algoritma ders 22: omuzda taşıyarak (USACO) kod: 41-dijkstra.cpp

ISINMA V-7-I1

Ulaşılamayan şehir

Kitapçığın girdisine ulaşılamayan dördüncü bir şehir ekledik. Program onun uzaklığı için `SONSUZ`'u (10^{18}) basıyor; bunu `-1` yapın. Tek satır. Sonra sorun: `SONSUZ` neden `1e18`, neden `numeric_limits<long long>::max()` değil? (İpucu: `u + sure` taşar mı?)

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 4 1 2 6 1 3 2 3 2 3 1 3 4

```
0 5 2 -1
```

Dizin: `alistirma/kod/V-7-I1/`

ALIŞTIRMA V-7-A1

Yolu da yazın

Uzaklık yetmez, rota lazım: `nereden[su] = bu` kaydını uzaklık iyileştiğinde tutun; sonunda n'den geriye yürüyün (V-4-Z1'deki gibi). Çıktı: 1'den her şehre uzaklık satırı, sonra `1 -> n` yolu. Girdi beş şehir, yedi hat (yönlü). Bir şehre iki yoldan aynı uzaklıkla varılırsa hangisi kaydedilir?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 7 1 2 4 1 3 1 3 2 2 2 4 5 3 4 8 4 5 3 3 5 10

```
1: 0
2: 3
3: 1
4: 8
5: 11
1 -> 5: 1 3 5 (uzaklık 11)
```

Dizin: `alistirma/kod/V-7-A1/`

ALIŞTIRMA V-7-A2 Dijkstra'yı kırın

Kitapçığın yedinci alıştırması. Girdide eksi ağırlıklı bir bağ var (beş şehir). Aynı programda iki hesap yapın: *klasik* Dijkstra ve *n−1 tur gevşetme* (Bellman–Ford'un en sade hâli, V.9'da ayrıntısı). Klasik Dijkstra: sıradan çıkan şehri bir `bitti` dizisinde işaretleyin ve bitmiş şehre bir daha dokunmayın (ne yeniden işleyin ne uzaklığını güncelleyin). Dikkat, kitapçığın `41-dijkstra.cpp`'si farklı: o yalnız eski kaydı atlar, 2'yi yeniden açar ve bu girdide doğru sonucu verir; neden? İki sonucu alt alta yazın; hangi şehirlerde ayrışıyorlar, neden? Dijkstra 2'yi "işi bitti" sayıp bir daha bakmıyor (bitmiş şehre dokunmuyor); eksi bağ o kararı sonradan bozuyor, hata 5'e de taşınıyor.

çıktı BEKLENEN-CİKTİ · GİRDİ: 5 5 1 2 1 2 5 1 1 3 5 3 4 -6 4 2 1

```
Dijkstra : 0 1 5 -1 2
gevşetme : 0 0 5 -1 1
```

Dizin: `alistirma/kod/V-7-A2/`

ZORLAYICI V-7-Z1

Eski kayıt

`if (uzaklik[bu] < u) continue;` satırı ne yapıyor? Bir şehir sıraya birden çok kere girebilir (her iyileşmede bir kayıt); eskisi çıkınca atlanır. Satırı sayaçlı yapın: kaç kayıt atlandı? Sonra satırı silip programı yine çalıştırın: sonuç değişir mi, kaç hat gevşetildi? İki sayıyı da yazın. Girdi V-7-A1'inki.

DÜŞÜNCE

Aynı şehir sıraya birden çok kere girebilir; eski kayıt çıkınca atlanmalı.

YAKLAŞIM

`if (uzaklik[bu] < u) { ++atlanan; continue; }`; ikinci sürümde satırı silip her çıkışta gevşetme sayacını artırın. İki sürümü iki işlev yapın, aynı girdiyle çağırın.

İLK SATIR

```
long long atlanan{0}, gevsetme{0};
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 7 1 2 4 1 3 1 3 2 2 2 4 5 3 4 8 4 5 3 3 5 10

```
1: 0
2: 3
3: 1
4: 8
5: 11
atlanan eski kayıt: 2
satırsız: aynı, gevşetilen hat: 7 yerine 9
```

Dizin: `alistirma/kod/V-7-Z1/`

KEŞİF V-7-K1 Uçuş indirimi

CSES 1195 (*Flight Discount*): bir uçuşun bedelini yarıya indirme hakkınız var (bir kere, aşağı yuvarlayarak); 1'den n'e en ucuz yolculuk? Hile: her şehrin iki hâli var, "indirim kullanılmadı" ve "kullanıldı"; Dijkstra'yı `2n` düğümlü bu çizgede çalıştırın. Girdi CSES örneği; yanıt 2. Bir adım ötesi: indirim hakkı k olsaydı?

DÜŞÜNCE

Her şehrin iki hâli; indirim kullanılmadan ve kullanılarak varmak farklı düğümler.

YAKLAŞIM

`uzaklik[sehir][0/1]`; her hat için iki geçiş: aynı hâlde tam bedelle, 0'dan 1'e yarı bedelle (`s / 2`). Yanıt `min(uzaklik[n][0], uzaklik[n][1])`.

İLK SATIR

```
priority_queue<tuple<U, Nokta, int>, vector<...>, greater<>>
sira;
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 4 1 2 3 2 3 1 1 3 7 2 1 5

```
2
```

Dizin: `alistirma/kod/V-7-K1/`

V.8

Floyd–Warshall: herkesten herkese

Üç döngü, o kadar. Kitapçığın `42-floyd.cpp`'si, CSES'in *Shortest Routes II*'si, yolu geri kurmak ve erişilebilirlik matrisi.

1 1 1 1

YANIT YERİNE

kitapçık V.5: Floyd–Warshall algoritma ders 24: bütün uzaklıklar floyd-warshall (kanıt)
kod: 42-floyd.cpp

ISINMA V-8-I1

Bir yol daha

Kitapçığın dört şehrine bir yol ekleyin: `yol(1, 3, 1)`. Matristeki hangi hücreler değişir? Önce kâğıtta: 1'den 3'e artık 1; 2'den 4'e? 1 üzerinden 5+1+... Hesaplayın, sonra çalıştırın.

çıktı — BEKLENEN-CİKTİ

```
      1   2   3   4
    ---------------
1 |   0   3   1   8
2 |   3   0   2   9
3 |   1   2   0   7
4 |   8   9   7   0
```

Dizin: `alistirma/kod/V-8-I1/`

ALIŞTIRMA V-8-A1

En kısa yollar II

CSES 1672 (*Shortest Routes II*): n şehir, m çift yönlü yol, q soru; her soru için iki şehir arasındaki en kısa uzaklık, yol yoksa `-1`. Girdi CSES örneği. Aynı iki şehir arasında birden çok yol olabilir: `min` ile okuyun. `SONSUZ + SONSUZ` taşmasın diye `1e15`'in neden yeterli olduğunu düşünün.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 3 5 1 2 5 1 3 9 2 3 3 1 2 2 1 1 3 1 4 3 2

```
5
5
8
-1
3
```

Dizin: `alistirma/kod/V-8-A1/`

ZORLAYICI V-8-Z1

Yolu geri kurmak

Floyd–Warshall uzaklığı veriyor, yolu değil. İkinci bir matris tutun: `sonraki[a][b]`, a'dan b'ye giderken ilk adım. Yol okununca (`f[a][b] > f[a][o] + f[o][b]`) `sonraki[a][b] = sonraki[a][o]` . Sonunda a'dan b'ye yolu `sonraki` 'yi izleyerek yazın. Girdi kitapçığın dört şehri ve iki soru (`1 4` , `2 4`).

DÜŞÜNCE

`sonraki[a][b]` a'dan b'ye ilk adım; her iyileşmede ara şehrin ilk adımını devralır.

YAKLAŞIM

Başta `sonraki[a][b] = b` her yol için; üç döngüde `if (f[a][o] + f[o][b] < f[a][b]) { f[a][b] = ...; sonraki[a][b] = sonraki[a][o]; }` . Yol: `a` 'dan `b` 'ye `sonraki` 'yi izleyin.

İLK SATIR

`for (int p{a}; p != b; p = sonraki[p][b]) cout << p << " "; cout << b;`

çıktı BEKLENEN-CİKTİ · GİRDİ: 4 4 2 1 2 5 1 4 9 2 3 2 3 4 7 1 4 2 4

```
1 -> 4: 1 4 (uzaklık 9)
2 -> 4: 2 3 4 (uzaklık 9)
```

Dizin: `alistirma/kod/V-8-Z1/`

KEŞİF V-8-K1 Kim kime ulaşır?

Ağırlıkları atın, `bool` kalsın: `erisir[a][b] = erisir[a][b] or (erisir[a][o] and erisir[o][b])`. Yönlü çizgede erişilebilirlik matrisi (*geçişli kapanış*). Girdi kitapçığın yedi noktalı yönlü çizgesi; matrisi basın, sonra her nokta için kaç noktaya ulaştığını. n = 500'de n^3 = $1{,}25 \cdot 10^8$: kaba kurala göre bir saniye; deneyin.

DÜŞÜNCE

Uzaklık yerine evet/hayır; üç döngü aynı, işlem `or` ve `and`.

YAKLAŞIM

`erisir[a][b] = erisir[a][b] or (erisir[a][o] and erisir[o][b]);` — `o` en dışta. Sonra her satırın `true` sayısı.

İLK SATIR

```
for (int o{1}; o <= n; ++o) for (int a{1}; a <= n; ++a) for (int b{1}; b <= n; ++b) erisir[a][b] = erisir[a][b] or (erisir[a][o] and erisir[o][b]);
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 7 9 1 2 1 3 2 5 3 4 4 6 4 7 5 7 6 1 7 5

```
1 1 1 1 1 1 1
0 1 0 0 1 0 1
1 1 1 1 1 1 1
1 1 1 1 1 1 1
0 0 0 0 1 0 1
1 1 1 1 1 1 1
0 0 0 0 1 0 1
1: 7 noktaya ulaşır
2: 3 noktaya ulaşır
3: 7 noktaya ulaşır
4: 7 noktaya ulaşır
5: 2 noktaya ulaşır
6: 7 noktaya ulaşır
7: 2 noktaya ulaşır
```

Dizin: `alistirma/kod/V-8-K1/`

V.9

Bellman–Ford: eksi ağırlıklar da olabilir

n−1 tur gevşetme; n'inci turda hâlâ iyileşen varsa çevrim. Kitapçığın ödüllü tünelleri (en büyük skor), CSES'in *High Score* ve *Cycle Finding* soruları.

1 1 1 1

> **YANIT YERİNE**
>
> kitapçık V.5: Bellman–Ford algoritma ders 25: en iyi yol ve sonsuz çevrim algoritma ders 26: Bellman–Ford kanıtı

ISINMA V-9-I1 Kaç tur yetti?

Program kitapçığın skor sürümü: her tünelin ödülü/cezası, 1'den n'e en büyük skor, n−1 tur. Çoğu çizgede daha az tur yeter: bir turda hiçbir skor değişmediyse durun ve kaç tur attığınızı yazın. Girdi CSES'in *High Score* örneği (yanıt 5). Neden n−1 tur hep *yeterli*, ama çoğu zaman *gerekli* değil?

ana.cpp — BAŞLANGIÇ

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <stack>
#include <string>
#include <algorithm>
using namespace std;
using Nokta = int;
template <class T> using Dizi = vector<T>;

// V-9-I1 · Kaç tur yetti? Erken durun, tur sayısını yazın
using S = long long;
const S SONSUZ{(S)1e18};
struct Tunel { int a, b; S x; };

int main() {
  int n{0}, m{0};
  cin >> n >> m;
  Dizi<Tunel> tuneller(m);
  for (auto& t : tuneller) cin >> t.a >> t.b >> t.x;
  Dizi<S> skor(n + 1, -SONSUZ);
  skor[1] = 0;
  for (int k{1}; k < n; ++k)
    for (auto [a, b, x] : tuneller)
      if (skor[a] != -SONSUZ) skor[b] = max(skor[b], skor[a] + x);
  cout << "en büyük skor: " << skor[n] << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 5 1 2 3 2 4 -1 1 3 -2 3 4 7 1 4 4

```
en büyük skor: 5
2 turda durdu
```

Dizin: `alistirma/kod/V-9-I1/`

ALIŞTIRMA V-9-A1

Yüksek skor

CSES 1673 (*High Score*): 1'den n'e en büyük skor; sonsuza kadar artırılabiliyorsa `-1`. n'inci turda hâlâ iyileşen bir tünel varsa *o tünelin vardığı nokta* bir kârlı çevrimde ya da onun etkisinde. Ama dikkat: çevrim n'e ulaşmıyorsa sonsuz skor olmaz. Bu girdi (CSES örneği) çevrimsiz; yanıt 5. V-9-Z1 çevrimli hâlini verecek.

çıktı BEKLENEN-CİKTİ · GİRDİ: 4 5 1 2 3 2 4 -1 1 3 -2 3 4 7 1 4 4

```
5
```

Dizin: `alistirma/kod/V-9-A1/`

ZORLAYICI V-9-Z1

Çevrim n'e ulaşmıyorsa

İki girdi düşünün: birinde kârlı çevrim 1'den n'e giden yolun üstünde, ötekinde kenarda, n'e ulaşmayan bir çıkmazda. İkincide yanıt `-1` değil. V-9-A1'in çözümü bunu nasıl ayırıyor? Girdide iki çizge art arda (her biri `n m` ile başlıyor); programınız ikisi için de yanıt versin. Kârlı çevrim 2-3-2 ikisinde de var; farkı 3'ten n'e yol olup olmaması.

DÜŞÜNCE

Kârlı çevrim yalnız `n`'e ulaşıyorsa sonsuz skor demek.

YAKLAŞIM

n'inci turda iyileşen noktaları işaretleyin; işaretli bir noktadan `n`'e yol var mı diye (tersten ya da düz) gezin. Girdiyi `while (cin >> n >> m)` ile iki kere okuyun.

İLK SATIR

```
while (cin >> n >> m) { Dizi<Tunel> t(m); for (auto& x : t) cin >> x.a >> x.b >> x.x; cout << coz(n, t) << endl; }
```

çıktı BEKLENEN-CİKTİ · GİRDİ: 4 5 1 2 1 2 3 2 3 2 2 3 4 1 1 4 10 5 5 1 2 1 2 3 2 3 2 2 1 4 3 4 5 1

```
-1
4
```

Dizin: `alistirma/kod/V-9-Z1/`

KEŞİF V-9-K1 Çevrim bulma

CSES 1197 (*Cycle Finding*): çizgede toplam ağırlığı eksi bir çevrim var mı; varsa yazın. Bu sefer en *küçük* uzaklık (min), başlangıç noktası yok: bütün uzaklıkları 0'dan başlatın (sanki her noktaya sıfır maliyetli bir kaynak bağlı). n'inci turda hâlâ iyileşen bir nokta bulun; `nereden` zincirini n kere geri sarıp çevrimin içine girdiğinizden emin olun, sonra çevrimi okuyun. Girdi CSES örneği; beklenen çıktı bizim sürümünki, başka çevrim de doğru.

DÜŞÜNCE

Eksi çevrim: n'inci turda hâlâ iyileşen bir nokta var; nereden zincirini geri sarınca çevrimin içine düşersiniz.

YAKLAŞIM

Bütün uzaklıklar 0; n tur gevşetme; son turda iyileşen `x`'i n kere `nereden` ile geri sarın (artık çevrimdesiniz), sonra kendine dönene kadar toplayın ve ters çevirin.

İLK SATIR

```
Nokta p{x}; for (int k{0}; k < n; ++k) p = nereden[p]; Nokta bas{p}; do { cevrim.push_back(p); p = nereden[p]; } while (p != bas);
```

çıktı BEKLENEN-CİKTİ · GİRDİ: 4 5 1 2 1 2 4 1 3 1 1 4 1 -3 4 3 -2

```
YES
4 1 2 4
```

Dizin: `alistirma/kod/V-9-K1/`

V.10

Geri dönüş: eşlemle çizge, özyinelemeyle oda

Çizge kodunun altında hep eski tanıdıklar var: komşuluk listesi bir eşlem, derinlemesine gezi bir özyineleme. İki kısa soru, ikisi de o tanıdıkları adıyla çağırıyor.

2

YANIT YERİNE

kitapçık II.2: Eşlem kitapçık IV.1: Kendini çağıran işlev kitapçık V.1: Çizge nedir kitapçık V.3: Kaç oda var

ALIŞTIRMA V-10-A1

Dereceler, eşlemle

Girdi: bağ sayısı *m*, sonra *m* satırda yönsüz bağ (`A B`). Çizgeyi `map<string, set<string>>` olarak kurun: her düğüm komşularının kümesine. Çıktı: her düğüm ve derecesi (komşu sayısı) ada göre sıralı, son satırda en çok komşusu olan düğüm. Küme sayesinde aynı bağ iki kere gelse de derece şişmez; eşlem sayesinde düğümleri önceden saymanıza gerek yok.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 A B A C B C C D D E

```
A: 2
B: 2
C: 3
D: 2
E: 1
en çok komşu: C (3)
```

Dizin: `alistirma/kod/V-10-A1/`

ALIŞTIRMA V-10-A2

Odaları özyinelemeyle sayın

Kitapçığın “kaç oda var” sorusu, bu sefer gezi kalıbı yerine doğrudan IV. bölümün özyinelemesiyle: `doldur(s, k)` kareyi işaretlesin, dört komşusunu çağırsın ve doldurduğu kare sayısını döndürsün. Girdi: satır ve sütun sayısı, sonra harita (`.` boş, `#` duvar). Çıktı: oda sayısı ve en büyük odanın kare sayısı. Taban durumunu ilk satıra yazın: sınır dışı, duvar ya da zaten dolu.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 6 ..#..# ..#..# ###### #...#.

```
4 oda
en büyük oda: 4 kare
```

Dizin: `alistirma/kod/V-10-A2/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

“Bu bölümü geçtim mi?” sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` “Doğru!” desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

`V-4-Z1` Genel çizgede en kısa yol

`V-3-Z1` Yol ağı tasarımı

`V-6-Z2` En güvenli kare

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.

BÖLÜM VI

Dinamik Programlama

Kitapçığın son bölümü bir cümleyi reçeteye çeviriyor: büyük problemi küçük parçalara böl, her parçayı bir kere çöz. Burada altı konu, yirmi sekiz alıştırma: Fibonaççi'yi iki biçimde yazmak, ızgarada yol saymak, zar dizileri, bozuk paralar, kanıtın neden şart olduğu ve yola devam. Keşifler CSES'in ve AtCoder'ın DP bölümlerinden; yarım bırakılan `bul2()` burada da yarım, ama son Keşif ona tırmanıyor.

Hazırlık · I. İlk Adımlar · II. Veriyi Düzenlemek · III. Zanaat · IV. Özyineleme · V. Çizgeler · VI. Dinamik Programlama

VI.1

Aslında bunu zaten yaptınız

Bölüm IV'teki tek satırlık bellek dinamik programlamanın ta kendisiydi. Burada aynı Fibonaççi'yi iki biçimde yazıyoruz: yukarıdan aşağı (özyineleme + bellek) ve aşağıdan yukarı (çizelge). Kitapçığın önerdiği sıra: YA ile düşün, AY ile yaz.

1 2 1

> YANIT YERİNE
>
> kitapçık VI.1: Aslında bunu zaten yaptınız dp-sade: YA ve AY ders 7: memoizasyon kod: 30-fib.cpp

ISINMA VI-1-I1

Çizelgeyle Fibonaççi

Program Fibonaççi'yi Bölüm IV'ün yöntemiyle, yukarıdan aşağı ve bellekle hesaplıyor. `fibAY`'yi yazın: özyineleme yok, 0'dan n'e dolan bir çizelge (`Dizi<S>`). Üç n için iki sonuç yan yana çıksın; eşit olmalılar. Hangisi daha kısa? Hangisinin kanıtı tanımın kendisi?

ana.cpp — BAŞLANGIÇ

```
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <algorithm>
#include <cassert>
using namespace std;
using S = long long;
template <class T> using Dizi = vector<T>;

// VI-1-I1 · Çizelgeyle Fibonaççi
// Çıktı: "n=10: YA 55, AY 55" biçiminde üç satır (10, 50, 90)
map<int, S> bellek;
S fibYA(int n) {                // yukarıdan aşağı + bellek (Bölüm IV)
  if (n < 2) return n;
  if (bellek.contains(n)) return bellek[n];
  return bellek[n] = fibYA(n - 1) + fibYA(n - 2);
}

S fibAY(int n) {                // aşağıdan yukarı: çizelgeyi 0'dan n'e doldurun
  return 0;
}

int main() {
  for (int n : {10, 50, 90})
    cout << "n=" << n << ": YA " << fibYA(n) << ", AY " << fibAY(n) << endl;
  return 0;
}
```

çıktı — BEKLENEN-CİKTİ

```
n=10: YA 55, AY 55
n=50: YA 12586269025, AY 12586269025
n=90: YA 2880067194370816120, AY 2880067194370816120
```

Dizin: `alistirma/kod/VI-1-I1/`

ALIŞTIRMA VI-1-A1

İki değişken yeter

Çizelgenin her hücresi yalnız önceki iki hücreye bakıyor. Öyleyse çizelgeyi saklamayın: iki değişkenle (`a`, `b`) yazın, bellek sabit. n = 90 ve n = 92 için yazdırın. n = 93'ü neden yazdırmadık? (Bölüm I'in taşmasını hatırlayın: `long long`'un sınırı $9{,}22 \cdot 10^{18}$.)

çıktı — BEKLENEN-CİKTİ

```
fib(90) = 2880067194370816120
fib(92) = 7540113804746346429
```

Dizin: `alistirma/kod/VI-1-A1/`

ALIŞTIRMA VI-1-A2

Bellek olarak vector

Kitapçığın belleği `map<int, S>`; her bakış O(log n). n'i bildiğimize göre `Dizi<S> bellek(n + 1, -1)` de olur: O(1) bakış, “henüz yok” için `-1`. `fibVec`'i yazın, üç sürümün çağrı sayısını yazdırın. Girdi n = 30. Belleksizle bellekli arasındaki oran kaç?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 30

```
fib(30) = 832040
belleksiz: 2692537 çağrı
map ile  : 59 çağrı
vector ile: 59 çağrı
```

Dizin: `alistirma/kod/VI-1-A2/`

ZORLAYICI VI-1-Z1

Derin çağrı

Kitapçık uyarıyor: YA “derin çağrılarda yığın taşabilir”. Program n = 1 000 000 için YA + bellekle (mod 10^9+7) hesaplamaya çalışıyor. Çalıştırın: ne oluyor? (Terminalde `ulimit -s` yığının boyunu söyler.) Sonra aynı sonucu AY ile, iki değişkenle hesaplayın; o taşmaz. Çıktı tek satır. Bir yan soru: mod alırken `(a + b) % MOD` taşar mı?

DÜŞÜNCE

Bir milyon iç içe çağrı yığını taşırır; döngü taşırmaz.

YAKLAŞIM

Aşağıdan yukarı: `a = 0, b = 1`, n kere `(a, b) = (b, (a + b) % MOD)`. Mod toplamı $2 \cdot 10^9$'u geçmez, `long long`'da güvenli.

İLK SATIR

```
S a{0}, b{1}; for (int k{0}; k < n; ++k) { S c{(a + b) % MOD}; a = b; b = c; }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 1000000

```
fib(1000000) mod 1e9+7 = 918091266
```

Dizin: `alistirma/kod/VI-1-Z1/`

VI.2

Izgarada kaç yol var?

Project Euler 15: sağa ve aşağı giderek köşeden köşeye kaç yol? Kitapçığın `50-izgara.cpp`'si aynı soruyu dört kere çözüyor. Burada çizelgeyi görmek, dikdörtgen, engel, toplamak yerine en küçüğü seçmek ve yolu geri kurmak.

1 2 1 1

YANIT YERİNE

kitapçık VI.2: Izgarada kaç yol var ders 6: PE 15 algoritma ders 27: DP'ye giriş dp/README: patikalar ve engelli saha kod: 50-izgara.cpp

ISINMA VI-2-I1 Çizelgeyi görün

Kitapçığın “Deneyin”i: 3×3 çizelgeyi kâğıda çizin. Sonra makineye çizdirin: `yol3`'e `bool yazdir = false` girdisi ekleyin; doğruysa çizelgeyi satır satır yazsın (sayılar tek boşlukla). `main`'in başına `yol3(4, true)` koyun; dört eski satır olduğu gibi kalsın. Çizelgede tanıdık bir üçgen göreceksiniz; adı?

çıktı — BEKLENEN-CİKTİ

```
1 1 1 1 1
1 2 3 4 5
1 3 6 10 15
1 4 10 20 35
1 5 15 35 70
12x12 belleksiz : 2704156  (5408311 çağrı)
12x12 bellekli  : 2704156  (289 çağrı)
20x20 çizelgeyle: 137846528820
20x20 tek satır : 137846528820
```

Dizin: `alistirma/kod/VI-2-I1/`

ALIŞTIRMA VI-2-A1 Dikdörtgen ızgara

Kitapçığın ızgarası kare. r satır, c sütunu girdiden okuyun (dosya bitene kadar çiftler); kitapçığın *tek satır bellek* numarasıyla hesaplayın. Girdi: `2 2`, `3 4`, `10 10`, `20 20`. 20×20 kitapçığın sayısını vermeli. Sonra kâğıtta: 3×4'ün yanıtı 35, bu `C(7, 3)`; neden?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 2 2 3 4 10 10 20 20

```
2x2: 6
3x4: 35
10x10: 184756
20x20: 137846528820
```

Dizin: `alistirma/kod/VI-2-A1/`

ALIŞTIRMA VI-2-A2

Engelli ızgara

AtCoder DP H, kitapçığın son alıştırmalarından: bazı kareler duvar (`#`), yol sayısı mod 10^9+7. Kitapçık "tek satır değişecek" diyor: duvar karesine kaç yol gelir? Girdi AtCoder'ın örneği. Sol üst kareyi 1'le başlatıp çizelgeyi doldurun; dp/README'deki ders iki konumlamayı da anlatıyor.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 4 ...# .#.. ....

```
3
```

Dizin: `alistirma/kod/VI-2-A2/`

ZORLAYICI VI-2-Z1

Toplamak yerine en küçüğü

Project Euler 81: her karede bir sayı; sağa ve aşağı giderek sol üstten sağ alta en ucuz yol. Aynı çizelge, tek fark: iki komşudan geleni toplamak yerine küçüğünü alıp karenin sayısını ekliyorsunuz. Beş adımlık reçeteyi yazın: işlev ne, taban ne, formül ne? Girdi PE'nin 5×5 örneği; yanıt 2427 olmalı.

DÜŞÜNCE

Her karenin en ucuz yolu: üst ve soldan gelenin küçüğü artı karenin sayısı.

YAKLAŞIM

`t[0][0] = m[0][0]`; ilk satır ve sütun tek komşudan; gerisi `min(t[r-1][c], t[r][c-1]) + m[r][c]`. Yanıt `t[n-1][n-1]`.

İLK SATIR

```
t[r][c] = m[r][c] + min(r > 0 ? t[r-1][c] : SONSUZ, c > 0 ? t[r][c-1] : SONSUZ);
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 131 673 234 103 18 201 96 342 965 150 630 803 746 422 111 537 699 497 121 956 805 732 524 37 331

```
en ucuz yol: 2427
```

Dizin: `alistirma/kod/VI-2-Z1/`

KEŞİF VI-2-K1 Yolu geri kurmak

VI-2-Z1'in programı maliyeti veriyor, yolu değil. Bölüm V'teki “nereden geldim” numarası burada da işler: sağ alt kareden başlayın, üst ve sol komşudan hangisi `t[r][c] - m[r][c]`'ye eşitse oraya gidin, sol üste varınca ters çevirin. Karelerin sayılarını sırayla yazın; PE'nin sayfasındaki yolla aynı mı? Sonra: AtCoder DP H'yi gönderin; PE 81'in 80×80 dosyasını indirip çözün (*girdi dosyasını okuma* alıştırması da olur).

DÜŞÜNCE

Çizelge dolunca yolu sondan başa okuyun: hangi komşu bu kareye getirdi?

YAKLAŞIM

`(r, c)`'den geriye: `t[r][c] - m[r][c]` üst komşuya eşitse yukarı, yoksa sola; (0, 0)'da durun. Topladığınız sayıları ters çevirin.

İLK SATIR

```
while (r > 0 or c > 0) { if (r > 0 and t[r-1][c] == t[r][c] - m[r][c]) --r; else --c; yol.push_back(m[r][c]); }
```

çıktı

```
BEKLENEN-CİKTİ · GİRDİ: 5 131 673 234 103 18 201 96 342 965 150 630 803 746 422 111 537 699 497 121 956 805 732 524 37 331
```

```
yol: 131 201 96 342 746 422 121 37 331
toplam: 2427
```

Dizin: `alistirma/kod/VI-2-K1/`

VI.3

Beş adımlık reçete; zar dizileri

İşlev, taban, formül, kanıt, kod. Kitapçığın birinci sorusu: toplamı n olan kaç zar dizisi var (CSES DP 1, `52-zar.cpp`). Burada reçeteyi dört kez uyguluyorsunuz; her seferinde önce kâğıda, sonra koda.

1 2 1 1

YANIT YERİNE

kitapçık VI.3: Beş adımlık reçete dp-sade: zar dizileri algoritma ders 1-8: ilk DP soruları kod: 52-zar.cpp

ISINMA VI-3-I1 Dört yüzlü zar

Kitapçıkta altı yüzlü zarla ilk yedi değer ikinin kuvvetleri gibi gidiyor, f(7) = 63'te bozuluyor. Zarı dört yüzlü yapın (1–4) ve f(0)..f(10)'u yazdırın. Düzen nerede bozuluyor, neden orada? Formülü kâğıda yazın: f(n) = f(n−1) + … + f(n−4).

çıktı — BEKLENEN-CİKTİ

```
f(0) = 1
f(1) = 1
f(2) = 2
f(3) = 4
f(4) = 8
f(5) = 15
f(6) = 29
f(7) = 56
f(8) = 108
f(9) = 208
f(10) = 401
```

Dizin: `alistirma/kod/VI-3-I1/`

ALIŞTIRMA VI-3-A1 Zar kombinasyonları

CSES 1633 (*Dice Combinations*): n girdiden okunur (10^6'ya kadar), yanıt mod 10^9+7. Kitapçığın beş satırını girdili programa çevirin; girdi n = 1 000 000. Çıktı yalnız sayı. CSES'e gönderin. Çizelge bir milyon `long long`: kaç megabayt?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 1000000

```
874273980
```

Dizin: `alistirma/kod/VI-3-A1/`

ALIŞTIRMA VI-3-A2

Yukarıdan aşağı zar

Aynı soruyu YA yazın: özyinelemeli `f(n)`, bellek olarak `Dizi<S>` (−1: henüz yok), çağrı sayacı. Girdi n = 50. AY'yle aynı sonucu veriyor mu? Kaç çağrı yapıldı; n'e göre yaklaşık kaç katı? (Her değer bir kere hesaplanır, her hesap altı çağrı.)

çıktı — BEKLENEN-CİKTİ · GİRDİ: 50

```
AY: 660641036
YA: 660641036
çağrı: 301
```

Dizin: `alistirma/kod/VI-3-A2/`

ZORLAYICI VI-3-Z1

Tam k zar

Yeni soru, aynı reçete: toplamı n olan ve *tam k zarlı* dizi sayısı. İşlev iki girdili: f(n, k). Taban: f(0, 0) = 1, k = 0 ve n > 0 ise 0. Formül: son zar 1..6, kalanı k−1 zarla. Kanıt iki cümle. Çizelge iki boyutlu. Girdi `10 3`. Sağlama: k = 1..n üzerinden f(10, k)'leri toplayın; kitapçığın f(10)'una (492) eşit çıkmalı. Neden?

> **DÜŞÜNCE**
>
> İki boyutlu durum: kaç zar, hangi toplam; son zar 1..6 olabilir.

> **YAKLAŞIM**
>
> `f[j][x] = f[j-1][x-1] + … + f[j-1][x-6]` (x−z ≥ 0 olanlar); `f[0][0] = 1`. Sağlama: `j` üstünden toplam, kitapçığın tek boyutlu `f(10)`'u.

> **İLK SATIR**
>
> `for (int j{1}; j <= k; ++j) for (int x{1}; x <= n; ++x) for (int z{1}; z <= 6 and z <= x; ++z) f[j][x] += f[j-1][x-z];`

çıktı — BEKLENEN-CİKTİ · GİRDİ: 10 3

```
toplamı 10 olan 3 zarlı dizi: 27
k=1..10 toplamı: 492 (= f(10))
```

Dizin: `alistirma/kod/VI-3-Z1/`

KEŞİF VI-3-K1

Hepsini yazın

Saymak başka, görmek başka. Bölüm IV'ün özyinelemesiyle toplamı 4 olan *bütün* zar dizilerini satır satır yazın (küçükten büyüğe), sonra kaç tane olduğunu f(4)'le karşılaştırın. İki program aynı şeyi sayıyor; biri listeyi kurmadan sayıyor. n = 100 için listeyi yazmak neden imkânsız (f(100) kaç basamaklı, tahmin edin)? Sonra CSES 1633'ü gönderin.

> **DÜŞÜNCE**
>
> Saymak yerine kurmak: IV. bölümün geri dönüşlü aramasıyla her diziyi yazın.

> **YAKLAŞIM**
>
> `yaz(kalan)`: `kalan == 0` ise diziyi yazdır ve say; yoksa `z = 1..min(6, kalan)` için `dizi.push_back(z); yaz(kalan - z); dizi.pop_back();`.

> **İLK SATIR**
>
> `for (int z{1}; z <= 6 and z <= kalan; ++z) { dizi.push_back(z); yaz(kalan - z); dizi.pop_back(); }`

çıktı — BEKLENEN-CİKTİ

```
1 1 1 1
1 1 2
1 2 1
1 3
2 1 1
2 2
3 1
4
toplam: 8, f(4) = 8
```

Dizin: `alistirma/kod/VI-3-K1/`

VI.4

En az kaç bozuk para? Diziler mi, kümeler mi?

Kitapçığın ikinci ve üçüncü soruları (CSES DP 2, 3, 4): aceleci yöntem neden yanılır, iki döngünün yeri değişince neden 8 yerine 3 çıkar. `51-bozukpara.cpp` üç soruyu bir arada çözüyor; burada parçalara ayırıp girdili hâle getiriyorsunuz.

1 2 2 1

YANIT YERİNE

kitapçık VI.3: bozuk paralar algoritma ders 28: DP kanıtları algoritma ders 29: kanıttan koda dp-sade: bozuk para soruları kod: 51-bozukpara.cpp

ISINMA VI-4-I1

Aceleci yanılsın

Paraları `{1, 4, 6}`, hedefi 8 yapın. Aceleci 6 + 1 + 1 der (3 para); en iyi 4 + 4 (2 para). Programın ilk bloğu bunu hesaplar; `cout` satırlarındaki yazıları da (paralar, aceleci açıklaması) yeni duruma göre düzeltin. İkinci blok olduğu gibi kalsın. Hangi para kümelerinde aceleci hep doğru olur, aklınıza bir tane geliyor mu?

çıktı — BEKLENEN-CİKTİ

```
paralar: 1, 4, 6   hedef: 8
en az para sayısı : 2
  (aç gözlü çözüm 6+1+1 yani 3 para kullanırdı)

paralar: 2, 3, 5   hedef: 9
kaç farklı dizi : 8
kaç farklı küme : 3
```

Dizin: `alistirma/kod/VI-4-I1/`

ALIŞTIRMA VI-4-A1

En az para

CSES 1634 (*Minimizing Coins*): ilk satırda para sayısı n ve hedef x, ikinci satırda paralar. En az para sayısı; hedefe varılamıyorsa `-1`. Girdi CSES'in örneği. `SONSUZ` için `x + 1` yeter (ders 29'un numarası); neden? CSES'e gönderin.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 11 1 5 7

```
3
```

Dizin: `alistirma/kod/VI-4-A1/`

ALIŞTIRMA VI-4-A2

Para kombinasyonları I ve II

CSES 1635 (*Coin Combinations*) ve 1636 aynı programda: dizi sayısı ve küme sayısı, ikisi de mod 10^9+7. Girdi biçimi VI-4-A1'inki; CSES örneği `2 3 5` ile 9: dizi 8, küme 3. İki döngü, iki sıra; hangisi hangisi, ezberlemeden söyleyebilmelisiniz.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 9 2 3 5

```
dizi: 8
küme: 3
```

Dizin: `alistirma/kod/VI-4-A2/`

ZORLAYICI VI-4-Z1

Hangi paralar?

En az para sayısı 3; peki hangi üç para? İkinci bir dizi tutun: `son[t]`, t hedefine en iyi çözümde son konan para. `f[t]` iyileştiğinde onu da yazın; sonunda x'ten geriye `son`'u izleyin. Girdi CSES örneği. Birden çok en iyi çözüm olabilir; sizinki paraları hangi sırada denediğinize bağlı. Beklenen çıktı, paraları girdideki sırayla deneyip yalnız *katı* iyileşmede yazan sürümünki.

DÜŞÜNCE

Çizelge en az sayıyı verir; hangi paranın son konduğunu da yazarsanız geriye yürüyebilirsiniz.

YAKLAŞIM

`f[t]` güncellenirken `son[t] = p`; sonunda `t = x`'ten `t -= son[t]` ile sıfıra inene kadar paraları toplayın.

İLK SATIR

```
if (t - p >= 0 and f[t - p] + 1 < f[t]) { f[t] = f[t - p] + 1; son[t] = p; }
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 11 1 5 7

```
3 para: 1 5 5
```

Dizin: `alistirma/kod/VI-4-Z1/`

ZORLAYICI İKİLİ VI-4-Z2

Aceleci ne zaman doğru?

Gerçek paralar (1, 5, 10, 25) için aceleci yöntem her hedefte doğru; `{1, 5, 7}` için değil. Bunu makineye kanıtlatmayın, *sınatın*: 1'den 100'e her hedef için aceleci sayıyı ve DP sayısını hesaplayın, kaç hedefte ayrıştıklarını ve ilk ayrışan hedefi yazın. İki para kümesi için. Sonra düşünün: aceleciyi doğru kılan özellik ne? (Adı "kanonik para sistemi"; kâğıt için bir soru.)

DÜŞÜNCE

Aceleci: büyükten küçüğe sığdıkça al; DP: en az; ikisini 100 hedefte karşılaştırın.

YAKLAŞIM

`aceleci`: paraları büyükten küçüğe gezip `sayi += hedef / p; hedef %= p;`. `enAz`: kitapçığın çizelgesi. Döngüde ilk farkı ve fark sayısını tutun.

İLK SATIR

```
for (S hedef{1}; hedef <= 100; ++hedef) if (aceleci(hedef, paralar) != enAz(hedef, paralar)) { ++fark; if (ilk == 0) ilk = hedef; }
```

çıktı — BEKLENEN-ÇİKTİ

```
1 5 10 25: 0 fark
1 5 7: 26 fark, ilki hedef 10
```

Dizin: `alistirma/kod/VI-4-Z2/`

KEŞİF VI-4-K1

Takım işi, makineyle

~2 saat

Kitapçığın “Takım işi”: hedef 5, paralar {2, 3}; biriniz dizi kodunu, öteki küme kodunu kâğıtta işletsin. Sonra makine aynı şeyi yapsın: dizi sürümünde her hedef adımından sonra, küme sürümünde her para adımından sonra çizelgenin tamamını yazdırın. Kâğıttakiyle karşılaştırın; 5 için 2 ve 1 çıkmalı. Sonra CSES DP bölümünün ilk dördünü (1633–1636) gönderin; hepsinde yeşil görünce bu bölüm bitmiş sayılır.

DÜŞÜNCE

İki kodun her adımından sonra çizelgenin tamamı; kâğıttaki izle karşılaştırma.

YAKLAŞIM

Dizi sürümünde dış döngü hedef: her `x` bitince `yaz(d)`; küme sürümünde dış döngü para: her `p` bitince `yaz(k)`. Başlangıç `d[0] = k[0] = 1`.

İLK SATIR

```
for (int x{1}; x <= hedef; ++x) { for (S p : paralar) if (x - p >= 0) d[x] += d[x - p]; cout << "x=" << x << ": "; yaz(d); }
```

çıktı — BEKLENEN-ÇİKTİ

```
dizi (önce hedef, sonra para):
x=1: 1 0 0 0 0 0
x=2: 1 0 1 0 0 0
x=3: 1 0 1 1 0 0
x=4: 1 0 1 1 1 0
x=5: 1 0 1 1 1 2
küme (önce para, sonra hedef):
p=2: 1 0 1 0 1 0
p=3: 1 0 1 1 1 1
```

Dizin: `alistirma/kod/VI-4-K1/`

VI.5

Kanıt neden bu kadar önemli?

Yanlış bir formül küçük örneklerde doğru yanıt verebilir; test kanıtın yerini tutmaz. Burada yanlış formülün nerede bozulduğunu görmek, kanıtı makineyle *sağlamak*, Floyd–Warshall'ın döngü sırasını bozmak ve yarım bırakılan `bul2()`.

1 1 1 1

YANIT YERİNE

kitapçık VI.4: Kanıt neden bu kadar önemli algoritma ders 25-26: kanıtlar floyd-warshall: DP kanıtı dp/README: bul2 yarım

ISINMA VI-5-I1

Küçük örnekte doğru, aslında yanlış

Zar dizileri için biri şu formülü önerdi: g(n) = 2·g(n−1) (n ≥ 2), g(0) = g(1) = 1. n = 0..6'da kitapçığın f'siyle birebir aynı, yedi değer! Programa g'yi ekleyin, n = 0..10 için ikisini yan yana yazdırın ve ilk ayrıştıkları n'i bulun. Neden tam orada? (Kitapçığın “f(0) listeden düşüyor” cümlesi.) Altı testle geçen bir formülün yanlış olabileceğini bir daha unutmayın.

ana.cpp — BAŞLANGIÇ

```cpp
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <algorithm>
#include <cassert>
using namespace std;
using S = long long;
template <class T> using Dizi = vector<T>;

// VI-5-I1 · Küçük örnekte doğru, aslında yanlış
// Çıktı: "n doğru yanlış" başlığı, 11 satır, sonra "ilk ayrışma: n=?"
int main() {
  const int n{10};
  Dizi<S> f(n + 1, 0), g(n + 1, 0);
  f[0] = 1;
  for (int x{1}; x <= n; ++x)
    for (int zar{1}; zar <= 6; ++zar)
      if (x - zar >= 0) f[x] += f[x - zar];
  // g: g[0] = g[1] = 1, sonra bir öncekinin iki katı
  cout << "n doğru yanlış" << endl;
  for (int x{0}; x <= n; ++x) cout << x << " " << f[x] << " " << g[x] << endl;
  // ilk ayrışma
  return 0;
}
```

çıktı — BEKLENEN-ÇİKTİ

```
n doğru yanlış
0 1 1
1 1 1
2 2 2
3 4 4
4 8 8
5 16 16
6 32 32
7 63 64
8 125 128
9 248 256
10 492 512
ilk ayrışma: n=7
```

Dizin: `alistirma/kod/VI-5-I1/`

ALIŞTIRMA VI-5-A1

Kanıtı sağlayın

Kanıt kâğıtta; makine onu *sağlar*. Paralar {2, 3, 5}: dizi ve küme sayılarını iki yoldan hesaplayın. Biri kitapçığın DP'si; öteki Bölüm IV'ün kaba kuvveti: dizileri özyinelemeyle tek tek kurup sayın, kümeleri de paraları artmayan sırada kurarak. Hedef 1..12'de hepsi eşleşiyor mu? Eşleşmesi kanıt değil (neden?), ama bir yazım hatasını anında yakalar.

çıktı — BEKLENEN-CİKTİ

```
dizi: 12 hedefte eşleşti
küme: 12 hedefte eşleşti
```

Dizin: `alistirma/kod/VI-5-A1/`

ZORLAYICI İKİLİ VI-5-Z1

Döngü sırasını bozun

Kitapçık: “formülü gördüğünüzde k'nın en dış döngü olması gerektiği kendiliğinden anlaşılıyor”. Anlaşılmazsa ne olur? Floyd–Warshall'ı iki kere yazın: k en dışta ve k en içte. Girdi dört şehirlik yönlü bir çizge: zincir 2→3→4→1 (hepsi 1). İki sürümün 2→1 yanıtını yazın (ulaşılamıyorsa −1). Bozuk sürüm neden yanılıyor; kanıtın hangi cümlesini çiğniyor? (Şehir numaralarını değiştirince bozuk sürüm bazen doğru çıkar; neden?)

DÜŞÜNCE

`k` ara şehir; o en içteyken `f[a][b]` daha küçük `k`'lar için henüz güncellenmemiş olabilir.

YAKLAŞIM

İki işlev, döngü sırası tek fark: `(k, a, b)` ve `(a, b, k)`. 2→3→4→1 zincirinde bozuk sürüm 2→1'i bulmak için 2→3→4 ve 4→1'in önceden hazır olmasına muhtaç.

İLK SATIR

`for (int a{1}; a <= n; ++a) for (int b{1}; b <= n; ++b) for (int k{1}; k <= n; ++k) f[a][b] = min(f[a][b], f[a][k] + f[k][b]);  // bozuk`

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 3 2 3 1 3 4 1 4 1 1

```
k en dışta: 2->1 = 3
k en içte : 2->1 = -1
```

Dizin: `alistirma/kod/VI-5-Z1/`

KEŞİF VI-5-K1 bul2'yu bitirin

Kitapçığın (ve ders notlarının) yarım bıraktığı işlev: iki yazının en uzun ortak *altdizisi* (bitişik harfler). Yedi deneme hazır; başlangıç dosyası ilk denemede duruyor (`assert`). Eksik birkaç satır: `m` şimdiye kadarkinden uzunsa `enUzun`'u ve bittiği yeri güncelleyin. Hepsi geçince "7 deneme geçti". Devamı: aynı işi YA (özyineleme + bellek) ile yazın; sonra CSES 3403'e bakın: orada "altdizi" bitişik *değil*, formül değişir. Nasıl?

DÜŞÜNCE

`tane[k1+1][k2+1]` bu karede biten ortak parçanın boyu; en uzununu ve bittiği yeri hatırlayın.

YAKLAŞIM

`if (m > enUzun) { enUzun = m; k = k1 + 1; }`; sonunda `y1.substr(k - enUzun, enUzun)`. Yedi denemeyi birer birer geçirin.

İLK SATIR

```
if (m > enUzun) { enUzun = m; k = k1 + 1; }
```

çıktı — BEKLENEN-CİKTİ

```
7 deneme geçti
```

Dizin: `alistirma/kod/VI-5-K1/`

VI.6

Yola devam

Kitapçık altı bölümü "haftada üç soru, kitabı okuyun, yarışmalara girin, bir şey inşa edin" diye kapatıyor. Burada dördü de var: AtCoder'ın ilk DP sorusu, ders 18'in sınav sorusu, yaşam oyunu ve sıradaki CSES soruları.

1 1 2

YANIT YERİNE

kitapçık: Son söz, Yola devam yaşam oyunu ders 18: sınav soruları algoritma ders 30 Rekabetçi Programlama El Kitabı (TR)

ALIŞTIRMA VI-6-A1

Zıplayan kurbağa

AtCoder DP A, kitapçığın önerdiği klasik: n taş, i'nci taşın yüksekliği h_i; kurbağa 1'den n'e 1 ya da 2 taş atlayarak gider, her atlayışın bedeli $|h_i - h_j|$. En ucuz bedel. Beş adım: işlev f(i), taban f(1) = 0, formül iki seçenekten küçüğü. Girdi AtCoder'ın üçüncü örneği (yanıt 40). Gönderin.

çıktı — BEKLENEN-CİKTİ · GİRDİ: 6 30 10 60 10 60 50

```
40
```

Dizin: `alistirma/kod/VI-6-A1/`

ZORLAYICI VI-6-Z1

Başı sonu aynı altdizi

Ders 18'in sınav sorusu: `{4, 2, 3, 1, 3, 2, 1}` dizisinde başı ve sonu aynı olan altdiziler var; toplamı en büyük olanınki 11 (`2 3 1 3 2`). Verilen dizi için bu toplamı bulan `altToplam`'ı yazın. Kaba kuvvet O(n²) kolay; dersin ipucu `std::map`: her değerin ilk göründüğü konumu tutun, ön toplamlarla (*prefix sum*) O(n log n). İkisini de yazın, aynı çıksın. Girdi: n ve dizi.

DÜŞÜNCE

Başı ve sonu aynı değer: ilk görüldüğü konumla şimdiki konum arasındaki toplam.

YAKLAŞIM

Ön toplam `on[i] = d[0] + … + d[i-1]`; `map<S, int> ilk` değerin ilk konumu; her `j` için değer daha önce `i`'de görüldüyse aday `on[j+1] - on[i]`.

İLK SATIR

```
if (ilk.contains(d[j])) enIyi = max(enIyi, on[j + 1] - on[ilk[d[j]]]); else ilk[d[j]] = j;
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 7 4 2 3 1 3 2 1

```
kaba: 11
eşlemle: 11
```

Dizin: `alistirma/kod/VI-6-Z1/`

KEŞİF VI-6-K1

Yaşam oyunu

~bir hafta

Kitapçığın “bir şey inşa edin” önerisi için en keyifli başlangıç: Conway'in yaşam oyunu (dört kural, ders notunda). Girdi: satır, sütun, nesil sayısı ve tahta (`#` canlı, `.` cansız); tahtanın dışı cansız. O kadar nesil ilerletip tahtayı yazın. Girdideki desen bir *planör*: dört nesilde bir sağ alta kayar. Sonra tahtayı büyütün, nesilleri terminalde art arda yazdırın (`sleep` ile), kendi desenlerinizi deneyin; periyodik desenlerle ilgili yazı ders notunda.

DÜŞÜNCE

Her nesil, her hücre: sekiz komşudan kaçı canlı; dört kural yeni tahtayı yazar.

YAKLAŞIM

Yeni tahtayı ayrı bir `vector<string>`'e kurun (yerinde değiştirmeyin); komşu sayımı için 8 yönlü döngü ve sınır denetimi; sonunda `tahta = yeni`.

İLK SATIR

```
int canli{komsuSay(r, c)}; yeni[r][c] = (tahta[r][c] == '#') ? ((canli == 2 or canli == 3) ? '#' : '.') : (canli == 3 ? '#' : '.');
```

çıktı — BEKLENEN-CİKTİ · GİRDİ: 5 5 4 ....#.. ...#. .### .....

```
.....
.....
...#.
....#
..###
```

Dizin: `alistirma/kod/VI-6-K1/`

KEŞİF VI-6-K2

Haftada üç soru

her hafta

Kitapçığın programı: CSES Problem Set'i sırayla, haftada üç soru. DP bölümünün sıradaki dördü: Removing Digits, Grid Paths, Book Shop, Array Description. Her biri için önce beş adımlık reçeteyi kâğıda yazın (işlev, taban, formül, kanıt), sonra kodlayın; bu dizin sizin. Ve kitapçığın "en önemlisi": öğrendiğiniz bir konuyu iki sayfada bir arkadaşınıza anlatın. Koco cildi de sizi bekliyor: aynı reçete, çizen bir dil.

DÜŞÜNCE

Dört soru, dört reçete; kod reçeteden sonra.

YAKLAŞIM

Her soru için önce kâğıda: işlev ne (durum), taban, formül, kanıt; sonra kitapçığın çizelge kalıbı. Takıldığınızda CSES'in kitabındaki ilgili bölüm (depoda Türkçesi).

İLK SATIR

`// f(x): ... (bir cümle); taban: ...; formül: ...` yorumuyla başlayın.

Dizin: `alistirma/kod/VI-6-K2/` — beklenen çıktı yok

VI.7

Geri dönüş: çizelgeyi eşlemle, ızgarayı iki yolla

Dinamik programlamanın çizelgesi bir dizi olmak zorunda değil; II. bölümün eşlemi de olur. Ve bir ızgara problemi hem V. bölümün gezisiyle hem bu bölümün çizelgesiyle çözülebilir: ikisi aynı soruyu mu yanıtlıyor?

2

YANIT YERİNE

kitapçık II.2: Eşlem kitapçık V.3: Labirentte en kısa yol kitapçık VI.1: Izgarada kaç yol var

ALIŞTIRMA VI-7-A1

Çizelgeyi eşlemle tutun

Izgarada yol sayma, yukarıdan aşağı ve bellek olarak bir `map<pair<int,int>, long long>`: `yol(s, k)` *s*×*k* ızgarada sol üstten sağ alta (yalnız sağ ve aşağı) kaç yol olduğunu versin; taban `s == 1 || k == 1`. Girdi: kaç soru, sonra her satırda *s k*. Çıktı: her soru için `5x5: 70 yol`, son satırda eşlemde kaç kayıt biriktiği. Aynı eşlemi üç soruda da kullanın; 16×16'dan sonra kayıt sayısı neden 256 değil de 225?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 3 2 2 5 5 16 16

```
2x2: 2 yol
5x5: 70 yol
16x16: 155117520 yol
bellekte 225 kayıt
```

Dizin: `alistirma/kod/VI-7-A1/`

ALIŞTIRMA VI-7-A2

Engelli ızgara, iki yolla

Girdi: satır ve sütun sayısı, sonra ızgara (`S` başlangıç sol üstte, `C` çıkış sağ altta, `#` engel). İki soru, iki yöntem: V. bölümün enlemesine gezisiyle dört yöne yürüyerek en az kaç adımda çıkışa varılır; VI. bölümün çizelgesiyle yalnız sağa ve aşağı giderek kaç farklı yol var (engel kareye 0). Çıktı: `en kısa yol: 7 adım` ve `sağ-aşağı yol sayısı: 4`. İki yanıt birbirinden bağımsız; neden?

çıktı — BEKLENEN-CİKTİ · GİRDİ: 4 5 S..#. .#... ...#. #...C

```
en kısa yol: 7 adım
sağ-aşağı yol sayısı: 4
```

Dizin: `alistirma/kod/VI-7-A2/`

SINAV

BÖLÜM SINAVI · 30 DAKİKA

"Bu bölümü geçtim mi?" sorusunun yanıtı: saati kurun, üç soruyu kitapçığa ve ipuçlarına bakmadan çözün, her birinde `make test` "Doğru!" desin. Üçü de otuz dakikada geçiyorsa bölüm sizin; ikisi geçiyorsa bir hafta sonra yeniden; biri geçiyorsa bölümün alıştırmalarına dönün.

`VI-3-Z1` Tam k zar

`VI-4-Z1` Hangi paralar?

`VI-2-Z1` Toplamak yerine en küçüğü

Keşif soruları için ölçüt, öğretmen de aynısını kullanır: *çalışıyor* (bir girdiyle gösterin), *okunaklı* (adlar anlaşılır, tekrar yok), *test var* (en az bir `assert` ya da altın dosya). Üçü de evetse tam puan.